

**PEMANTAUAN SINYAL JANTUNG PASIEN
MENGUNAKAN SENSOR
ELEKTROKARDIOGRAM BERBASIS LORAWAN
DAN KOMPRESI DATA**

***ELECTROCARDIOGRAM (ECG)-BASED PATIENT HEART
SIGNAL MONITORING SYSTEM USING LORAWAN AND
DATA COMPRESSION***

TUGAS AKHIR

Disusun sebagai syarat kuliah Tugas Akhir

Program Studi S1 Teknik Elektro

Disusun oleh:

MAYCO IKHSAN HANAFI

1102213061



**Universitas
Telkom**

FAKULTAS TEKNIK ELEKTRO

UNIVERSITAS TELKOM

BANDUNG

2025

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Tugas Akhir yang berjudul "Pemantauan Sinyal Jantung Pasien Menggunakan Sensor Elektrokardiogram Berbasis LoRaWAN dan Kompresi Data". Tugas Akhir ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Teknik (S.T.) pada Program Studi S1 Teknik Elektro, Fakultas Teknik Elektro, Universitas Telkom.

Penulis menyadari sepenuhnya bahwa karya ini masih memiliki keterbatasan. Oleh karena itu, saran dan kritik konstruktif dari pembaca sangat diharapkan untuk penyempurnaan di masa mendatang. Penulis berharap laporan ini dapat memberikan kontribusi yang berarti dalam pengembangan teknologi pemantauan kesehatan berbasis Internet of Things (IoT), serta menjadi referensi bagi penelitian lebih lanjut di bidang yang sama.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah mendukung penyelesaian tugas akhir ini. Ucapan terima kasih secara khusus disampaikan pada halaman berikutnya.

Bandung, 16 Juli 2025

Mayco Ikhsan Hanafi

UCAPAN TERIMA KASIH

Dengan penuh rasa hormat dan syukur, penulis menyampaikan terima kasih yang sebesar-besarnya kepada pihak-pihak yang telah memberikan dukungan moral, ilmiah, dan teknis selama proses penyusunan Tugas Akhir ini, antara lain:

1. Allah SWT, atas segala limpahan rahmat, kesehatan, dan kekuatan yang diberikan kepada penulis selama menyelesaikan Tugas Akhir ini.
2. Orang tua dan keluarga tercinta, atas doa, dukungan, serta kasih sayang yang tiada henti mengiringi setiap langkah penulis.
3. Bapak Dr. Junartha Halomoan, S.T., M.T. selaku Dosen Pembimbing I, dan Bapak Ir. Porman Pangaribuan, M.T. selaku Dosen Pembimbing II, yang telah memberikan bimbingan, arahan, serta masukan yang sangat berarti dalam pelaksanaan penelitian dan penyusunan laporan ini.
4. Bapak Irwan Purnama, S.Si., M.Sc.Eng., Ph.D., selaku Dosen Wali, yang telah membimbing dan membina penulis selama masa perkuliahan.
5. Syifana Wulan Suci dan Ade Surya Saputra, selaku rekan seperjuangan yang telah banyak membantu, memberikan masukan, serta menjadi teman diskusi selama proses penyusunan Tugas Akhir ini.
6. Teman-teman Kos PA, yang telah memberikan semangat, kebersamaan, dan lingkungan yang mendukung selama masa studi hingga penyusunan Tugas Akhir ini.
7. Teman-teman seangkatan, rekan satu laboratorium, serta seluruh pihak yang tidak dapat disebutkan satu per satu, atas kerja sama, dukungan, dan kebersamaan selama proses pengerjaan Tugas Akhir ini.

ABSTRAK

Pemantauan sinyal elektrokardiogram (ECG) secara *real-time* sangat krusial untuk deteksi dini kondisi jantung. Namun, transmisi data ECG melalui jaringan nirkabel seperti LoRaWAN menghadapi tantangan signifikan terkait keterbatasan *bandwidth*, meskipun teknologi ini unggul dalam efisiensi daya. Penelitian ini berfokus pada perancangan dan implementasi sistem pemantauan ECG yang inovatif, berbasis ESP32 dan Raspberry Pi Zero, dengan integrasi sensor AD8232 dan modul LoRa SX1276. Sistem ini dirancang untuk mengatasi keterbatasan *bandwidth* melalui pendekatan kompresi data yang cerdas.

Untuk mengurangi ukuran data secara drastis sebelum transmisi, sinyal ECG dikompresi menggunakan kombinasi metode *Discrete Wavelet Transform* (DWT) dan algoritma *Zlib*. Proses kalibrasi *heart rate* (HR) juga diterapkan untuk meningkatkan akurasi pembacaan dibandingkan dengan alat referensi. Hasil pengujian menunjukkan efektivitas metode kompresi, di mana ukuran data berhasil dikurangi hingga lebih dari 70% (dari 15.000 *byte* menjadi sekitar 3.840 *byte*). Kualitas rekonstruksi sinyal tetap terjaga dengan sangat baik, dibuktikan oleh *Percentage Root Mean Square Difference* (PRD) sebesar 2,98% dan *Correlation Coefficient* (CC) sebesar 0,9996.

Sistem ini tidak hanya berhasil dalam kompresi data, tetapi juga menunjukkan efisiensi tinggi dalam pengiriman data melalui jaringan LoRaWAN, mengurangi waktu transmisi hingga lebih dari 50%. Keberhasilan ini menjadikan sistem sangat potensial untuk aplikasi pemantauan pasien, baik di lingkungan yang membutuhkan jangkauan luas dan konsumsi daya rendah, maupun untuk penggunaan mandiri di rumah. Dengan demikian, penelitian ini berkontribusi pada pengembangan solusi pemantauan kesehatan jarak jauh yang lebih efisien dan andal.

Kata kunci: ECG, AD8232, LoRaWAN, Kompresi DWT, Zlib, PRD, Raspberry Pi Zero, ESP32

ABSTRACT

Real-time monitoring of electrocardiogram (ECG) signals is crucial for early detection of heart conditions. However, transmitting ECG data over wireless networks such as LoRaWAN faces significant challenges related to bandwidth limitations, even though this technology excels in energy efficiency. This research focuses on the design and implementation of an innovative ECG monitoring system based on ESP32 and Raspberry Pi Zero, integrating the AD8232 sensor and LoRa SX1276 module. The system is designed to address bandwidth limitations through an intelligent data compression approach.

To drastically reduce data size before transmission, ECG signals are compressed using a combination of Discrete Wavelet Transform (DWT) and the Zlib algorithm. A heart rate (HR) calibration process is also applied to enhance reading accuracy compared to reference devices. The test results demonstrate the effectiveness of the compression method, where the data size was successfully reduced by more than 70% (from 15,000 bytes to approximately 3,840 bytes). The quality of the reconstructed signal remains very high, evidenced by a percentage root mean square difference (PRD) of 2.98% and a correlation coefficient (CC) of 0.9996.

The system not only succeeded in data compression but also showed high efficiency in data transmission over the LoRaWAN network, reducing transmission time by more than 50%. This success makes the system highly potential for patient monitoring applications, both in environments requiring wide coverage and low power consumption, as well as for standalone use at homes. Thus, this research contributes to the development of more efficient and reliable remote health monitoring solutions.

Keywords: ECG, AD8232, LoRaWAN, DWT Compression, Zlib, PRD, Raspberry Pi Zero, ESP32

DAFTAR ISI

KATA PENGANTAR	i
UCAPAN TERIMA KASIH.....	ii
ABSTRAK	iii
ABSTRACT.....	iv
DAFTAR ISI.....	v
DAFTAR GAMBAR	viii
DAFTAR TABEL.....	xi
DAFTAR SINGKATAN	xii
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan.....	3
1.4 Manfaat Hasil Penelitian.....	3
1.5 Batasan Masalah	3
1.6 Metode Penelitian	4
BAB II TINJAUAN PUSTAKA.....	6
2.1 Elektrokardiogram (ECG)	6
2.1.1 Sinyal ECG, dan <i>Heart Rate</i> (HR).....	6
2.1.2 Algoritma Pan-Tompkins.....	7
2.1.3 Sensor ECG AD8232	7
2.2 Kompresi Data ECG	8
2.2.1 Teknik Kompresi Data ECG	9
2.2.2 <i>Discrete Wavelet Transform</i> (DWT).....	10
2.2.3 <i>Quantization</i> (Kuantisasi)	10
2.2.4 Zlib (LZ77).....	11

2.3	<i>Long Range Wide Area Network (LoRaWAN)</i>	11
2.3.1	Arsitektur Jaringan LoRaWAN.....	11
2.3.2	Lapisan Protokol LoRaWAN	12
2.3.3	Keunggulan LoRaWAN untuk Transmisi ECG.....	13
2.3.4	Studi Kasus Pemanfaatan LoRa dalam IoT Kesehatan.....	13
2.4	<i>Parameter</i> Kualitas Transmisi.....	13
2.4.1	Received Signal Strength Indicator (RSSI).....	14
2.4.2	<i>Signal-to-Noise</i> Ratio (SNR)	14
2.4.3	Interpretasi dan Peran dalam LoRaWAN	15
BAB III PERANCANGAN SISTEM		16
3.1	Desain Sistem	16
3.1.1	Diagram Blok.....	16
3.1.2	Fungsi dan Fitur	18
3.2	Design Perangkat Keras.....	18
3.2.1	Spesifikasi Komponen	21
3.3	Desain Perangkat Lunak.....	29
BAB IV HASIL PERCOBAAN DAN ANALISIS		34
4.1	Pengujian Sensor ECG	34
4.1.1	Skenario Pengujian Sensor ECG	34
4.1.2	Hasil Pengujian Sensor ECG Sebelum Kalibrasi	39
4.1.3	Hasil Pengujian Sensor ECG sesudah kalibrasi.....	44
4.1.4	Analisis Pengujian sensor	47
4.2	Pengujian Kompresi Data ECG	49
4.2.1	Skenario Pengujian Kompresi Data ECG.....	49
4.2.2	Hasil Pengujian Kompresi Data ECG	50
4.2.3	Analisis Pengujian Kompresi Data ECG.....	52

4.3	Pengujian Pengiriman Data melalui LoRaWAN	54
4.3.1	Skenario Pengujian Pengiriman Data ECG.....	54
4.3.2	Hasil Pengiriman Data Melalui LoRaWAN tanpa kompresi	55
4.3.3	Hasil Pengiriman Data Melalui LoRaWAN dengan kompresi	56
4.3.4	Validasi Hasil Data ECG dari Transmisi LoRaWAN.....	57
4.3.5	Analisis Hasil Transmisi Melalui LoRaWAN	58
4.3.6	Analisa Lambatnya Proses Transmisi Data.....	60
4.4	Estimasi Konsumsi Daya dan Waktu Operasi	61
4.5	Hasil Implementasi Sistem	62
BAB V KESIMPULAN DAN SARAN.....		67
5.1	Kesimpulan.....	67
5.2	Saran	68
DAFTAR PUSTAKA		69
LAMPIRAN <i>SOURCE CODE</i>		72

DAFTAR GAMBAR

Gambar 2.1 Gelombang EKG dan komponen-komponennya [2].....	6
Gambar 2.2 Blok diagram fungsional AD8232	8
Gambar 3.1 Diagram blok sistem	16
Gambar 3.2 Ilustrasi komunikasi jaringan sistem.....	17
Gambar 3.3 Rangkaian pengirim data sinyal ECG	19
Gambar 3.4 SX1303 LoRa gateway shield untuk Raspberry Pi 5	19
Gambar 3.5 Dari atas ke bawah, Desain 3D kotak komponen tampak atas dan tampak samping	20
Gambar 3.6 Sensor AD8232	21
Gambar 3.7 ESP32.....	22
Gambar 3. 8 Raspberry Pi Zero	23
Gambar 3.9 Modul LoRa SX1276	24
Gambar 3.10 Modul LoRa gateway SX1303.....	25
Gambar 3.11 Mikrokomputer Raspberry Pi 5.....	26
Gambar 3.12 Baterai 18650	28
Gambar 3.13 Modul pengisian daya IP5306.....	28
Gambar 3.14 Flowchart End-Node	30
Gambar 3.15 Flowchart MQTT gateway	32
Gambar 4.1 Penempatan elektroda pada tubuh.....	35
Gambar 4.2 Perekaman data HR dari Pulse Oximeter.....	35
Gambar 4.3 Sinyal ECG subjek 1 kondisi 1	36
Gambar 4.4 Cuplikan sinyal ECG selama 10 detik subjek 1 kondisi 1	36
Gambar 4.5 Sinyal ECG subjek 1 kondisi 2	36
Gambar 4.6 Cuplikan sinyal ECG selama 10 detik subjek 1 kondisi 2	36
Gambar 4.7 sinyal ECG subjek 2 kondisi 1	36
Gambar 4.8 Cuplikan sinyal ECG selama 10 detik subjek 2 kondisi 1	37
Gambar 4.9 sinyal ECG subjek 2 kondisi 2	37
Gambar 4.10 Cuplikan sinyal ECG selama 10 detik subjek 2 kondisi 2	37
Gambar 4.11 sinyal ECG subjek 3 kondisi 1	37
Gambar 4.12 Cuplikan sinyal ECG selama 10 detik subjek 3 kondisi 1	37
Gambar 4.13 sinyal ECG subjek 3 kondisi 2	38

Gambar 4.14 Cuplikan sinyal ECG selama 10 detik subjek 3 kondisi 2	38
Gambar 4.15 sinyal ECG subjek 4 kondisi 1	38
Gambar 4.16 Cuplikan sinyal ECG selama 10 detik subjek 4 kondisi 2	38
Gambar 4.17 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 1 kondisi 1	41
Gambar 4.18 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 1 kondisi 2	41
Gambar 4.19 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 2 kondisi 1	41
Gambar 4.20 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 2 kondisi 2	42
Gambar 4.21 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 3 kondisi 1	42
Gambar 4.22 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 3 kondisi 2	42
Gambar 4.23 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 4 kondisi 1	43
Gambar 4.24 Perbandingan nilai HR Pulse Oximeter dengan HR dari sensor pada subjek 4 kondisi 2	43
Gambar 4.25 Grafik regresi linear dari data yang telah didapatkan	45
Gambar 4.26 Hasil pengujian HR dengan menggunakan regresi linear	45
Gambar 4.27 Kompresi data pada sistem dengan Raspberry Pi 0	50
Gambar 4.28 Hasil dekompresi pada sinyal ECG pada data subjek 1	51
Gambar 4. 29 Data ECG setelah kompresi dengan deteksi R-Peak.....	51
Gambar 4.30 Perbandingan hasil perhitungan HR pada data asli dengan data hasil dekompresi pada subjek 1.....	52
Gambar 4.31 Ilustrasi Tempat Pengujian Sistem.....	54
Gambar 4.32 Sinyal asli ECG dari ESP32.....	57
Gambar 4.33 Sinyal setelah difilter dan dikompres di Raspberry Pi Zero.....	57
Gambar 4.34 Sinyal ECG pada Raspberry Pi 5 yang diterima dari hasil transmisi LoRaWAN.....	58
Gambar 4.35 Tampilan dashboard web pemantauan	63

Gambar 4.36 Tampilan Detail Pasien (a) bagian atas (b) bagian bawah	64
Gambar 4.37 Pop-up peringatan HR tidak normal pasien	64
Gambar 4.38 Form Input/Edit Pasien	65
Gambar 4.39 Tampilan Log Data.....	65

DAFTAR TABEL

Tabel 2.1 Perbandingan metode kompresi ECG	9
Tabel 2.2 Keunggulan transmisi LoRaWAN	13
Tabel 2.3 Studi kasus tantangan penggunaan LoRa.....	13
Tabel 2. 4 Interpretasi RSSI dan SNR pada LoRaWAN	15
Tabel 3.1 Perbandingan spesifikasi sensor ECG.....	21
Tabel 3.2 Perbandingan spesifikasi mikrokontroler.....	22
Tabel 3. 3 Perbandingan spesifikasi alat kompresi	23
Tabel 3.4 Perbandingan spesifikasi LoRa Transceiver	24
Tabel 3.5 Perbandingan spesifikasi LoRa gateway.....	25
Tabel 3.6 Perbandingan spesifikasi mikrokomputer	26
Tabel 3.7 Tabel perbandingan spesifikasi baterai	27
Tabel 3.8 Perbandingan spesifikasi modul pengisian daya.....	28
Tabel 4.1 Data subjek untuk pengambilan data	34
Tabel 4.2 Nilai HR sebelum kalibrasi	40
Tabel 4.3 Hasil pengujian perhitungan HR manual	47
Tabel 4.4 Tabel metrik evaluasi	48
Tabel 4.5 Tabel performa metode kompresi yang digunakan.....	50
Tabel 4.6 Tabel catatan waktu transmisi LoRaWAN tanpa kompresi.....	55
Tabel 4.7 Tabel catatan waktu transmisi LoRaWAN dengan kompresi	56
Tabel 4.8 Pengujian nilai tegangan baterai	61
Tabel 4.9 Struktur Tabel patients	66
Tabel 4.10 Struktur Tabel heart_rate	66
Tabel 4.11 Struktur Tabel ecg_data	66

DAFTAR SINGKATAN

- ADC: *Analog-to-Digital Converter*
- BER: *Bit Error Rate*
- BLE: *Bluetooth Low Energy*
- BPM: *Beats Per Minute*
- CC: *Correlation Coefficient*
- CPU: *Central Processing Unit*
- CSS: *Chirp Spread Spectrum*
- DCT: *Discrete Cosine Transform*
- DWT: *Discrete Wavelet Transform*
- ECG: *Electrocardiogram*
- HR: *Heart Rate*
- IoT: *Internet of Things*
- LoRa: *Long Range*
- LoRaWAN: *Long Range Wide Area Network*
- LZMA: *Lempel-Ziv-Markov chain Algorithm*
- MAE: *Mean Absolute Error*
- MQTT: *Message Queuing Telemetry Transport*
- NiMH: *Nickel-Metal Hydride*
- PRD: *Percentage Root Mean Square Difference*
- RMSE: *Root Mean Square Error*
- RLE: *Run-Length Encoding*
- RSSI: *Received Signal Strength Indicator*
- SNR: *Signal-to-Noise Ratio*
- TTN: *The Things Network*
- UART: *Universal Asynchronous Receiver-Transmitter*

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Kesehatan kardiovaskular merupakan aspek krusial dalam sistem pelayanan medis modern, mengingat tingginya prevalensi penyakit jantung di seluruh dunia. Electrocardiogram (ECG) adalah salah satu metode non-invasif paling umum untuk mendeteksi dan memantau aktivitas listrik jantung secara real-time, serta digunakan dalam diagnosis aritmia, iskemia, dan gangguan konduksi lainnya [1], [2].

Penerapan Internet of Things (IoT) dalam bidang kesehatan telah memungkinkan pengembangan sistem pemantauan jarak jauh yang hemat daya dan berbiaya rendah. Sensor bioelektrik seperti AD8232 banyak digunakan karena konsumsi dayanya yang rendah dan kemampuannya untuk mendeteksi sinyal ECG dengan baik dalam aplikasi wearable atau portable [3], [4].

Untuk mengekstrak informasi penting dari sinyal ECG, digunakan algoritma deteksi R-peak seperti Pan-Tompkins yang terbukti efektif dalam mendeteksi interval R-R dan menghitung *Heart Rate* (HR) secara akurat [5]. Namun, transmisi data ECG mentah secara langsung memerlukan bandwidth besar dan meningkatkan konsumsi daya transmisi. Oleh karena itu, teknik kompresi seperti Discrete Wavelet Transform (DWT), Discrete Cosine Transform (DCT), dan algoritma lossless seperti Zlib, LZ4, dan LZMA digunakan untuk mengurangi beban data sebelum dikirimkan [6], [7].

Sejumlah penelitian sebelumnya telah dilakukan dalam bidang ini. Katzis et al. [7] menunjukkan bagaimana DWT dan LZ4 dapat digunakan untuk memperkecil ukuran data ECG secara signifikan untuk transmisi melalui LoRaWAN. Penelitian ini berhasil mengurangi *delay* dan memelihara kualitas sinyal, namun belum mengimplementasikan algoritma deteksi detak jantung secara real-time. Kumar et al. [6] melakukan kajian mendalam tentang teknik compressive sensing dan menyarankan integrasi metode kompresi dengan deep learning untuk efisiensi tinggi. Meski demikian, pendekatan tersebut masih relatif kompleks untuk diterapkan pada perangkat IoT bertenaga rendah.

Sümbül [3] mengembangkan sistem pengiriman sinyal ECG secara mobile menggunakan ESP8266, namun belum mencakup optimasi kompresi data maupun protokol transmisi LoRaWAN. Sementara itu, Muñiz et al. [4] membangun sistem pemantauan pasien dengan AD8232 dan LoRaWAN, tetapi belum membahas performa sinyal terkait SNR dan RSSI secara menyeluruh. Penelitian klasik oleh Pan dan Tompkins [5] masih menjadi rujukan utama dalam hal deteksi R-peak, meskipun beberapa studi terkini mulai mengusulkan varian algoritmik berbasis machine learning untuk akurasi yang lebih baik.

Protokol komunikasi seperti LoRaWAN menjadi pilihan unggulan karena mendukung komunikasi jarak jauh dengan konsumsi daya rendah. LoRaWAN terdiri atas tiga lapisan utama: physical layer, MAC (Medium Access Control), dan application layer. Parameter kualitas transmisi seperti Received Signal Strength Indicator (RSSI) dan Signal-to-Noise Ratio (SNR) menjadi indikator penting untuk menilai keandalan sistem [8].

Dengan menggabungkan akuisisi sinyal ECG, ekstraksi fitur HR, kompresi data, dan transmisi melalui LoRaWAN, sistem pemantauan kesehatan yang efisien, dan hemat daya dapat dikembangkan. Penelitian ini bertujuan untuk merancang sistem terintegrasi tersebut dan menganalisis performanya dari sisi akurasi dan efisiensi transmisi, dengan menyempurnakan keterbatasan dari studi-studi sebelumnya.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Perancangan sistem pengiriman data ECG berdaya rendah menggunakan LoRaWAN yang dapat mengoptimalkan efisiensi energi untuk pemantauan pasien.
2. Mengimplementasikan teknik kompresi data yang dapat memengaruhi transmisi data ECG melalui LoRaWAN
3. Perancangan sistem penyimpanan dan visualisasi data ECG yang efektif untuk mendukung pemantauan pasien oleh tim medis.

1.3 Tujuan

Tujuan dari penelitian ini adalah:

1. Merancang dan menganalisis sistem pemantauan ECG melalui integrasi sensor ECG dan modul komunikasi LoRa.
2. Mengevaluasi efektivitas teknik kompresi dalam menurunkan konsumsi daya transmisi LoRaWAN, dengan target rasio kompresi minimal 2:1 tanpa kehilangan informasi diagnostik penting.
3. Mengembangkan sistem penyimpanan data berbasis *database* dan visualisasi sinyal ECG untuk mendukung kegiatan pemantauan pasien oleh tim medis.

1.4 Manfaat Hasil Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Memberikan solusi pemantauan ECG yang andal, efisien, dan berkelanjutan untuk mendukung perawatan pasien jangka panjang.
2. Memfasilitasi proses pemantauan pasien yang lebih efektif melalui sistem penyimpanan dan visualisasi data ECG.
3. Memberikan kontribusi dalam pengembangan teknologi *Internet of Things* (IoT) di bidang kesehatan, khususnya dalam optimalisasi pemanfaatan LoRaWAN untuk transmisi data medis berdaya rendah.

1.5 Batasan Masalah

Untuk memfokuskan penelitian ini, beberapa batasan masalah ditetapkan sebagai berikut:

1. Pengujian sistem dilakukan dalam lingkungan simulasi ruangan berukuran 4x4 meter, mewakili satu kamar pasien.
2. Sistem dirancang untuk pemantauan ECG berkelanjutan, dengan asumsi perawatan dan pemeliharaan rutin tanpa mengganggu operasi sistem.
3. Penelitian berfokus pada akuisisi sinyal ECG menggunakan konfigurasi 3-lead standar (RA, LA, RL).
4. Aspek keamanan dan privasi data pasien tidak menjadi fokus utama penelitian ini, namun tetap mempertimbangkan standar keamanan dasar dalam transmisi dan penyimpanan data.
5. Visualisasi data pada antarmuka web ditujukan untuk tujuan *monitoring* dan bukan untuk interpretasi atau diagnosis medis secara klinis.

6. Penelitian ini tidak mencakup aspek klinis atau interpretasi medis yang mendalam dari data ECG yang dikumpulkan, melainkan berfokus pada aspek teknis sistem pemantauan.
7. Pengujian akurasi *heart rate* (HR) dari sensor ECG dilakukan dengan membandingkan hasil perhitungan algoritma Pan-Tompkins terhadap alat *pulse oximeter* komersial sebagai referensi.

1.6 Metode Penelitian

Penelitian ini akan menggunakan pendekatan kombinasi antara studi teoritis, perancangan, implementasi, dan pengujian empiris. Metode penelitian yang akan diterapkan meliputi:

1. Studi Literatur

Dilakukan kajian mendalam terhadap literatur terkait pemantauan ECG, teknologi LoRaWAN, kompresi data, dan sistem IoT dalam bidang kesehatan. Fokus kajian meliputi prinsip kerja, kelebihan, kekurangan, dan penerapan teknologi-teknologi tersebut.

2. Perancangan Sistem

- a. Arsitektur Sistem: Dirancang arsitektur sistem yang terdiri dari sensor ECG, modul LoRa, *gateway* LoRaWAN, dan *server* data.
- b. Algoritma: Dikembangkan algoritma pengolahan sinyal ECG dan algoritma kompresi data.
- c. Protokol Komunikasi: Ditetapkan protokol komunikasi antara komponen sistem.

3. Implementasi

- a. Pembuatan *Prototype*: Dibangun *prototipe* sistem sesuai dengan rancangan.
- b. Pemrograman: Dikembangkan *firmware* untuk mikrokontroler dan aplikasi *server* menggunakan bahasa pemrograman yang sesuai.

4. Pengujian empiris

- a. Pengujian Fungsional: Dilakukan pengujian untuk memastikan semua komponen sistem berfungsi dengan baik dan sesuai dengan spesifikasi.
- b. Pengukuran Kinerja: Dilakukan pengukuran terhadap konsumsi daya, latensi transmisi, tingkat kesalahan bit (BER), dan kualitas sinyal ECG setelah kompresi.

5. Analisis data

- a. Analisis Kuantitatif: Data hasil pengukuran dianalisis menggunakan statistik deskriptif dan inferensial untuk menguji hipotesis penelitian.
- b. Visualisasi Data: Hasil analisis disajikan dalam bentuk grafik dan tabel untuk memudahkan interpretasi.

6. Evaluasi

- a. Perbandingan: Kinerja sistem yang dikembangkan dibandingkan dengan sistem pemantauan ECG konvensional.
- b. Optimasi: Dilakukan optimasi pada parameter sistem untuk meningkatkan kinerja

7. Proyeksi Pengguna

Penelitian ini diharapkan dapat digunakan oleh beberapa pengguna seperti:

a. Rumah Sakit

1. Meningkatkan efisiensi pemantauan pasien, terutama untuk pasien dengan kondisi kronis atau yang membutuhkan pemantauan intensif.
2. Mengurangi beban kerja tenaga medis, sehingga memungkinkan mereka untuk fokus pada perawatan pasien yang lebih kompleks.
3. Meningkatkan kualitas data pasien, sehingga mendukung pengambilan keputusan klinis yang lebih akurat.

b. Industri Peralatan Medis

1. Menjadi referensi dalam pengembangan perangkat medis yang lebih inovatif, hemat biaya, dan ramah lingkungan.
2. Membuka peluang pasar baru untuk produk-produk pemantauan kesehatan berbasis IoT.

c. Akademisi dan Peneliti

Menyediakan dasar yang kuat untuk penelitian lebih lanjut di bidang IoT untuk kesehatan, khususnya dalam pemantauan tanda vital pasien.

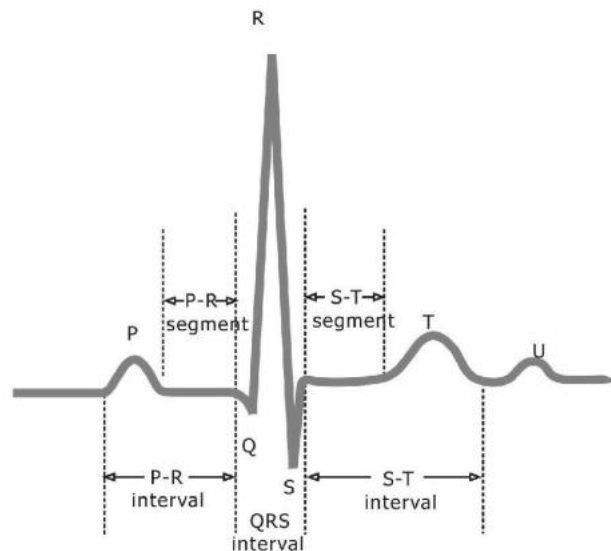
BAB II

TINJAUAN PUSTAKA

2.1 Elektrokardiogram (ECG)

Electrocardiogram (ECG) adalah sinyal bioelektrik yang dihasilkan oleh aktivitas listrik jantung dan direkam melalui elektroda yang diletakkan di permukaan tubuh. Sinyal ini terdiri dari komponen gelombang P, QRS, dan T yang masing-masing merepresentasikan depolarisasi atrium, depolarisasi ventrikel, dan repolarisasi ventrikel [1], [9]. Struktur morfologi dari sinyal ECG digunakan untuk menganalisis berbagai kondisi kardiak seperti aritmia, iskemia, dan blok jantung.

2.1.1 Sinyal ECG, dan *Heart Rate* (HR)



Gambar 2.1 Gelombang EKG dan komponen-komponennya [2]

Sinyal ECG memiliki amplitudo yang kecil, berkisar antara 0.5–5 mV, dengan rentang frekuensi dominan 0.05–100 Hz, sehingga rentan terhadap *noise* seperti EMG, *baseline wander*, dan interferensi daya 50/60 Hz [10]. Untuk itu, sistem akuisisi memerlukan penguatan dan penyaringan sinyal yang tepat agar hasil yang diperoleh dapat digunakan untuk deteksi fitur dengan akurasi tinggi.

Heart Rate (HR) adalah parameter vital yang dapat diperoleh dengan menghitung selisih waktu antar puncak R (RR-interval) dari *QRS complex*. HR memiliki makna diagnostik dalam deteksi abnormalitas seperti bradikardia (<60 bpm), takikardia (>100 bpm), maupun fibrilasi [5], [10].

2.1.2 Algoritma Pan-Tompkins

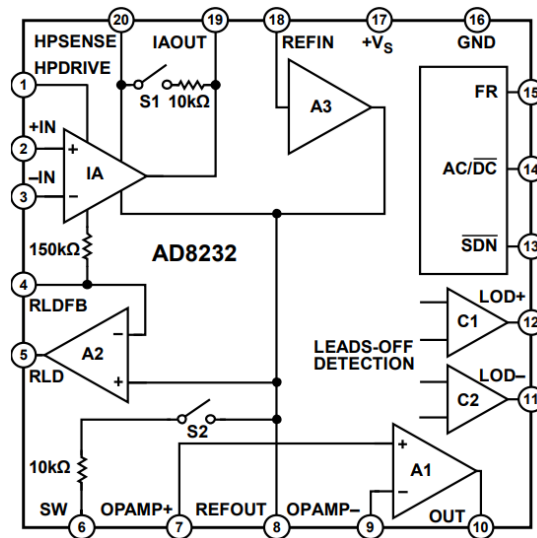
Algoritma Pan-Tompkins merupakan metode klasik namun masih relevan hingga kini untuk mendeteksi puncak R dari sinyal ECG. Algoritma ini bekerja secara *real-time* dengan kombinasi tahapan *filtering band-pass* (5–15 Hz), derivatif, pengkuadratan, integrasi jendela, dan *threshold* adaptif [5].

Meski efisien dan ringan, Pan-Tompkins memiliki keterbatasan terhadap noise tinggi dan variasi morfologi sinyal. Untuk itu, banyak penelitian mengembangkan modifikasi dari algoritma ini. Sahu et al. [10] mengusulkan varian yang memperbaiki sensitivitas terhadap *baseline drift* dan menurunkan waktu pemrosesan menjadi 52 ms menggunakan teknik *smoothing Savitzky-Golay* dan klasifikasi berbasis *slope*.

Selain itu, pendekatan kombinasi seperti *Complex-Pan-Tompkins-Wavelets* (CPTW) [11] menggunakan gabungan tiga pendekatan: algoritma Christov (*lead complex*), Pan-Tompkins, dan *Discrete Wavelet Transform* (DWT), sehingga dapat mengkompensasi kelemahan satu algoritma dengan kelebihan algoritma lain. Algoritma ini mampu mencapai sensitivitas hingga 99.57% pada *database INCART*.

2.1.3 Sensor ECG AD8232

Sensor AD8232 merupakan rangkaian *analog front-end* (AFE) yang dirancang khusus untuk penguatan dan penyaringan sinyal bioelektrik seperti ECG. Sensor ini digunakan secara luas karena konsumsi dayanya rendah, bentuknya kompak, dan kemudahan integrasi dengan platform *open-source* seperti Arduino [4], [9].



Gambar 2.2 Blok diagram fungsional AD8232

Dalam studi oleh Das dan Roy [9], AD8232 berhasil diintegrasikan dalam sistem portabel berbasis Arduino Nano dengan transmisi data melalui Bluetooth dan pencatatan data ke SD card. Sensor ini mendukung filtrasi analog dan menyediakan *output* digital oleh ADC mikrokontroler. Selain itu, Gomez et al. [12] menunjukkan bahwa AD8232 dapat digunakan secara efisien dalam sistem *monitoring real-time* berbasis ESP32 dan LoRa, menjadikannya sangat cocok untuk sistem kesehatan berbasis IoT.

2.2 Kompresi Data ECG

Dalam sistem pengawasan kesehatan berbasis IoT, seperti pengiriman sinyal ECG melalui LoRaWAN, kompresi data berperan penting dalam mengurangi beban transmisi, konsumsi daya, dan penggunaan *bandwidth*. Karena keterbatasan kapasitas transmisi dan energi pada perangkat *edge* seperti sensor ECG *wearable*, diperlukan metode kompresi yang efisien namun tetap mempertahankan kualitas sinyal yang memadai untuk diagnosis klinis.

Beragam metode kompresi telah dikembangkan, mulai dari transformasi (seperti DWT dan DCT), kuantisasi, hingga teknik *encoding* seperti Huffman, LZ77 (Zlib), dan LZMA. Masing-masing metode memiliki kekuatan dan kelemahan tergantung pada konteks aplikasinya (*real-time*, *offline*, atau *e-health secured*).

2.2.1 Teknik Kompresi Data ECG

Beberapa metode kompresi data ECG telah dikembangkan untuk mengatasi tantangan transmisi data yang efisien, terutama dalam sistem berbasis IoT. Metode-metode ini memiliki kelebihan dan kekurangan masing-masing, yang dirangkum dalam tabel berikut:

Tabel 2.1 Perbandingan metode kompresi ECG

Ref	Metode	Kelebihan	Kekurangan
[13]	DWT	Menyediakan kompresi efisien untuk sinyal non-stasioner; pelestarian struktur QRS yang baik	Perlu pemilihan <i>threshold</i> yang tepat, komputasi lebih tinggi dibanding DCT
[14]	DCT + Quant + Golomb	Efisien dalam domain frekuensi, PRD rendah dengan CR > 10, sederhana untuk implementasi blok	Kurang optimal untuk sinyal <i>abrupt</i> seperti detak jantung abnormal
[15]	DWT + DPCM + Run-Length	Memisahkan <i>Region of Interest</i> (RoI); kombinasi lossy dan lossless adaptif	Kompleksitas tinggi; butuh segmentasi ECG yang presisi
[16]	Zlib (LZ77)	Kompresi <i>lossless</i> stabil, cocok untuk transmisi file data klinis	Latensi dan komputasi relatif lebih tinggi untuk <i>real-time</i>
[17]	LZMA	CR sangat tinggi hingga >18, efisien untuk penyimpanan <i>offline</i>	Kurang cocok untuk sistem <i>real-time</i> ; pemrosesan berat
[18]	PWM + Quant	Hanya 9 operasi per sampel; sangat cocok untuk sistem <i>ultra-low power real-time</i>	CR terbatas; belum banyak diuji pada <i>database</i> besar
[19]	SVD	Preservasi sinyal yang baik bahkan pada distorsi tinggi; cocok untuk klasifikasi	Tidak cocok untuk sistem <i>real-time</i> ; lebih untuk <i>preprocessing</i> sebelum klasifikasi
[20]	Adaptive Fourier (AFD + SS)	CR sangat tinggi (17–44); ada enkripsi terintegrasi untuk <i>e-health</i>	Implementasi relatif kompleks; <i>overhead</i> lebih besar
[21]	VSQ + Arithmetic Coding	Gabungan wavelet + vector quantization + coding yang efisien	Perlu pelatihan awal model kuantisasi dinamis

2.2.2 Discrete Wavelet Transform (DWT)

DWT sangat populer dalam kompresi sinyal ECG karena kemampuannya merepresentasikan sinyal non-stasioner dalam domain waktu dan frekuensi. DWT memungkinkan pemisahan komponen frekuensi tinggi (seperti *QRS complex*) dan rendah (seperti gelombang P/T) secara terstruktur. Proses kompresi dilakukan dengan memilih koefisien *wavelet* yang memiliki nilai dominan berdasarkan *Energy Packing Efficiency* (EPE) [13].

Pada penelitian oleh Shinde et al. [13], digunakan tiga tipe *wavelet* (Haar, Bior1.1, dan Db2) dengan hasil kompresi yang memuaskan. Diperoleh Compression Ratio (CR) 16.33 dan PRD serendah 1.02%, yang artinya sinyal dapat direkonstruksi dengan kehilangan informasi minimal.

Selain itu, dalam pendekatan *hybrid* [15], DWT digabungkan dengan DPCM dan *run-length encoding*. Bagian penting sinyal seperti QRS (RoI) dikompresi *lossless*, sementara bagian lainnya secara *lossy*, sehingga *trade-off* antara efisiensi dan akurasi klinis dapat dicapai secara optimal

2.2.3 Quantization (Kuantisasi)

Kuantisasi adalah proses mengurangi banyaknya nilai berbeda dalam representasi sinyal dengan memetakannya ke level tertentu. Ini digunakan baik setelah transformasi (DWT, DCT) maupun langsung ke sinyal ECG. Jenis kuantisasi yang umum adalah *uniform quantization* dan *non-uniform quantization*.

Contohnya, pada [14], setelah dilakukan DCT, kuantisasi dilakukan secara optimal terhadap blok sinyal, dan dilanjutkan dengan *Golomb-Rice coding*. Ini menghasilkan CR sebesar 10.4:1 untuk PRD 2.5%. Penyesuaian *step size* dan *zeroing threshold* pada proses kuantisasi dilakukan untuk menyeimbangkan antara entropi dan distorsi.

Di sisi lain, dalam pendekatan berbasis PWM [18], kuantisasi dilakukan dengan mengubah sinyal ECG menjadi representasi *Pulse Width Modulation* (PWM) 4-bit. Hal ini menurunkan kompleksitas dan cocok untuk sistem *wearable* dengan energi sangat rendah.

2.2.4 Zlib (LZ77)

Zlib adalah algoritma *lossless compression* berbasis LZ77 (Ziv-Lempel), yang bekerja dengan mengenali pola berulang pada data dan mengkodekannya dengan efisien. Dalam konteks ECG, Zlib digunakan setelah transformasi atau kuantisasi untuk menyimpan atau mengirimkan data secara efisien tanpa kehilangan [16].

Dalam studi klasik oleh Martinez et al. [16], LZ77 berhasil menurunkan ukuran sinyal ECG dengan akurasi tinggi dan tanpa menyebabkan artefak yang signifikan pada sinyal terdekompresi. Meskipun kompresi *lossless* biasanya tidak seefisien kompresi *lossy* dalam hal CR, keunggulannya adalah rekonstruksi sinyal 100% dan digunakan untuk kebutuhan medis yang ketat.

2.3 Long Range Wide Area Network (LoRaWAN)

Long Range Wide Area Network (LoRaWAN) merupakan protokol komunikasi berbasis *Low Power Wide Area Network* (LPWAN) yang dirancang untuk komunikasi jarak jauh, hemat daya, dan skalabel, menjadikannya ideal untuk sistem pemantauan kesehatan berbasis IoT, termasuk transmisi sinyal ECG. LoRaWAN menggunakan modulasi LoRa (*chirp spread spectrum*) pada lapisan fisik serta protokol jaringan terbuka untuk komunikasi data secara efisien.

2.3.1 Arsitektur Jaringan LoRaWAN

Arsitektur LoRaWAN secara umum terdiri dari empat komponen utama:

1. *End Device (Node)*: Sensor ECG seperti AD8232 terhubung ke mikrokontroler (ESP32/STM32) yang mengumpulkan dan memproses data, lalu mengirimkannya melalui LoRa.
2. *Gateway*: Bertindak sebagai jembatan antara perangkat *end device* dan jaringan. *Gateway* meneruskan data dari *node* ke server menggunakan konektivitas IP seperti WiFi atau Ethernet.
3. *Network Server*: Mengelola komunikasi dan *routing* data, serta menjalankan fungsi penting seperti autentikasi, deduplikasi, dan pengaturan *data rate* secara adaptif (ADR).

4. *Application Server*: Sistem *backend* yang menyimpan, memproses, dan menampilkan data ECG kepada pengguna atau tenaga medis melalui aplikasi.

Arsitektur ini mengadopsi topologi *star-of-stars*, menjadikannya lebih hemat daya dibanding teknologi seperti WiFi atau Bluetooth, serta lebih cocok untuk area luas seperti desa atau kota pintar [22].

2.3.2 Lapisan Protokol LoRaWAN

A. Physical Layer (PHY)

1. Menggunakan modulasi LoRa *Chirp Spread Spectrum* (CSS).
2. Menyediakan jangkauan hingga 15 km di area terbuka dan 2–5 km di area urban.
3. Parameter teknis utama: *Spreading Factor* (SF), *Bandwidth* (BW), *Coding Rate* (CR).
4. Memungkinkan komunikasi dalam kondisi sinyal lemah (SNR negatif).

LoRa memiliki ketahanan tinggi terhadap interferensi dan cocok untuk transmisi sinyal *biomedical* yang memiliki *bandwidth* rendah seperti ECG [23].

B. MAC Layer

1. LoRaWAN menggunakan model komunikasi asinkron berbasis ALOHA.
2. Mendukung tiga kelas komunikasi:
 - Kelas A: Efisiensi daya terbaik, cocok untuk perangkat *wearable*.
 - Kelas B: Mendukung sinkronisasi waktu untuk komunikasi dua arah terjadwal.
 - Kelas C: Komunikasi dua arah terus-menerus dengan konsumsi daya lebih tinggi.

Sistem pemantauan ECG secara periodik sangat ideal menggunakan Kelas A dengan konsumsi daya minimum dan pemrosesan event-driven [24].

C. Application Layer

1. Mengelola *payload* data, misalnya hasil kompresi sinyal ECG.
2. Mendukung *end-to-end encryption* (AES-128).
3. Data dapat ditampilkan dalam bentuk *dashboard* kesehatan, alarm kritis, dan rekam medis digital.

2.3.3 Keunggulan LoRaWAN untuk Transmisi ECG

Tabel 2.2 Keunggulan transmisi LoRaWAN

Parameter	Keterangan
Jangkauan	>10 km di rural, 2–5 km di urban
Konsumsi Energi	Ultra rendah, ideal untuk <i>wearable</i>
<i>Bandwidth</i>	Sesuai untuk data ECG yang telah dikompresi
Topologi	<i>Star-of-stars</i> , minimalkan beban <i>node</i>
Komunikasi	Asinkron, hemat daya, efisien
Biaya Infrastruktur	Rendah, tidak butuh menara BTS

LoRaWAN dapat menghemat hingga 90% konsumsi daya dibanding GSM/3G dan sangat cocok untuk pengiriman sinyal vital secara periodik dari perangkat *wearable* [25].

2.3.4 Studi Kasus Pemanfaatan LoRa dalam IoT Kesehatan

Tabel 2.3 Studi kasus tantangan penggunaan LoRa

Tantangan	Solusi Teknologi
Kapasitas data rendah	Kompresi (DWT, Kuantisasi, Huffman/Zlib)
<i>Collision</i> antar <i>node</i>	Penjadwalan ADR dan peningkatan slot
<i>Delay</i> tinggi di Kelas A	Tuning interval transmisi dan kelas komunikasi
Keamanan data	Enkripsi AES dan autentikasi perangkat

Penggunaan *Adaptive Data Rate* (ADR) terbukti mengurangi paket hilang dan efisiensi pengiriman data pada sistem *monitoring* kesehatan berbasis LoRa [24].

2.4 Parameter Kualitas Transmisi

Dalam sistem transmisi data nirkabel seperti LoRaWAN, kualitas sinyal yang diterima sangat memengaruhi keberhasilan pengiriman data. Dua parameter penting yang digunakan untuk mengevaluasi kualitas ini adalah RSSI (*Received Signal Strength Indicator*) dan SNR (*Signal-to-Noise Ratio*).

2.4.1 Received Signal Strength Indicator (RSSI)

RSSI mengukur kekuatan sinyal yang diterima oleh *receiver*, dalam satuan dBm. Makin besar nilai RSSI (mendekati 0), makin kuat sinyal yang diterima. Pada sistem LoRa, nilai RSSI biasanya berada pada kisaran -120 dBm hingga -30 dBm. Rumus Umum RSSI:

$$\text{RSSI} = P_t + G_t + G_r - L_p \quad (2.1)$$

Keterangan:

- P_t : Daya transmit (dBm)
- G_t : Gain antenna *transmitter* (dBi)
- G_r : Gain antenna *receiver* (dBi)
- L_p : *Path loss* (dB)

2.4.2 Signal-to-Noise Ratio (SNR)

SNR adalah ukuran perbandingan antara sinyal dan derau (*noise*) yang diterima. SNR positif berarti sinyal lebih kuat dari *noise*, sedangkan SNR negatif berarti *noise* lebih dominan dan ini umum terjadi pada LoRa. Rumus SNR (linear dan dB):

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}} \quad (2.2)$$

$$\text{SNR}_{\text{dB}} = 10 \cdot \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right) \quad (2.3)$$

Persamaan 2.2 merupakan rumus SNR linear dan persamaan 2.3 merupakan rumus dB). Nilai SNR LoRaWAN bisa berada di kisaran -20 dB hingga $+10$ dB. Sistem LoRa tetap bisa bekerja di $\text{SNR} < 0$, berkat modulasi CSS [27].

2.4.3 Interpretasi dan Peran dalam LoRaWAN

Tabel 2. 4 Interpretasi RSSI dan SNR pada LoRaWAN

Parameter	Rentang Umum (LoRa)	Interpretasi
RSSI	-120 dBm s/d -30 dBm	Semakin mendekati 0 → sinyal lebih kuat
SNR	-20 dB s/d +10 dB	>0 dB: sinyal bagus, <0 dB: banyak noise

- RSSI tinggi & SNR rendah → sinyal kuat tapi *noise* tinggi → bisa gagal kirim.
- RSSI rendah & SNR tinggi → sinyal lemah tapi bersih → tetap bisa diterima.

Relevansi terhadap Data ECG:

- ECG adalah sinyal sensitif → transmisi perlu kualitas tinggi.
- Sistem transmisi ECG yang buruk akan menyebabkan *packet loss*, *jitter*, atau distorsi data fisiologis.

Studi eksperimental oleh Krein et al. menunjukkan bahwa lingkungan *indoor* seperti gedung perkantoran dapat menyebabkan fluktuasi nilai RSSI hingga >20 dB, tergantung pada jumlah lantai dan dinding beton yang dilalui sinyal, sehingga penting untuk mempertimbangkan *loss* lingkungan pada sistem pemantauan kesehatan berbasis LoRa [28].

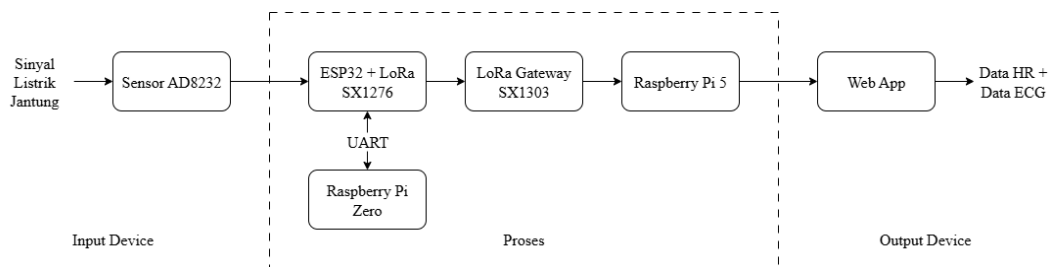
BAB III

PERANCANGAN SISTEM

3.1 Desain Sistem

Sistem ini dirancang untuk melakukan pengiriman sinyal listrik jantung secara nirkabel menggunakan LoRa. Perancangan sistem ini akan meliputi diagram blok sistem secara umum, perancangan perangkat keras, serta perancangan perangkat lunak. Pada perancangan perangkat keras akan dipaparkan spesifikasi komponen serta memastikan komponen dapat memenuhi kebutuhan yang diinginkan. Pada perancangan perangkat lunak akan membahas terkait sistem akuisisi data, metode komunikasi LoRa, serta metode kompresi yang digunakan.

3.1.1 Diagram Blok



Gambar 3.1 Diagram blok sistem

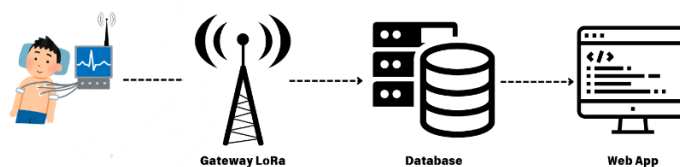
Gambar 3.1 menggambarkan blok diagram yang menunjukkan alur kerja dari sistem yang akan dirancang. Pada sistem ini, sensor AD8232 digunakan untuk mendeteksi sinyal listrik yang berasal dari aktivitas jantung pasien. Sinyal yang dihasilkan berupa sinyal analog yang mencerminkan aktivitas elektrokardiogram (ECG) pasien. Sensor ini dirancang khusus untuk memantau kondisi jantung pasien dengan akurat.

Modul ESP32 bertindak sebagai pengakuisisi data utama dalam sistem ini. Setelah menerima sinyal dari sensor AD8232, ESP32 melakukan akuisisi data, yaitu mengubah sinyal analog menjadi data digital secara real-time. Mikrokontroler ESP32 dipilih karena memiliki konsumsi daya rendah dan kemampuan komunikasi LoRa yang terintegrasi, sehingga sangat ideal untuk membaca dan mengirim data sinyal ECG secara efisien.

Namun, karena proses kompresi sinyal ECG menggunakan metode *Discrete Wavelet Transform* (DWT) dan algoritma Zlib memerlukan daya komputasi yang lebih besar dan memori yang lebih luas, maka digunakan Raspberry Pi Zero sebagai unit pengolah data tambahan. Raspberry Pi Zero menerima data dari ESP32 dan menjalankan proses filter serta kompresi sinyal sebelum dikirimkan melalui modul LoRa. Pemisahan tugas antara ESP32 dan Raspberry Pi Zero ini dilakukan untuk mengoptimalkan performa sistem dengan tetap menjaga efisiensi daya.

Data yang telah dikompresi oleh ESP32 kemudian dikirimkan melalui modul LoRa *Transmitter*. Teknologi LoRa dipilih karena memiliki jangkauan transmisi yang luas serta konsumsi daya yang rendah, yang sangat cocok untuk pemantauan pasien secara berkelanjutan, terutama di lingkungan yang membutuhkan pengiriman data jarak jauh tanpa menguras daya. LoRa *gateway* berfungsi sebagai penerima utama dari data yang dikirimkan melalui LoRa *Transmitter*. *Gateway* ini menangkap data yang dikirim dan mempermudah pengumpulan informasi dari banyak perangkat secara bersamaan, mengurangi kebutuhan akan pengaturan jaringan yang kompleks.

Setelah diterima oleh LoRa *gateway*, data akan diteruskan ke Raspberry Pi 5 untuk pemrosesan lebih lanjut. Di sini, data ECG dapat divisualisasikan dalam bentuk grafik yang memudahkan tim medis untuk memantau kondisi pasien. Selain itu, data tersebut juga disimpan untuk keperluan dokumentasi dan analisis jangka panjang. Untuk penyimpanan data, digunakan PostgreSQL dengan TimescaleDB, yang dirancang khusus untuk menangani data *time-series*. Hal ini memungkinkan pengelolaan data ECG secara efisien dan tetap menjaga performa meskipun data yang disimpan semakin besar seiring berjalannya waktu.



Gambar 3.2 Ilustrasi komunikasi jaringan sistem

3.1.2 Fungsi dan Fitur

Berdasarkan sistem yang akan dirancang, fungsi dan fitur sebagai berikut.

A. Fungsi:

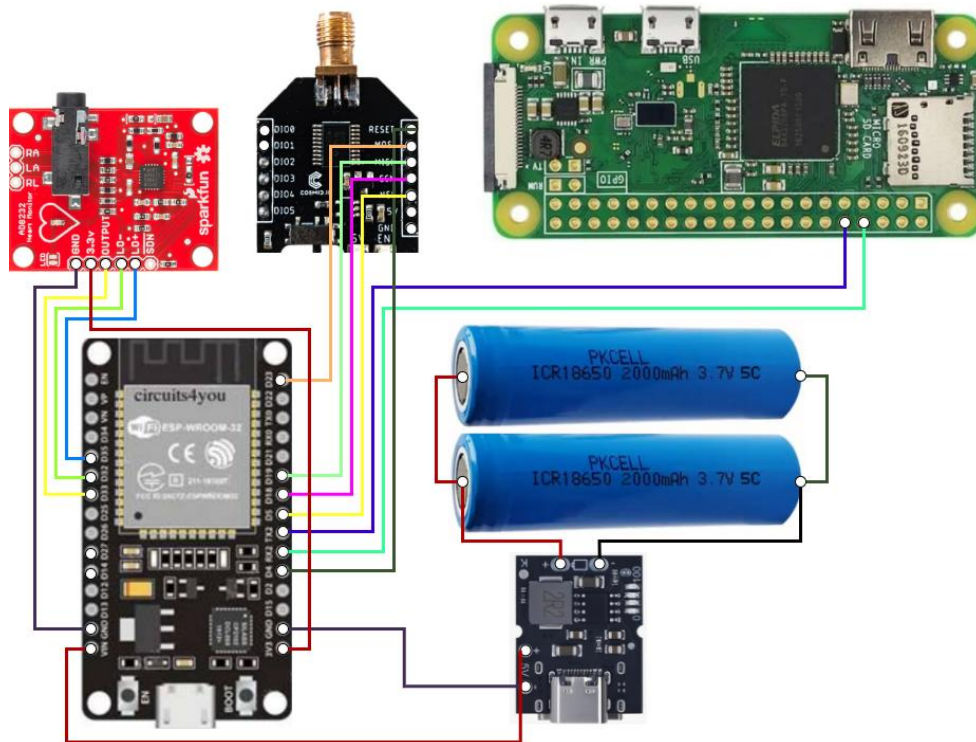
1. Sistem menggunakan sensor AD8232 untuk membaca sinyal listrik dari aktivitas jantung pasien.
2. Data ECG yang diterima oleh ESP32 diubah dari analog menjadi digital, kemudian dikompresi menggunakan teknik DWT untuk mengurangi ukuran data tanpa kehilangan informasi diagnostik penting.
3. Data yang telah dikompresi dikirim melalui jaringan LoRa ke *gateway*, dan meneruskannya ke mikrokomputer untuk disimpan dan divisualisasikan.

B. Fitur:

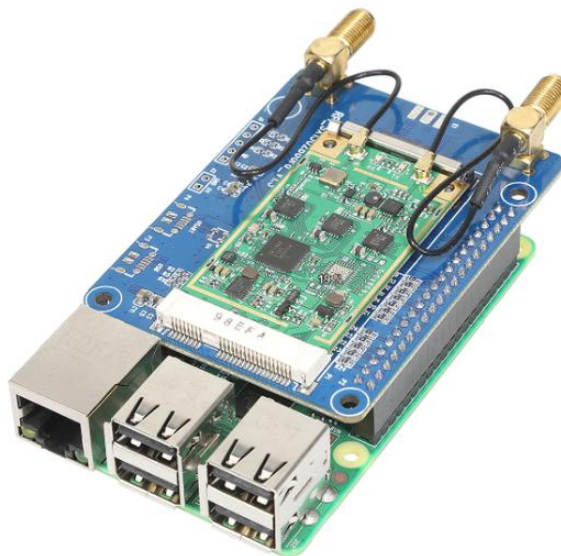
1. Sistem dirancang untuk konsumsi daya rendah, memungkinkan perangkat digunakan selama berminggu-minggu tanpa penggantian baterai.
2. Data ECG ditampilkan secara di layar komputer dokter untuk mendukung pengambilan keputusan cepat.
3. Sistem mendukung pemantauan hingga 3 pasien secara bersamaan.

3.2 Design Perangkat Keras

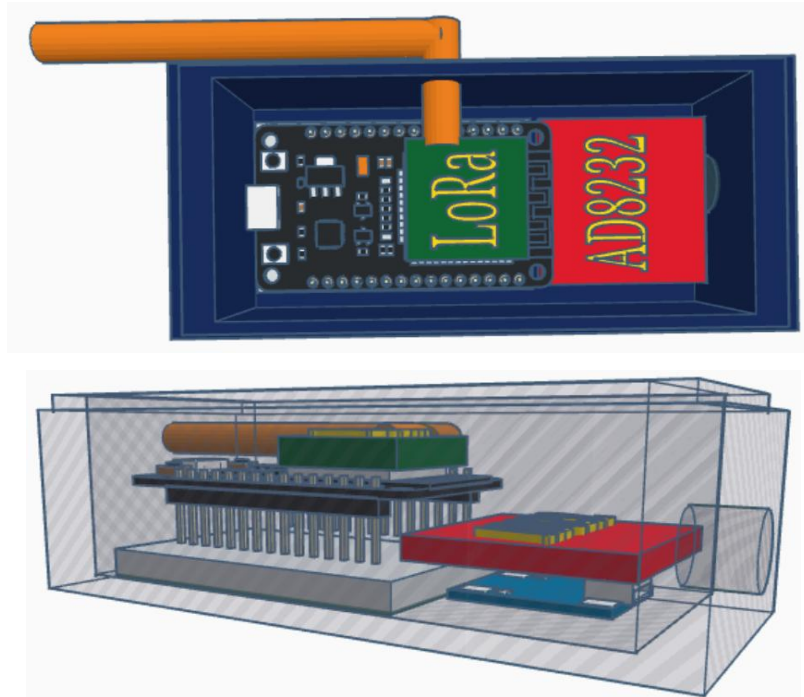
Perangkat Keras yang akan digunakan pada sistem antara lain sensor AD8232, ESP32, Raspberry Pi 0, SX1276 LoRa *transceiver*, SX1303 LoRa *gateway*, Raspberry Pi 5, *charging module*, baterai 18650 paralel. Komponen-komponen ini akan dirangkai sehingga menjadi sebuah alat yang dapat digunakan pengirim data ECG dengan LoRa.



Gambar 3.3 Rangkaian pengirim data sinyal ECG



Gambar 3.4 SX1303 LoRa gateway shield untuk Raspberry Pi 5



Gambar 3.5 Dari atas ke bawah, Desain 3D kotak komponen tampak atas dan tampak samping

Gambar 3.3 dan Gambar 3.4 menunjukkan skema rangkaian yang telah dirancang. Gambar 3.3 merupakan sistem yang digunakan untuk mengakuisisi data sinyal listrik jantung oleh sensor AD8232 dan dikompresi dengan Raspberry Pi 0. Lalu terdapat modul LoRa SX1276 yang digunakan untuk mengirim data ke *gateway*. Serta terdapat *charging module* dan baterai yang digunakan untuk memberi daya pada perangkat. Pada gambar 3.4 menunjukkan Raspberry Pi 5 yang dihubungkan dengan modul *gateway* SX1303 yang digunakan untuk menerima data yang dikirim dari perangkat LoRa, serta sebagai sistem pemrosesan utama untuk data yang diterima.

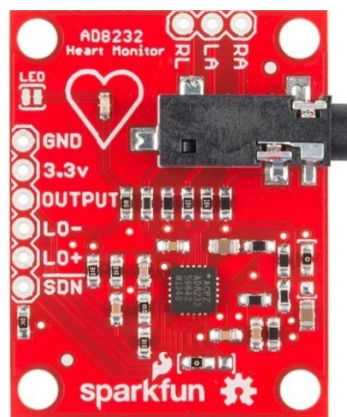
Gambar 3.5 merupakan desain *casing* 3D ini dirancang untuk menampung beberapa komponen utama, yaitu mikrokontroler, sensor ECG, modul LoRa, baterai, dan modul pengisian daya. Setiap komponen telah diatur secara optimal di dalam *casing* untuk memaksimalkan penggunaan ruang. Antena LoRa diletakkan secara strategis untuk meningkatkan kualitas sinyal, sementara sensor ECG dan modul pengisian daya mudah diakses untuk pemeliharaan atau penggantian jika

diperlukan. Material *casing* direncanakan untuk melindungi komponen dari debu dan benturan ringan, sekaligus mempertahankan desain yang ringkas dan portabel.

3.2.1 Spesifikasi Komponen

Perancangan sistem pengirim data menggunakan LoRa. Komponen yang digunakan meliputi sensor AD8232, ESP32, Raspberry Pi 0, SX1276 LoRa *transceiver*, SX1303 LoRa *gateway*, Raspberry Pi 5, *charging module*, baterai 18650. Berikut merupakan spesifikasi dari komponen yang digunakan:

1. Sensor ECG



Gambar 3.6 Sensor AD8232

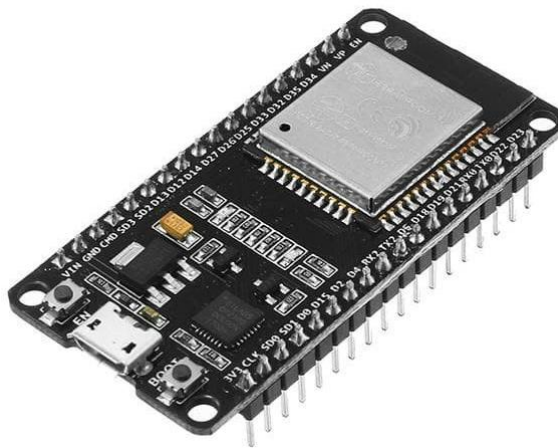
Sensor AD8232 adalah modul yang digunakan untuk membaca sinyal listrik dari aktivitas jantung. Modul ini dirancang untuk mendeteksi sinyal ECG dengan akurasi tinggi dan cocok untuk aplikasi pemantauan kesehatan. Sensor ini memiliki kemampuan untuk menyaring *noise* dari sinyal ECG sehingga menghasilkan data yang lebih bersih dan akurat. Sensor ini dipilih setelah dilakukan perbandingan spesifikasi dengan komponen serupa pada Tabel 3.1.

Tabel 3.1 Perbandingan spesifikasi sensor ECG

Parameter	Bobot (%)	AD8232	ADS1292R	MAX30003
Konsumsi Daya	25	0.5 mA	1.5 mA	1.2 mA
		0,25	0,25	0
Akurasi	25	12 bit	24 bit	16 bit
		0,25	0	0,25

Jumlah Saluran ECG	20	1	2	1
		0,2	0	0,2
Resolusi ADC	15	12 bit	24 bit	16 bit
		0,15	0	0,15
<i>Sampling Rate</i>	10	1 kHz	250 Hz	500 Hz
		0,1	0,1	0
Harga	5	70000	150000	120000
		0,05	0,05	-0,05
Total Nilai	100	1	0,4	0,55

2. Mikrokontroler



Gambar 3.7 ESP32

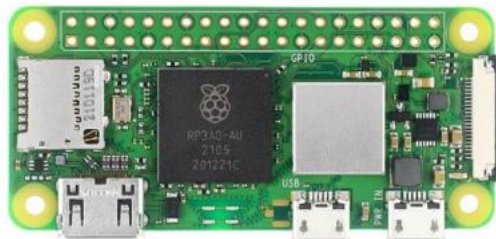
ESP32 adalah modul mikrokontroler yang digunakan untuk akuisisi data, kompresi, dan pengiriman data melalui LoRa. Modul ini memiliki performa tinggi dengan konsumsi daya rendah, serta mendukung komunikasi Wi-Fi dan Bluetooth yang dapat digunakan untuk pengembangan fitur tambahan. Mikrokontroler ini dipilih setelah dilakukan perbandingan spesifikasi dengan komponen serupa pada Tabel 3.2.

Tabel 3.2 Perbandingan spesifikasi mikrokontroler

Parameter	Bobot (%)	ESP32	Arduino Nano	Raspberry Pico
CPU	30	Dual-core 240 MHz	16 MHz	Dual-core 133 MHz

		0,3	-0,3	0
Memori	25	520 KB SRAM, hingga 16 MB Flash	2 KB SRAM, 32 KB Flash	264 KB SRAM, 2 MB Flash
		0,25	-0,25	0
I/O Pin	15	34	20	26
		0,15	0	0
Konsumsi Daya	20	160–240 mA	50 mA	30–50 mA
		0	0,2	0,2
Harga	10	Rp70.000	Rp50.000	Rp50.000
		0,1	0,1	0,1
Total Nilai	100	0,8	-0,25	0,3

3. Raspberry Pi 0



Gambar 3. 8 Raspberry Pi Zero

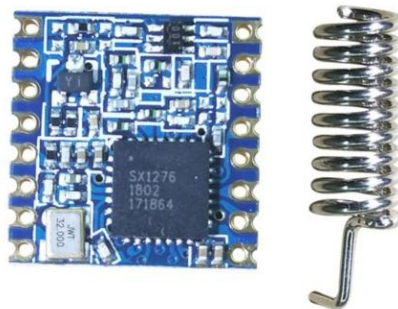
Raspberry Pi Zero 2W pada Gambar 3.8, merupakan mikrokomputer yang digunakan sebagai pengolah data atau proses kompresi sebelum dikirim melalui komunikasi LoRa. Pada Tabel 3.2 merupakan perbandingan yang telah dilakukan dalam memilih komponen untuk proses kompresi.

Tabel 3. 3 Perbandingan spesifikasi alat kompresi

Parameter	Bobot (%)	Raspberry Pi Zero	Raspberry Pi Pico	ESP32
CPU	30	1GHz quad-core 64-bit Arm Cortex-A53	133 MHz Dual-core ARM Cortex-M0	Dual-core Xtensa LX6, 240 MHz
		0,3	0	0
Memori	25	512 MB SDRAM	264 KB SRAM	520 KB SRAM
		0,25	-0,25	0

Kapabilitas	25	Linux (Raspberry Pi OS)	Micropython	ESP-IDF, Arduino, FreeRTOS
		0,25	0	0
Harga	20	Rp295.000	Rp100.000	Rp70.000
		-0,2	0,2	0,2
Total Nilai	100	0,6	-0,05	0,2

4. LoRa Transceiver



Gambar 3.9 Modul LoRa SX1276

Modul SX1276 digunakan untuk transmisi data ECG dari ESP32 ke LoRa Gateway. Modul ini dirancang untuk komunikasi jarak jauh dengan konsumsi daya yang sangat rendah. Perangkat ini dipilih setelah dilakukan perbandingan spesifikasi dengan komponen serupa pada Tabel 3.4.

Tabel 3.4 Perbandingan spesifikasi LoRa Transceiver

Parameter	Bobot (%)	SX1276	SX1262	SX127x
Konsumsi Daya	20	10 mA	6 mA	10 mA
		0	0,2	0
Maksimum Data Rate	20	100 kbps	300 kbps	37.5 kbps
		0	0,2	-0,2
Rentang Frekuensi	10	137 MHz - 1020 MHz	150 MHz - 960 MHz	137 MHz - 1020 MHz
		0,1	0	0
Jarak Jangkau	20	Hingga 10 km	Hingga 15 km	Hingga 10 km

		0	0,2	0
Harga	30	Rp80.000	Rp300.000	Rp80.000
		0,3	-0,3	0,3
Total	100	0,4	0,3	0,1

5. LoRa Gateway



Gambar 3.10 Modul LoRa gateway SX1303

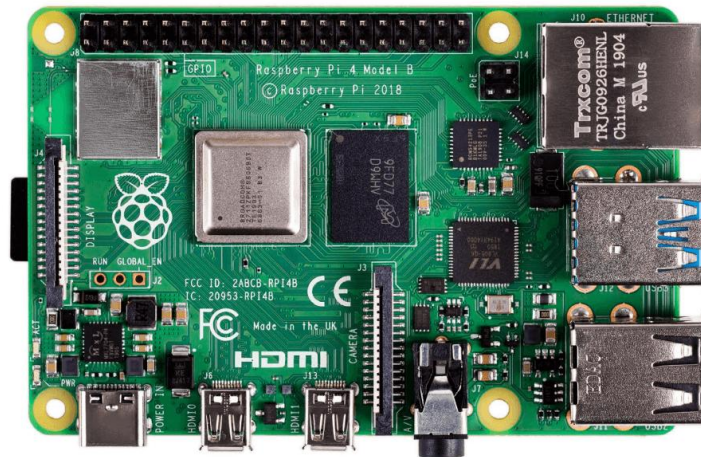
SX1303 adalah *gateway* LoRa yang digunakan untuk menerima data dari banyak perangkat LoRa dan mengirimkannya ke *server*. *Gateway* ini mendukung komunikasi dalam skala besar dengan latensi rendah. Perangkat ini dipilih setelah dilakukan perbandingan spesifikasi dengan komponen serupa pada Tabel 3.5.

Tabel 3.5 Perbandingan spesifikasi LoRa *gateway*

Parameter	Bobot (%)	SX1303	RFM95	RAK2287
Jumlah Kanal	25	8 LoRa, 1 FSK	1 LoRa	8 LoRa, 1 FSK
		0,25	-0,25	0,25
Konsumsi Daya	25	~115 mA	~120 mA	~1000 mW (1 W)
		0,25	0	-0,25
Jangkauan Frekuensi	20	150–960 MHz	137–1020 MHz	150–960 MHz
		0	0,2	0
Sensitivitas	20	-141 dBm	-137 dBm	-139 dBm
		0,2	-0,2	0

Konektivitas	10	SPI/I2C	SPI	SPI/UART
		0,1	0	0,1
Total	100	0,8	-0,25	0,1

6. Mikrokomputer



Gambar 3.11 Mikrokomputer Raspberry Pi 5

Raspberry Pi 5 digunakan sebagai *server* lokal untuk menyimpan data sementara sebelum dikirimkan ke *cloud*, sekaligus menjalankan aplikasi visualisasi data. Perangkat ini dipilih setelah dilakukan perbandingan spesifikasi dengan komponen serupa pada Tabel 3.6.

Tabel 3.6 Perbandingan spesifikasi mikrokomputer

Parameter	Bobot (%)	Raspberry Pi 5 Model B	Raspberry Pi 3 Model B+	Raspberry Pi 2 Model B
Performa Prosesor	30	Quad-core Cortex-A72, 1,5 GHz	Quad-core Cortex-A53, 1,4 GHz	Quad-core Cortex-A7, 900 MHz
		0,3	0	-0,3
Kapasitas RAM	25	2GB/4GB/8GB LPDDR4	1GB LPDDR2	1GB LPDDR2
		0,25	0	-0,25

Konektivitas	20	USB 3.0, Gigabit Ethernet, Bluetooth 5.0	USB 2.0, Ethernet, Bluetooth 4.2	USB 2.0, Ethernet
		0,2	0	-0,2
Dukungan HDMI	10	Dual micro HDMI (4K)	<i>Full-size</i> HDMI	<i>Full-size</i> HDMI
		0,1	0	-0,1
Daya	5	USB-C (5V/3A)	Micro USB (5V/2.5A)	Micro USB (5V/2A)
		0	0,05	0,05
Penyimpanan	10	microSD/USB HDD/SSD	microSD	microSD
		0,1	0	0
Total	100	0,95	0,05	-0,8

7. Baterai

Sistem pemantauan ECG berbasis LoRaWAN dirancang untuk konsumsi daya rendah. Sistem ini dirancang untuk bekerja minimal dua hari tanpa penggantian baterai, dengan asumsi konsumsi daya total sekitar 200-300 mA, sehingga kapasitas baterai harus cukup besar. Serta tegangan operasi sistem berkisar antara 3.3V hingga 5V. Selain itu, karena sistem ini dirancang untuk pemantauan pasien, baterai yang digunakan harus ringan dan portabel untuk mendukung mobilitas dan kenyamanan pasien.

Tabel 3.7 Tabel perbandingan spesifikasi baterai

Parameter	Bobot	Baterai 18650	Baterai LiPo	Baterai NiMH
Tegangan	25	3,7	3,7V	1,2V
		0,25	0,25	0
Kapasitas	25	3500mAh	2000mAh	2200mAh
		0,25	0,25	0,25
Daya tahan siklus	25	500-1000	300-500	200-300
		0,25	0	-0,25
Harga	25	Rp75.000	Rp75.000	Rp75.000

		0,25	0,25	0,25
Total	100	1	0,75	0,25



Gambar 3.12 Baterai 18650

Berdasarkan kelebihan masing-masing jenis baterai, baterai 18650 menjadi pilihan terbaik untuk digunakan. Baterai 18650 memiliki tegangan sebesar 3.7V, yang sesuai dengan kebutuhan sistem. Selain itu, baterai ini menawarkan kapasitas tinggi.

8. Modul pengisian daya



Gambar 3.13 Modul pengisian daya IP5306

Modul TP5100 digunakan untuk mengisi daya baterai Li-Po. Modul ini dirancang untuk memastikan pengisian daya yang aman dan efisien, serta dilengkapi dengan proteksi *overcharge*.

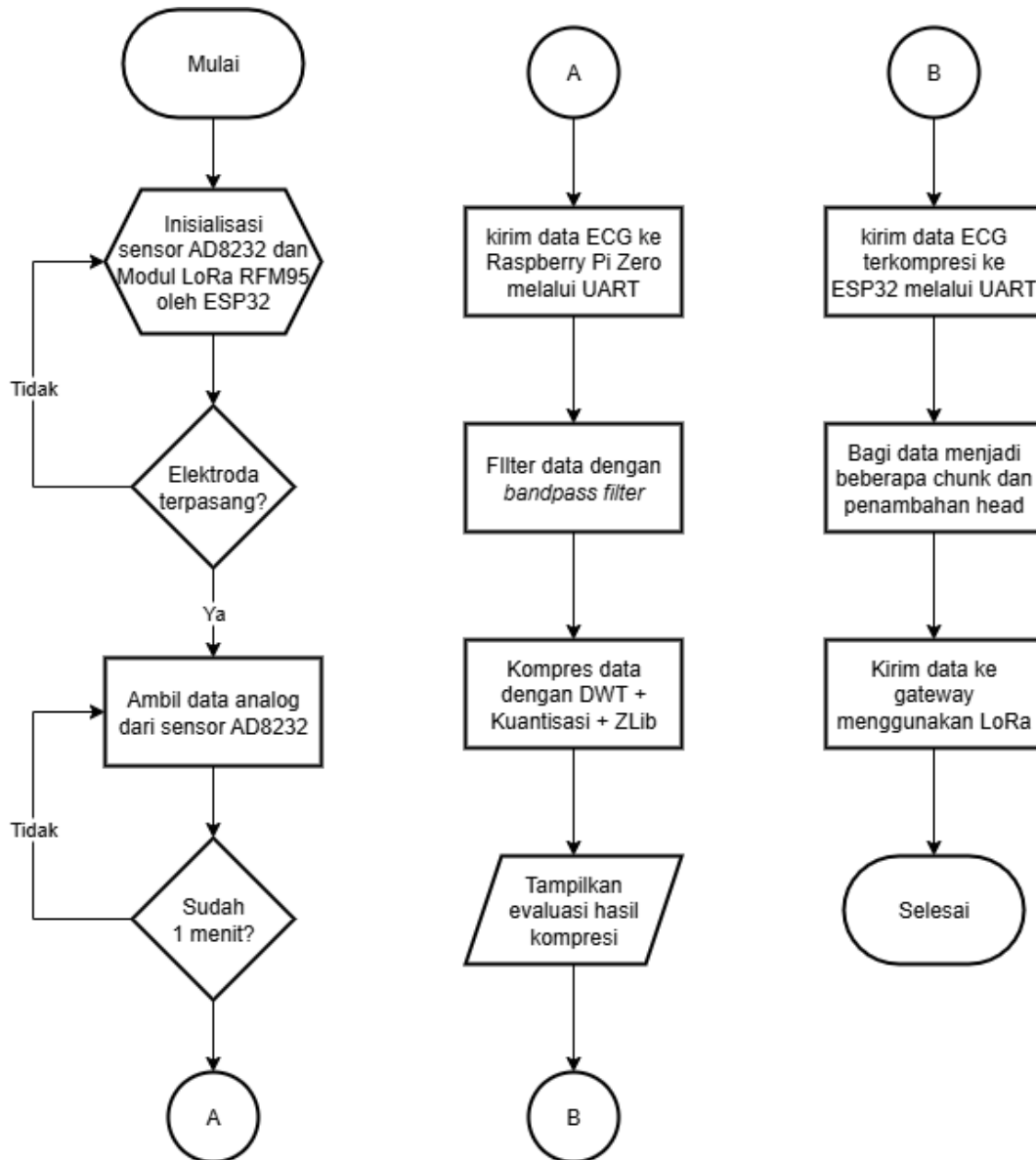
Tabel 3.8 Perbandingan spesifikasi modul pengisian daya

Fitur	Bobot (%)	TP5100	TP4056	IP5306
Arus Pengisian Maksimum	25	2A	1A	2A
		0,25	0	0,25
Step Up 5V	25	Tidak ada	Tidak ada	Ada
		0	0	0,25

Tipe	15	Type-C	Micro USB / Type-C	Type-C
		0,15	0,15	0,15
Tipe Baterai	10	Lithium- ion/Polymer	Lithium- ion/Polymer	Lithium- ion/Polymer
		0,1	0,1	0,2
Tegangan Pengisian	15	4.2V	4.2V	4,2V
		0,15	0,15	0,15
Harga	10	Rp20.000	Rp5.000	Rp20.000
		0,1	0,1	0,1
Total	100	0,75	0,5	1

3.3 Desain Perangkat Lunak

Desain perangkat lunak akan dirancang ke dalam 2 subsistem. Subsistem akan terdiri dari *End-Node* yang bertugas dalam pengambilan ECG, kompresi data, dan pengiriman data dengan LoRa, serta MQTT *Gateway* yang bertugas untuk menerima data, melakukan penyimpanan data pada database, dan menampilkan data pada antarmuka aplikasi web.



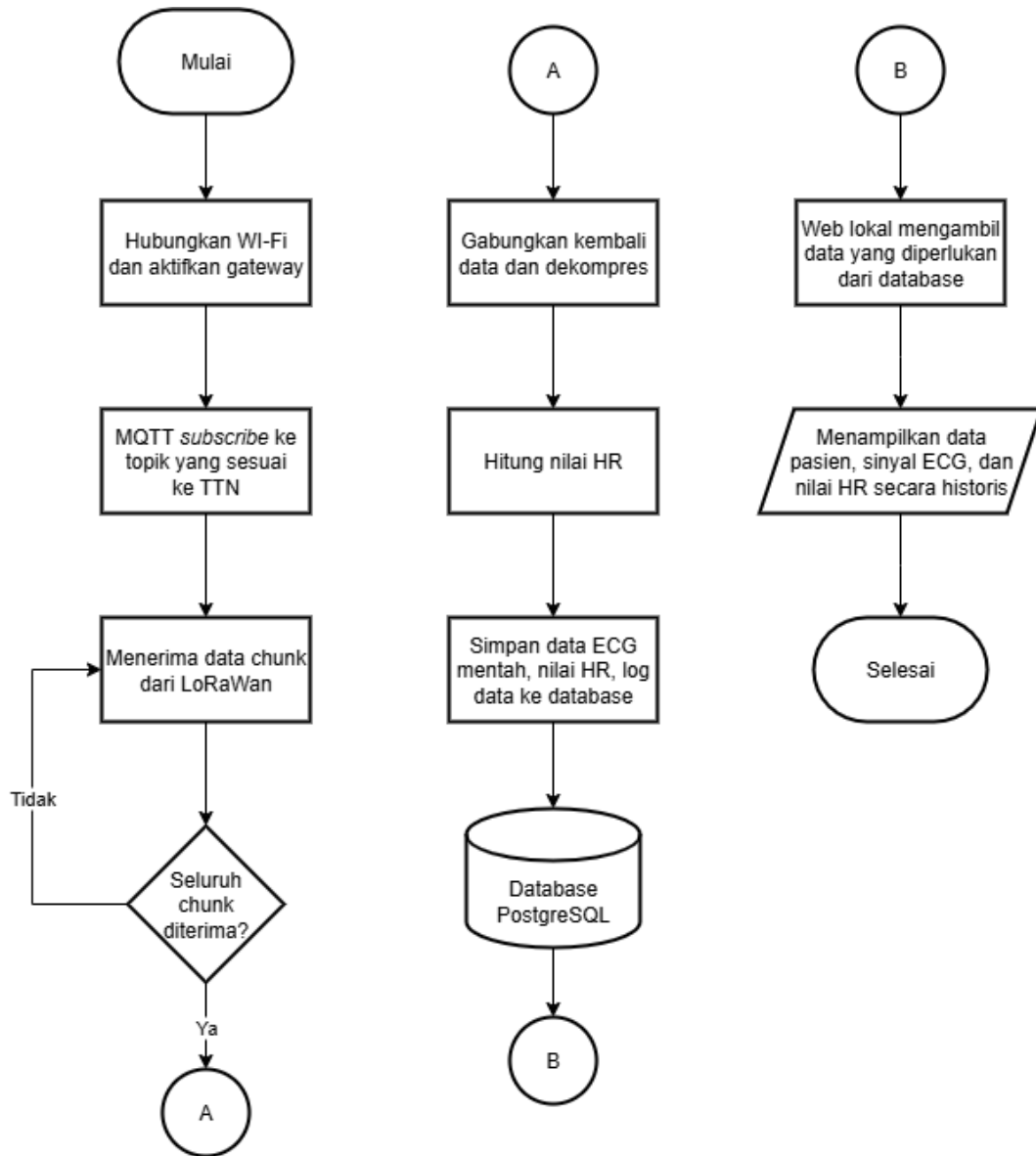
Gambar 3.14 *Flowchart End-Node*

Flowchart pada Gambar 3.14 mengilustrasikan proses akuisisi, pra-pemrosesan, dan transmisi data *electrocardiogram* (ECG) pada perangkat node akhir, yang dalam implementasi ini adalah mikrokontroler ESP32. Proses dimulai dengan inisialisasi sistem, termasuk konfigurasi komunikasi serial (UART dan LoRa) serta pin-pin input untuk sensor ECG dan deteksi *lead-off*. Setelah inisialisasi, sistem memasuki siklus pengukuran ECG selama durasi yang telah ditentukan (60 detik).

Selama siklus pengukuran, mikrokontroler secara kontinu memantau status *lead-off* untuk memastikan elektroda terpasang dengan benar. Jika terjadi *lead-off*,

akuisisi data akan dihentikan sementara. Apabila tidak ada *lead-off*, data analog dari sensor ECG akan dibaca dan disimpan ke dalam *buffer* internal pada laju sampel yang telah ditentukan (125 Hz). Proses ini berlanjut hingga *buffer* penuh atau durasi pengukuran tercapai.

Setelah data ECG terkumpul, sistem akan menandai bahwa data siap untuk dikirim. Data ECG mentah kemudian dikirimkan secara serial melalui UART ke Raspberry Pi Zero (RasPi0) untuk proses kompresi. Mikrokontroler akan menunggu respons dari RasPi0 yang berisi data ECG yang telah terkompresi dalam format heksadesimal. Setelah menerima data terkompresi, mikrokontroler akan mengurai dan mengubahnya kembali menjadi *byte array*. Data terkompresi ini kemudian dikirimkan melalui modul LoRa secara *chunked* (terbagi menjadi beberapa bagian kecil) untuk memastikan transmisi yang efisien dan sesuai dengan batasan ukuran *payload* LoRaWAN. Setiap *chunk* LoRa akan menyertakan *session ID*, nomor *chunk*, dan total *chunk* untuk memfasilitasi rekombinasi di sisi penerima. Setelah semua *chunk* terkirim, sistem akan mereset *buffer* dan memulai siklus pengukuran yang baru.



Gambar 3.15 *Flowchart* MQTT gateway

Flowchart pada Gambar 3.15 menjelaskan alur kerja gateway MQTT, yang bertanggung jawab untuk menerima data LoRaWAN dari node akhir, mendekode, memproses, dan menyimpannya ke dalam basis data, serta melakukan logging sistem. Gateway ini diimplementasikan menggunakan skrip Python yang berjalan pada platform yang terhubung ke jaringan The Things Network (TTN) melalui protokol MQTT.

Proses dimulai dengan inisialisasi klien MQTT dan koneksi ke broker MQTT TTN. Setelah berhasil terhubung, gateway akan berlangganan topik-topik uplink LoRaWAN yang relevan dari perangkat node akhir. Ketika pesan uplink diterima,

gateway akan mengekstrak *payload* LoRaWAN yang berisi data ECG terkompresi. Karena data dikirim secara *chunked* oleh node akhir, gateway akan mengumpulkan semua *chunk* berdasarkan *session ID* dan *device ID* hingga *payload* lengkap berhasil direkonstruksi.

Setelah *payload* lengkap terbentuk, data akan diproses. Ini melibatkan pemisahan data terkompresi dari metadata kompresi. Data ECG terkompresi kemudian didekompresi menggunakan algoritma *Discrete Wavelet Transform* (DWT) dan proses dekuantisasi. Sinyal ECG yang telah didekompresi akan melalui tahap pra-pemrosesan lebih lanjut, termasuk penyaringan *bandpass* untuk menghilangkan *noise* dan artefak. Selanjutnya, algoritma deteksi detak jantung (misalnya, Pan-Tompkins-like) diterapkan pada sinyal yang telah difilter untuk menghitung laju detak jantung (HR) secara *sliding window*.

Seluruh proses, mulai dari penerimaan pesan MQTT hingga penyimpanan data, dicatat ke dalam basis data PostgreSQL melalui fungsi *logging* khusus. Data HR dan sinyal ECG mentah yang telah diproses kemudian disimpan ke dalam tabel yang sesuai di basis data, dengan pembaruan informasi pasien terkait. Jika terjadi kesalahan pada tahap mana pun (misalnya, *payload* tidak lengkap, dekompresi gagal, atau masalah basis data), sistem akan mencatat pesan kesalahan dan, jika memungkinkan, menghapus *chunk* yang tidak lengkap untuk menghindari *memory leak*. Gateway terus beroperasi dalam mode *loop forever*, siap menerima dan memproses data uplink secara berkelanjutan.

BAB IV

HASIL PERCOBAAN DAN ANALISIS

4.1 Pengujian Sensor ECG

Pada bagian ini, dilakukan pengujian terhadap sistem pemantauan sinyal detak jantung yang telah dirancang. Pengujian ini bertujuan untuk mengevaluasi performa sistem dalam aspek-aspek yang telah ditentukan sebelumnya. Hasil pengujian disajikan dalam bentuk data untuk mendukung analisis lebih lanjut

4.1.1 Skenario Pengujian Sensor ECG

Pengujian dilakukan menggunakan sensor AD8232 yang berfungsi untuk menangkap sinyal listrik jantung (ECG) dari tubuh subjek manusia. Sensor ini terhubung ke mikrokontroler ESP32, yang melakukan pembacaan analog dengan frekuensi sampling sebesar 125 Hz. Dalam pengujian ini, digunakan 4 subjek yang masing-masing diuji dalam 2 kondisi berbeda, yaitu:

1. Kondisi Istirahat (*Resting Condition*):

Subjek berada dalam keadaan duduk tenang, rileks, dan tidak melakukan aktivitas fisik setidaknya 5 menit sebelum pengukuran dimulai. Tujuan dari kondisi ini adalah untuk memperoleh sinyal ECG yang stabil tanpa gangguan gerakan otot.

2. Kondisi Setelah Aktivitas (*Post-Activity Condition*):

Subjek diminta melakukan aktivitas fisik ringan terlebih dahulu, seperti naik-turun tangga selama 1 menit, kemudian dilakukan pengukuran. Tujuan dari kondisi ini adalah untuk melihat performa sensor dalam kondisi sinyal yang lebih dinamis dan kemungkinan adanya gangguan gerakan (*motion artifact*). Data keempat subjek tersebut dapat dilihat pada tabel 4.1 berikut:

Tabel 4.1 Data subjek untuk pengambilan data

Subjek	Usia	Jenis Kelamin	Pekerjaan	Kebiasaan Olahraga
1	23	Laki-laki	Mahasiswa	Jarang berolahraga
2	22	Laki-laki	Mahasiswa	Rutin angkat beban
3	22	Laki-laki	Mahasiswa	Rutin berlari
4	23	Laki-laki	Mahasiswa	Rutin latihan silat

Selama pengambilan data:

1. Subjek duduk dalam posisi rileks untuk meminimalkan gangguan artefak gerakan.
2. Bersihkan bagian yang akan dipasang elektroda dengan antiseptik alkohol dan tempelkan elektroda sesuai posisinya seperti pada gambar 4.1.
3. Data sinyal ECG pada serial monitor direkam dan disimpan dalam format csv selama 5 menit.
4. Hasil pengukuran detak jantung (*heart rate*) dihitung dari sinyal ini dan dibandingkan dengan alat pulse oximeter komersial sebagai referensi seperti pada gambar 4.2.

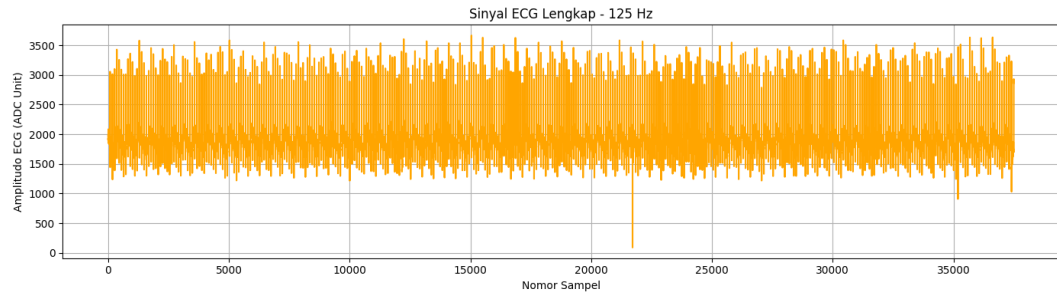


Gambar 4.1 Penempatan elektroda pada tubuh

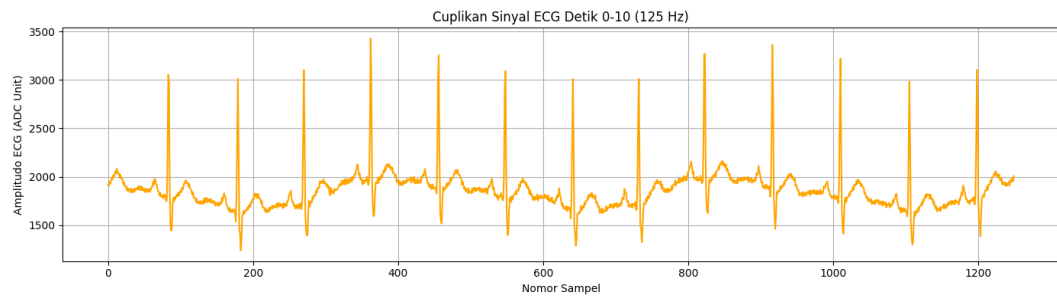


Gambar 4.2 Perekaman data HR dari *Pulse Oximeter*

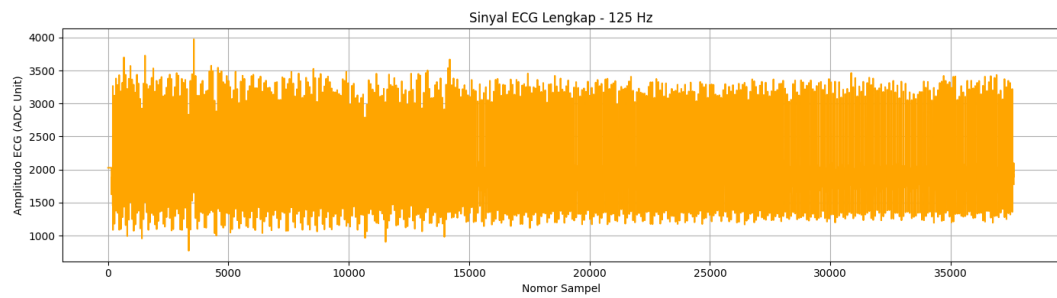
Hasil pengukuran sinyal ECG dapat dilihat pada gambar dibawah ini.



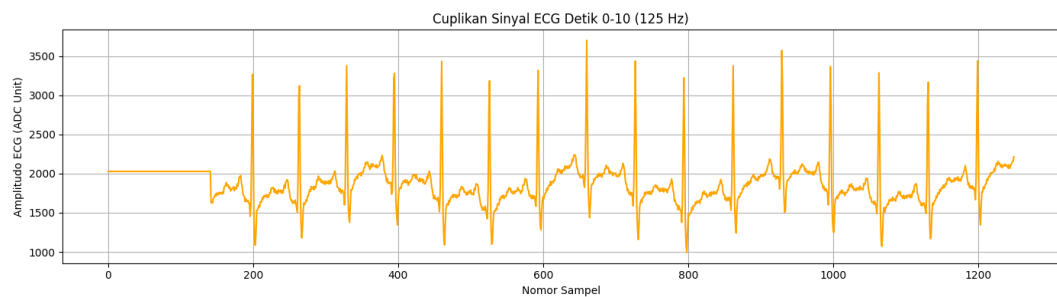
Gambar 4.3 Sinyal ECG subjek 1 kondisi 1



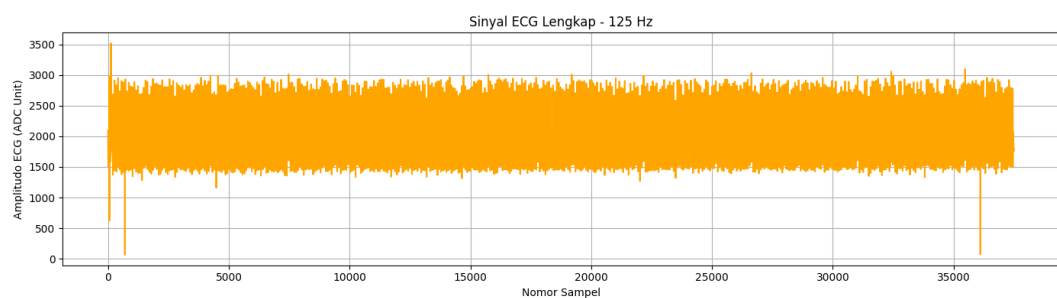
Gambar 4.4 Cuplikan sinyal ECG selama 10 detik subjek 1 kondisi 1



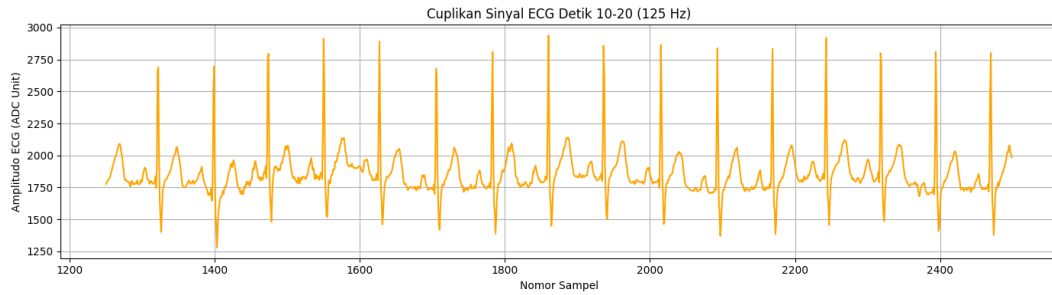
Gambar 4.5 Sinyal ECG subjek 1 kondisi 2



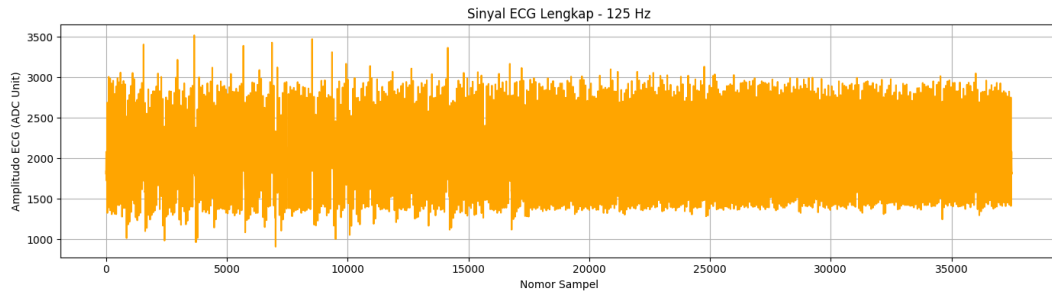
Gambar 4.6 Cuplikan sinyal ECG selama 10 detik subjek 1 kondisi 2



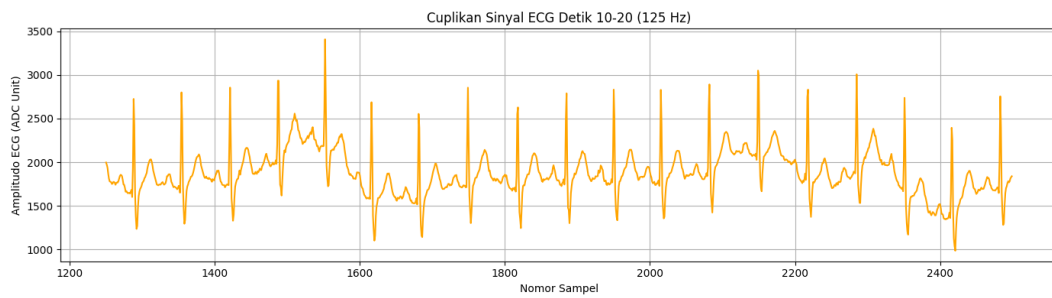
Gambar 4.7 sinyal ECG subjek 2 kondisi 1



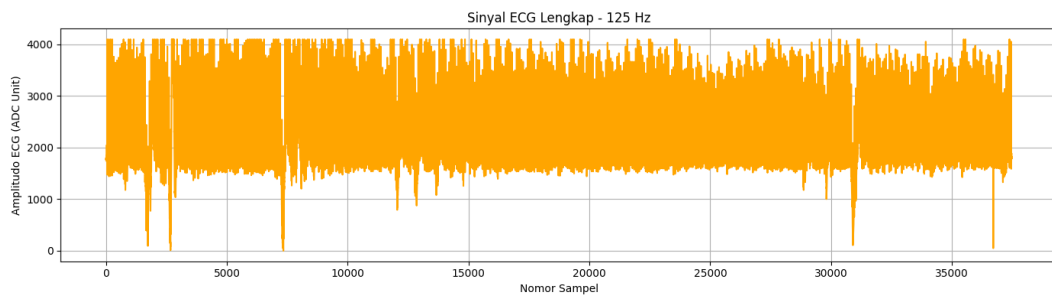
Gambar 4.8 Cuplikan sinyal ECG selama 10 detik subjek 2 kondisi 1



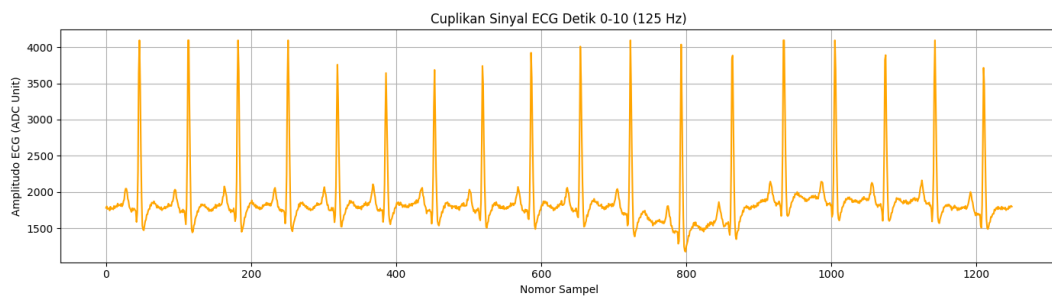
Gambar 4.9 sinyal ECG subjek 2 kondisi 2



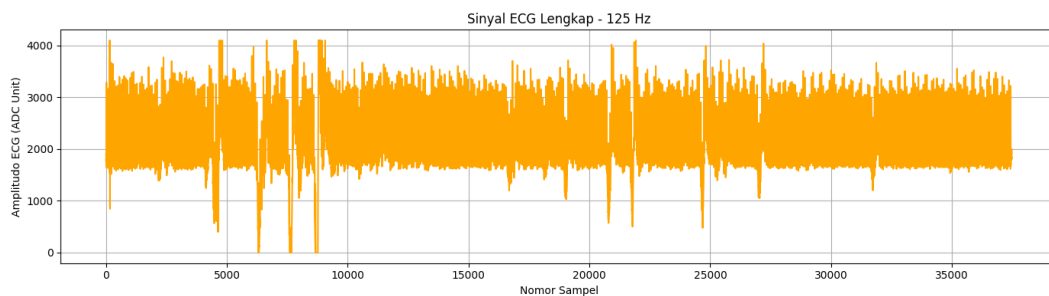
Gambar 4.10 Cuplikan sinyal ECG selama 10 detik subjek 2 kondisi 2



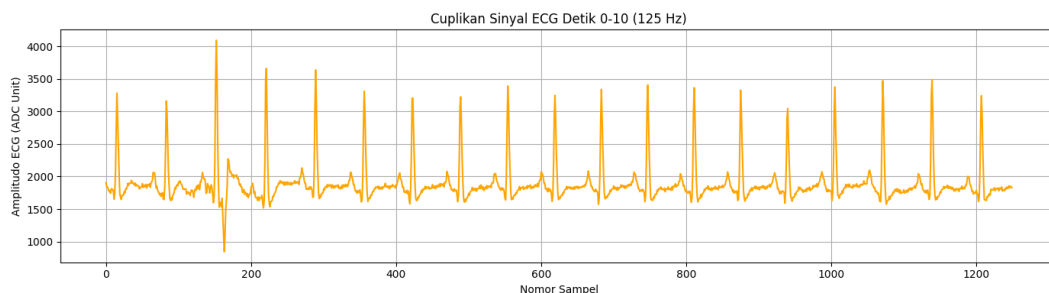
Gambar 4.11 sinyal ECG subjek 3 kondisi 1



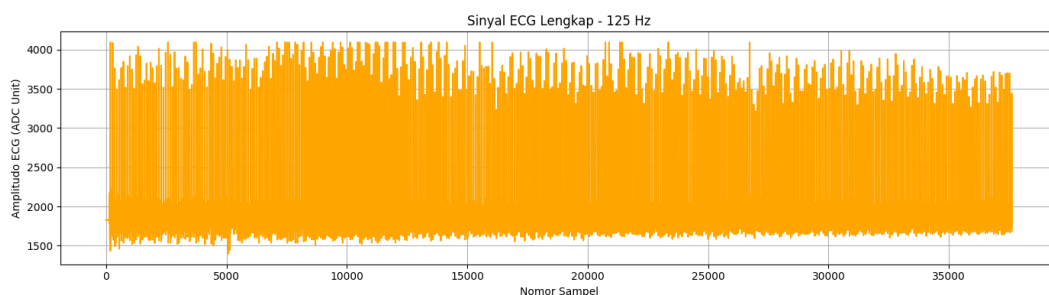
Gambar 4.12 Cuplikan sinyal ECG selama 10 detik subjek 3 kondisi 1



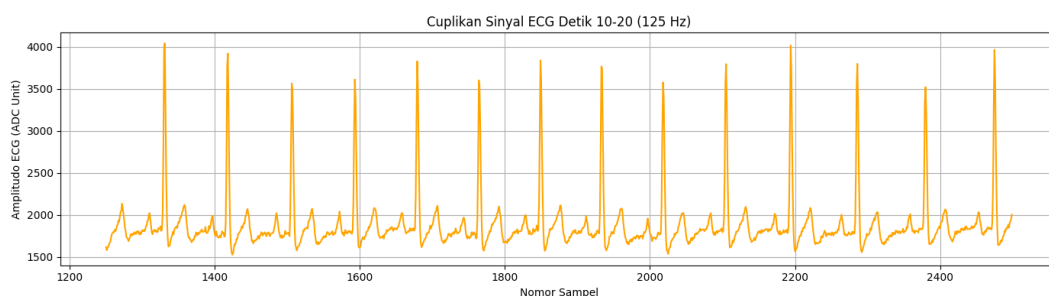
Gambar 4.13 sinyal ECG subjek 3 kondisi 2



Gambar 4.14 Cuplikan sinyal ECG selama 10 detik subjek 3 kondisi 2



Gambar 4.15 sinyal ECG subjek 4 kondisi 1



Gambar 4.16 Cuplikan sinyal ECG selama 10 detik subjek 4 kondisi 2

Gambar 4.3, 4.5, 4.7, 4.9, 4.11, 4.13, 4.15 menunjukkan sinyal ECG utuh selama kurang lebih 5 menit untuk masing-masing subjek dan kondisi. Terlihat bahwa pada beberapa bagian sinyal mengalami fluktuasi mendadak atau drop nilai amplitudo. Fenomena ini kemungkinan besar disebabkan oleh gangguan artefak gerakan (motion artifact), yaitu ketika otot tubuh subjek bergerak sehingga sinyal

listrik dari aktivitas otot (noise sinyal otot) ikut terbaca oleh sensor AD8232. Hal ini merupakan hal yang umum terjadi dalam pengukuran ECG berbasis elektroda permukaan, terutama pada subjek yang sulit untuk tetap diam selama proses akuisisi. Oleh karena itu, pembersihan sinyal (filtering) dan pemilihan waktu analisis menjadi penting dalam sistem ini agar deteksi puncak jantung tetap akurat.

Gambar 4.4, 4.6, 4.8, 4.10, 4.12, 4.14, 4.16 merupakan cuplikan sinyal ECG selama 10 detik ini memperlihatkan bentuk gelombang PQRST dari masing-masing subjek dalam kondisi berbeda. Terlihat bahwa bentuk dan morfologi gelombang ECG antar subjek tidak identik, meskipun pola dasar seperti kompleks QRS tetap muncul dengan jelas. Variasi ini bisa disebabkan oleh perbedaan posisi elektroda, struktur tubuh, aktivitas jantung individual, serta kualitas kontak elektroda dengan kulit. Namun secara umum, bentuk sinyal yang ditampilkan masih cukup representatif untuk digunakan dalam perhitungan *heart rate* (HR) dan proses deteksi puncak R. Kestabilan bentuk gelombang dalam durasi pendek ini juga menegaskan bahwa sistem mampu menangkap sinyal dengan baik dalam jangka waktu tertentu.

4.1.2 Hasil Pengujian Sensor ECG Sebelum Kalibrasi

Setelah sinyal ECG ditangkap menggunakan sensor AD8232, dilakukan perhitungan *Heart Rate* (HR) menggunakan algoritma deteksi puncak R (R-peak detection) dari metode Pan-Tompkins. Algoritma ini memproses sinyal ECG yang telah difilter untuk mengidentifikasi puncak-puncak R, yang merepresentasikan setiap detak jantung. Dari interval antar-puncak R (R-R interval), nilai HR dalam Beats Per Minute (BPM) kemudian dihitung. Algoritma ini bekerja dengan mencari puncak-puncak sinyal pada segmen QRS untuk menghitung interval antar detak jantung (R-R interval), yang kemudian dikonversi menjadi HR dalam satuan bpm.

Akurasi HR hasil perhitungan dari sensor ECG dievaluasi dengan membandingkannya terhadap nilai referensi HR dari pulse oximeter. Untuk menilai seberapa besar perbedaan antara kedua pengukuran tersebut, digunakan rumus:

$$Error (\%) = \left| \frac{HR_{Sensor} - HR_{Referensi}}{HR_{Referensi}} \right| \times 100\% \quad (4.1)$$

Tabel 4.2 Nilai HR sebelum kalibrasi

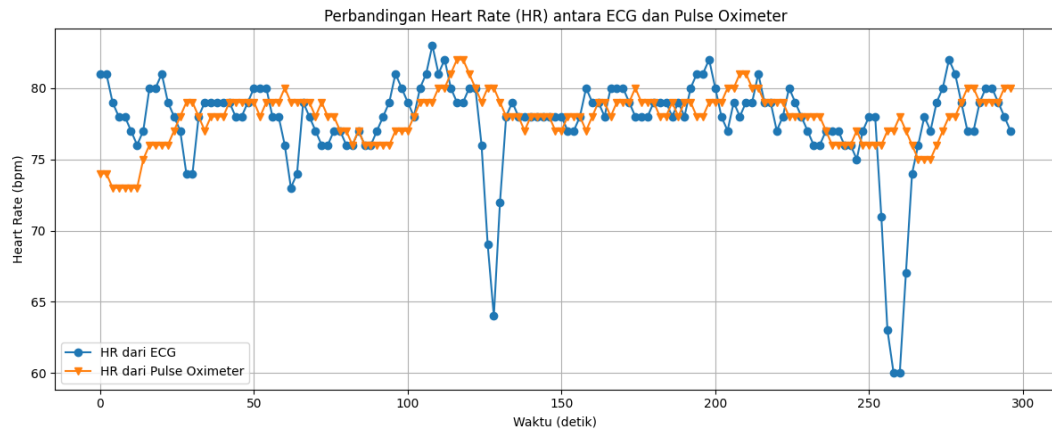
Subjek	Kondisi	Rata-rata HR Sensor (BPM)	Rata-rata HR Pulse Oximeter (BPM)	Rata-rata selisih HR (BPM)	Selisih maksimum (BPM)	Rata-rata Akurasi (%)
1	1	77,64	77,93	-0,29	18	96,91
1	2	95,03	96,65	-1,62	19	97,20
2	1	97,32	98,13	-0,81	21	97,27
2	2	102,78	104,44	-1,66	20	98,41
3	1	111,17	111,49	-0,32	23	97,23
3	2	108,4	109,93	-1,53	22	97,07
4	1	88,63	88,68	-0,05	18	96,49
4	2	96,07	97,14	-1,07	17	96,75

Data HR pada Tabel 4.2 dan grafik-grafik perbandingan HR pada Gambar 4.17 hingga Gambar 4.24 dihitung dari hasil pengolahan sinyal HR masing-masing subjek dalam dua kondisi, yaitu istirahat dan setelah aktivitas. Visualisasi perbandingan HR antara ECG dan Pulse Oximeter untuk masing-masing subjek dapat dilihat pada Gambar 4.17 sampai Gambar 4.24, di mana tren perbedaan dan akurasi antar perangkat tergambar dengan jelas.

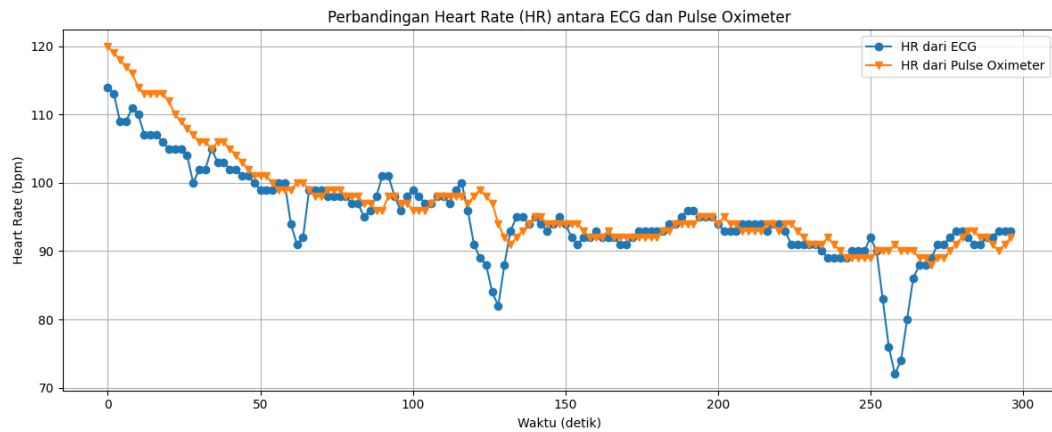
Berdasarkan Tabel 4.2, dapat dilihat bahwa rata-rata selisih HR antara Pulse Oximeter dan ECG untuk seluruh subjek berada di kisaran -0,05 hingga -1,66 BPM. Nilai negatif menunjukkan kecenderungan pembacaan HR dari Pulse Oximeter sedikit lebih tinggi dibandingkan HR dari ECG, namun selisihnya tergolong sangat kecil (kurang dari 2 BPM), sehingga masih dalam batas wajar. Selisih maksimum berkisar antara 17 hingga 23 BPM, yang umumnya terjadi akibat artefak sinyal atau perbedaan metode pengukuran detak jantung.

Dari segi akurasi, nilai rata-rata akurasi Pulse Oximeter terhadap ECG berada di atas 96%, dengan nilai tertinggi mencapai 97,27% pada Subjek 2 kondisi istirahat. Hal ini menunjukkan bahwa secara keseluruhan, Pulse Oximeter dapat

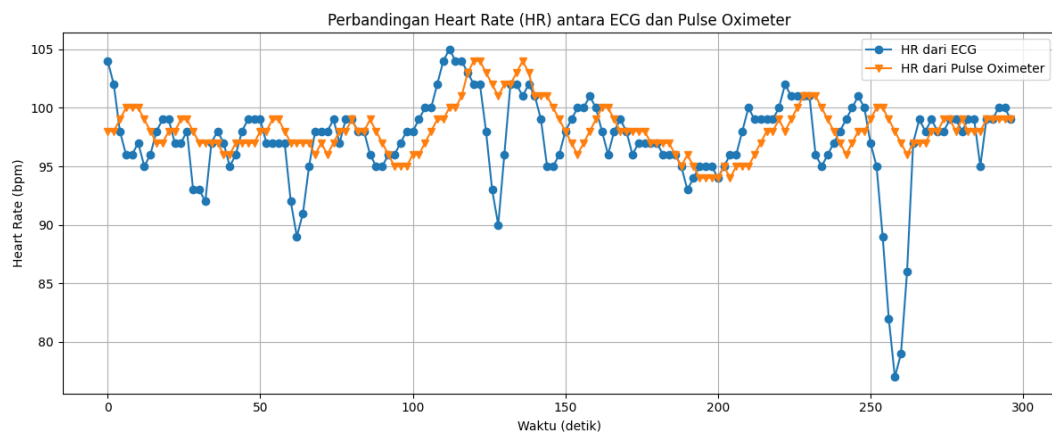
memberikan estimasi HR yang cukup akurat dan konsisten jika dibandingkan dengan hasil pengukuran HR menggunakan sinyal ECG.



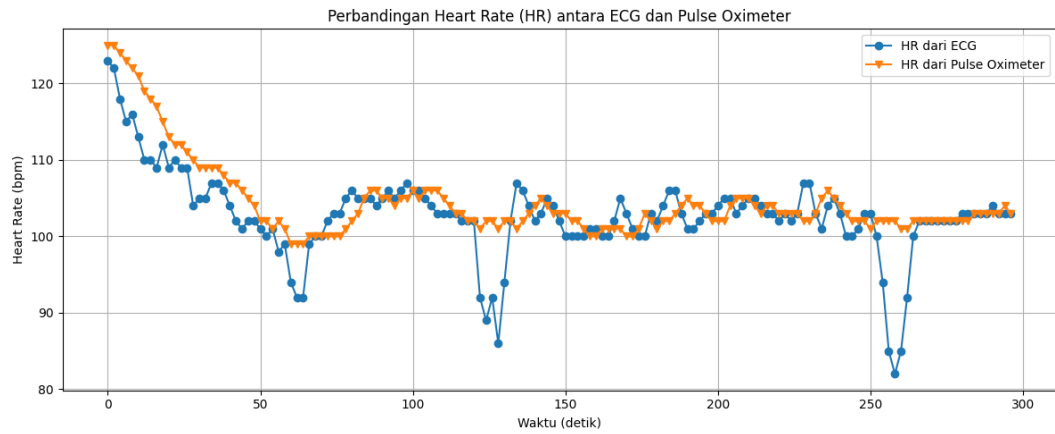
Gambar 4.17 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 1 kondisi 1



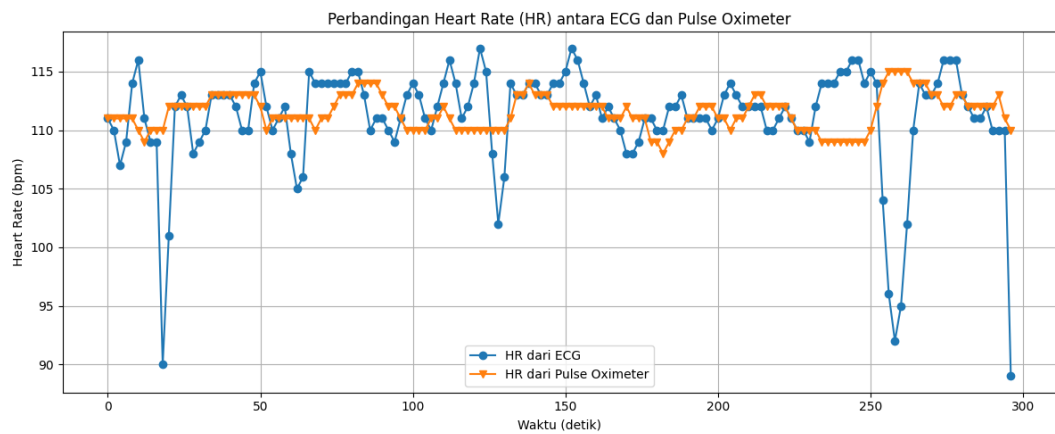
Gambar 4.18 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 1 kondisi 2



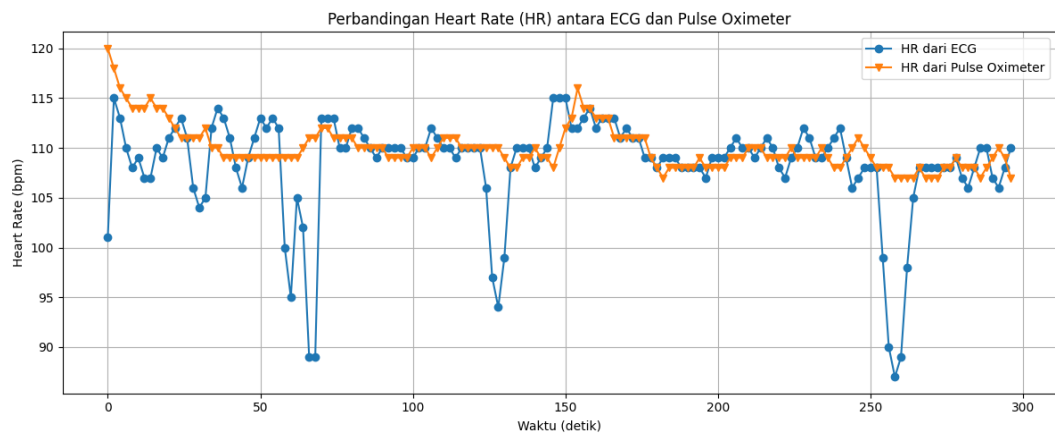
Gambar 4.19 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 2 kondisi 1



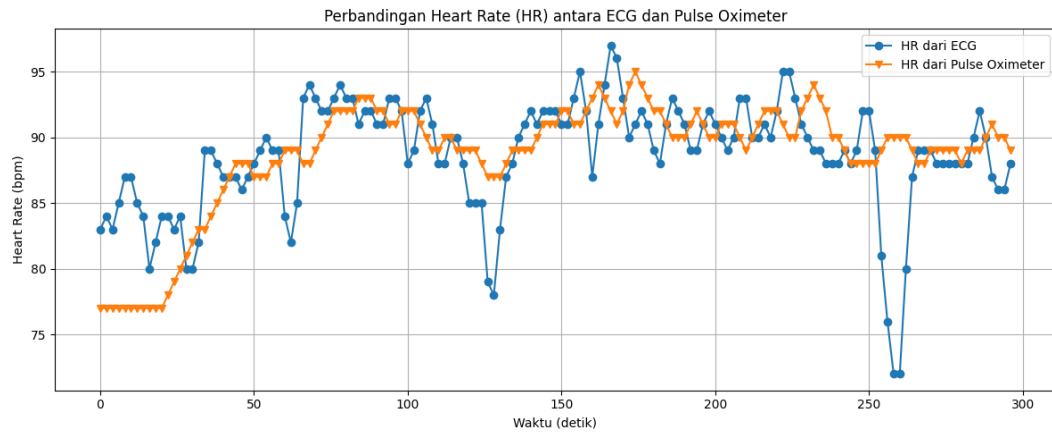
Gambar 4.20 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 2 kondisi 2



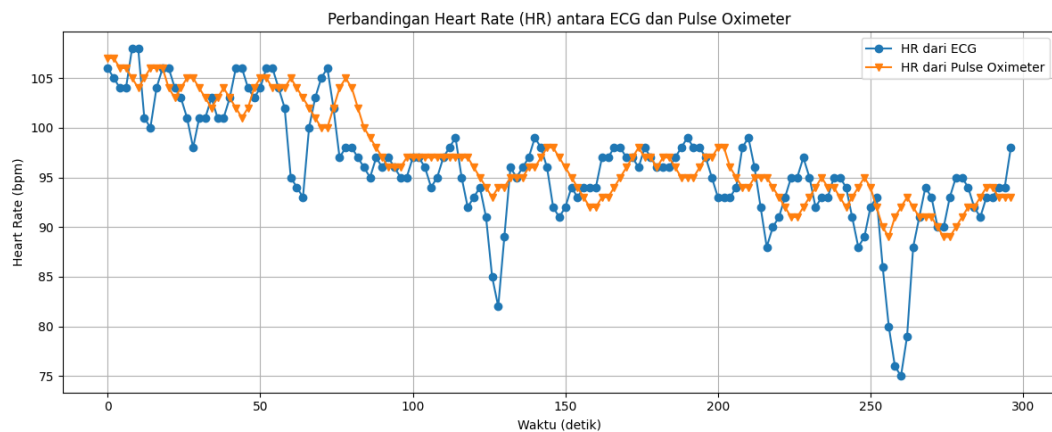
Gambar 4.21 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 3 kondisi 1



Gambar 4.22 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 3 kondisi 2



Gambar 4.23 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 4 kondisi 1



Gambar 4.24 Perbandingan nilai HR *Pulse Oximeter* dengan HR dari sensor pada subjek 4 kondisi 2

Gambar 4.17 sampai Gambar 4.24 menunjukkan perbandingan antara nilai *heart rate* (HR) yang dihitung dari sinyal ECG menggunakan algoritma Pan-Tompkins dengan HR yang diperoleh dari alat pulse oximeter sebagai referensi. Terlihat bahwa secara umum, pola HR antara kedua metode menunjukkan tren yang serupa, namun pada beberapa titik terjadi penurunan atau perbedaan nilai HR dari ECG. Penurunan atau deviasi nilai HR dari ECG dapat disebabkan oleh beberapa faktor, di antaranya:

1. Gangguan artefak gerakan (*motion artifact*): Pergerakan tubuh, pernapasan dalam, atau kontraksi otot lain di sekitar dada bisa memengaruhi kualitas sinyal ECG. Hal ini mengganggu deteksi puncak R yang digunakan dalam algoritma Pan-Tompkins, sehingga menyebabkan HR yang dihitung menjadi tidak akurat pada segmen tersebut.

2. Perbedaan metode pengukuran: Pulse oximeter mengukur HR berdasarkan fluktuasi volume darah (fotopletismografi), sedangkan ECG mengandalkan deteksi puncak sinyal listrik jantung. Perbedaan metode ini dapat menyebabkan perbedaan nilai HR, terutama pada saat detak jantung tidak stabil atau terdapat noise.
3. *Delay* dan resolusi waktu: Pulse oximeter biasanya menampilkan nilai HR sebagai rata-rata selama interval tertentu, sedangkan algoritma Pan-Tompkins bisa sensitif terhadap satu atau dua detak jantung yang tidak terdeteksi, sehingga fluktuasi nilai HR dari ECG bisa lebih tajam.

Meski demikian, hasil menunjukkan bahwa perbedaan HR antara ECG dan Pulse Oximeter masih dalam kisaran yang wajar (rata-rata selisih < 2 bpm) dengan akurasi di atas 96%, yang menunjukkan bahwa sistem mampu memberikan estimasi HR yang cukup andal untuk keperluan *monitoring* pasien dalam konteks non-diagnostik.

4.1.3 Hasil Pengujian Sensor ECG sesudah kalibrasi

Setelah memperoleh data *heart rate* (HR) dari sensor ECG, dilakukan proses kalibrasi regresi linier terhadap nilai referensi dari pulse oximeter. Kalibrasi ini bertujuan untuk memperkecil error absolut maupun relatif (dalam persen) antara kedua pengukuran. Metode ini menentukan hubungan matematis antara HR Sensor (x) dengan HR Pulse Oximeter (y), dengan *slope* (a) dan *intercept* (b).

$$y = ax + b \quad (4.2)$$

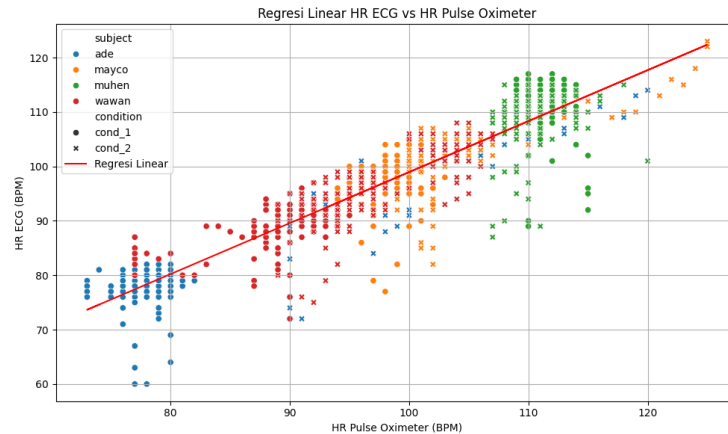
Untuk menghitung kemiringan atau *slope* (a) dari regresi linear, digunakan persamaan 4.3.

$$a = \frac{N \sum(xy) - \sum x \sum y}{N \sum x^2 - (\sum x)^2} \quad (4.3)$$

Selanjutnya, digunakan persamaan 4.4 untuk nilai *intercept* (b).

$$b = \frac{\sum y - a \sum x}{N} \quad (4.4)$$

Dari persamaan di atas, maka diperoleh nilai $a = 0.0466$ dan $b = 102.5792$.

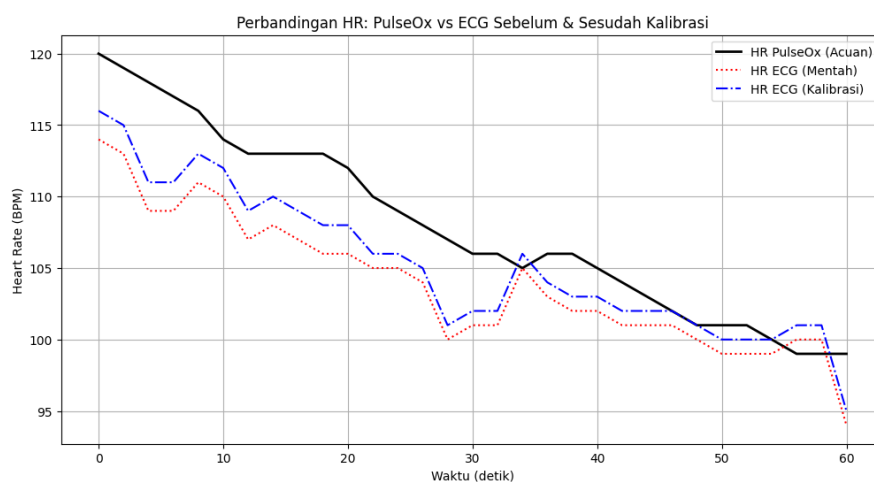


Gambar 4.25 Grafik regresi linear dari data yang telah didapatkan

Gambar 4.25 menunjukkan grafik regresi linear antara nilai *heart rate* (HR) dari sensor ECG terhadap HR referensi dari pulse oximeter. Dari hasil perhitungan diperoleh nilai slope (a) sebesar 0.0466 dan intercept (b) sebesar 102.5792, yang membentuk persamaan regresi:

$$y = 0.0466x + 102.5792 \quad (4.5)$$

Persamaan ini digunakan untuk mengkalibrasi output HR dari ECG agar lebih mendekati nilai dari pulse oximeter. Nilai slope yang kecil menunjukkan bahwa nilai HR dari ECG cenderung bervariasi sempit dan tidak langsung sebanding dengan referensi, sehingga perlu dikalibrasi. Nilai intercept yang tinggi menunjukkan adanya bias awal yang signifikan.



Gambar 4.26 Hasil pengujian HR dengan menggunakan regresi linear

Gambar 4.26 menunjukkan hasil pengujian HR setelah kalibrasi menggunakan persamaan di atas. Terlihat bahwa nilai HR yang dihasilkan lebih mendekati nilai dari pulse oximeter dan lebih konsisten dibandingkan sebelum kalibrasi. Ini menandakan bahwa regresi linear cukup efektif dalam memperbaiki akurasi pembacaan HR dari ECG.

Meskipun kalibrasi meningkatkan kesesuaian data, keterbatasan tetap ada, seperti ketergantungan terhadap kualitas sinyal input dan validitas model regresi hanya pada range data tertentu. Namun, pendekatan ini tetap berguna untuk mengurangi kesalahan sistematis yang terjadi akibat variasi subjek dan metode deteksi berbeda antara ECG dan pulse oximeter.

Selain mengacu pada pulse oximeter sebagai referensi, pengujian HR secara manual dilakukan dengan menghitung denyut nadi subjek secara langsung. Setiap subjek diminta duduk dalam keadaan tenang, lalu denyut nadi dihitung selama 15 detik dan dikalikan 4 untuk mendapatkan nilai dalam satuan BPM. Hasil pengukuran ini dibandingkan dengan pembacaan dari sensor ECG sebelum dan sesudah kalibrasi.

Langkah-Langkah Pengujian HR Manual dengan Hitung Denyut Nadi:

1. Subjek duduk dalam keadaan rileks selama 5 menit sebelum pengukuran.
2. Posisi tangan dan tubuh dibuat tetap untuk menghindari gerakan otot yang memengaruhi hasil.
3. Sensor ECG dan Pulse Oximeter dipasang ke tubuh subjek.
4. Pengukuran HR dilakukan secara bersamaan dengan 3 metode:
 - Sensor ECG (sebelum dan sesudah kalibrasi)
 - Pulse Oximeter
 - Manual (menghitung denyut nadi selama 15 detik lalu dikalikan 4)
5. Penghitungan denyut dilakukan oleh dua penguji secara bergantian untuk menghindari bias.
6. Proses dilakukan minimal 2 kali untuk konsistensi.
7. Bandingkan nilai HR dari ketiga metode.

8. Hitung selisih rata-rata HR dan akurasi dibandingkan Pulse Oximeter.

Tabel 4.3 Hasil pengujian perhitungan HR manual

Subjek	HR ECG (Sebelum Kalibrasi)	HR ECG (Setelah Kalibrasi)	HR Manual (Nadi)	HR Pulse Oximeter	Selisih HR Manual - Pulse	Akurasi Manual (%)
1	78	80	81	82	-1	98.78%
2	91	94	96	95	1	98.95%
3	69	72	74	73	1	98.63%
4	103	100	99	98	1	98.98%
5	85	87	88	88	0	100.00%
6	76	78	79	80	-1	98.75%
7	97	99	100	99	1	98.99%
8	88	89	90	91	-1	98.90%

Hasil pengujian ditunjukkan pada Tabel 4.3, di mana terlihat bahwa nilai HR setelah kalibrasi memiliki kesesuaian yang lebih tinggi terhadap nilai referensi dibandingkan sebelum kalibrasi. Akurasi metode manual juga cukup tinggi, dengan rata-rata di atas 98% dengan selisih rata-rata HR ± 0.88 bpm, sehingga dapat dijadikan alat validasi tambahan terutama dalam kondisi keterbatasan alat referensi digital.

4.1.4 Analisis Pengujian sensor

Untuk menilai performa hasil kalibrasi HR sensor terhadap referensi (pulse oximeter), digunakan tiga metrik evaluasi utama:

1. Mean Absolute Error (MAE)

Mengukur rata-rata selisih absolut antara nilai hasil kalibrasi dengan referensi.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.6)$$

- y_i : Nilai HR referensi (pulse oximeter)
- $\hat{y}_i$: Nilai HR hasil kalibrasi
- n : Jumlah data

2. Root Mean Square Error (RMSE)

Mengukur seberapa besar rata-rata kuadrat dari kesalahan antara hasil kalibrasi dan referensi.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.7)$$

Lebih sensitif terhadap error besar karena kuadrat.

3. Koefisien Determinasi (R^2 Score)

Menilai seberapa baik hasil kalibrasi menjelaskan variasi dari data referensi.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4.8)$$

- $\bar{y}$: Rata-rata nilai referensi

Tabel 4.4 Tabel metrik evaluasi

Metrik Evaluasi	Nilai
MAE (bpm)	2.85
RMSE (bpm)	4.264
R^2 Score	0.857

Tabel 4.4 menunjukkan hasil evaluasi performa kalibrasi regresi linear terhadap nilai *heart rate* (HR) dari sensor ECG dengan membandingkannya terhadap HR referensi dari pulse oximeter. Proses perhitungan HR menggunakan algoritma Pan-Tompkins, diterapkan metode *sliding window* dengan ukuran window 6 detik dan step 2 detik. Pemilihan parameter ini didasarkan pada hasil pengujian yang menunjukkan bahwa konfigurasi ini memberikan detail yang cukup baik dalam menangkap variasi HR, sambil tetap menjaga tingkat kesalahan yang dapat diterima.

Berdasarkan hasil pengujian, diperoleh nilai Mean Absolute Error (MAE) sebesar 2,85 bpm yang menunjukkan bahwa secara rata-rata, sistem hanya mengalami selisih sekitar 2,85 denyut per menit dibandingkan alat referensi. Nilai ini tergolong rendah dan masih dalam batas toleransi yang dapat diterima untuk sistem pemantauan non-diagnostik. Nilai Root Mean Square Error (RMSE) sebesar 4,26 bpm sedikit lebih tinggi dari MAE, yang mengindikasikan adanya beberapa titik ekstrem error yang mungkin disebabkan oleh noise atau kesalahan deteksi

puncak R pada segmen sinyal ECG yang terganggu. Meskipun begitu, nilai ini masih termasuk rendah untuk sistem pemantauan berbasis sinyal biomedis.

Selanjutnya, nilai koefisien determinasi (R^2 Score) sebesar 0,857 menunjukkan bahwa sebesar 85,7% variasi HR dari alat pulse oximeter dapat dijelaskan oleh model kalibrasi berbasis sinyal ECG. Nilai ini mengindikasikan adanya hubungan linier yang cukup kuat antara kedua metode pengukuran. Dengan demikian, dapat disimpulkan bahwa proses kalibrasi menggunakan regresi linear mampu meningkatkan akurasi pembacaan HR dari sensor ECG. Secara keseluruhan, sistem yang dirancang telah menunjukkan performa yang cukup baik dalam estimasi HR dan layak digunakan dalam aplikasi pemantauan pasien.

4.2 Pengujian Kompresi Data ECG

4.2.1 Skenario Pengujian Kompresi Data ECG

Pengujian ini bertujuan untuk mengevaluasi efektivitas metode kompresi data pada sinyal ECG yang diperoleh dari sensor AD8232. Data mentah hasil akuisisi dikirim dari mikrokontroler ESP32 ke Raspberry Pi Zero melalui komunikasi UART, kemudian dikompresi dengan kombinasi algoritma transformasi dan kompresi:

1. Transformasi DWT (Discrete Wavelet Transform) tipe Haar, level 3 ditambah kuantisasi 8 bit dan zlib
2. Metode kombinasi LZMA (Lempel-Ziv-Markov chain Algorithm dengan kuantisasi 8 bit
3. DCT (Discrete Cosine Transform) dikombinasikan dengan kuantisasi 8 bit dan zlib
4. Metode kuantisasi 8 bit dengan LZ4

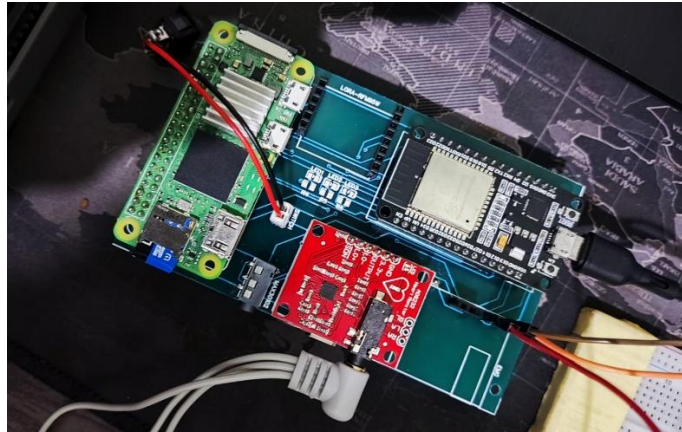
Rangkaian pengujian dilakukan dengan parameter sebagai berikut:

1. *Sampling Rate*: 125 Hz
2. Durasi Akuisisi: 1 menit
3. Format Data Awal: Integer 12-bit (rentang 0–4095)
4. Format Kompresi: Hasil kuantisasi dan encoding dikemas sebagai data biner, lalu dikonversi ke bentuk heksadesimal untuk ditransmisikan via LoRaWAN.

Setelah data dikompresi, sistem mencatat ukuran data sebelum dan sesudah kompresi untuk menghitung Compression Ratio (CR), serta menyimpan sinyal hasil dekompresi untuk mengevaluasi kualitas rekonstruksi sinyal menggunakan dua metrik:

1. PRD (Percent Root Difference): mengukur distorsi sinyal
2. CC (Correlation Coefficient): mengukur kesamaan bentuk sinyal

Pengujian ini dilakukan untuk menilai seberapa besar pengurangan ukuran data yang dapat dicapai tanpa mengorbankan kualitas sinyal yang signifikan, sehingga sistem dapat mengoptimalkan *bandwidth* transmisi LoRaWAN dalam aplikasi monitoring kesehatan.



Gambar 4.27 Kompresi data pada sistem dengan Raspberry Pi 0

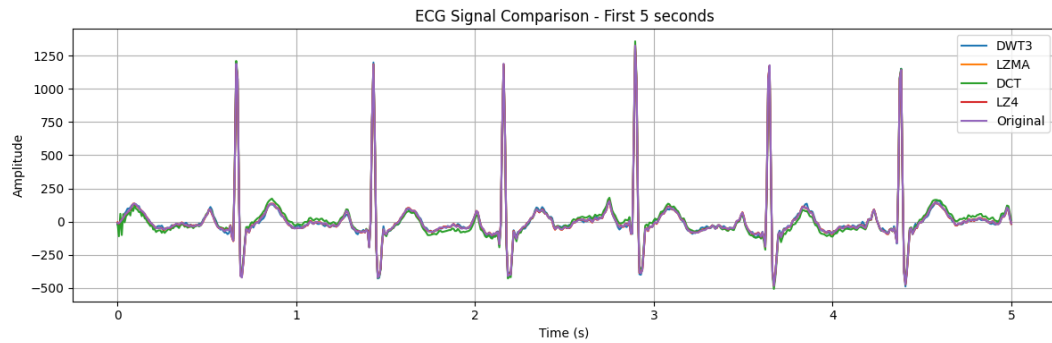
4.2.2 Hasil Pengujian Kompresi Data ECG

Pengujian dilakukan pada 4 metode kompresi yang akan digunakan sesuai dengan skenario pengujian. Hasil pengujian tersebut dapat dilihat pada tabel

Tabel 4.5 Tabel performa metode kompresi yang digunakan

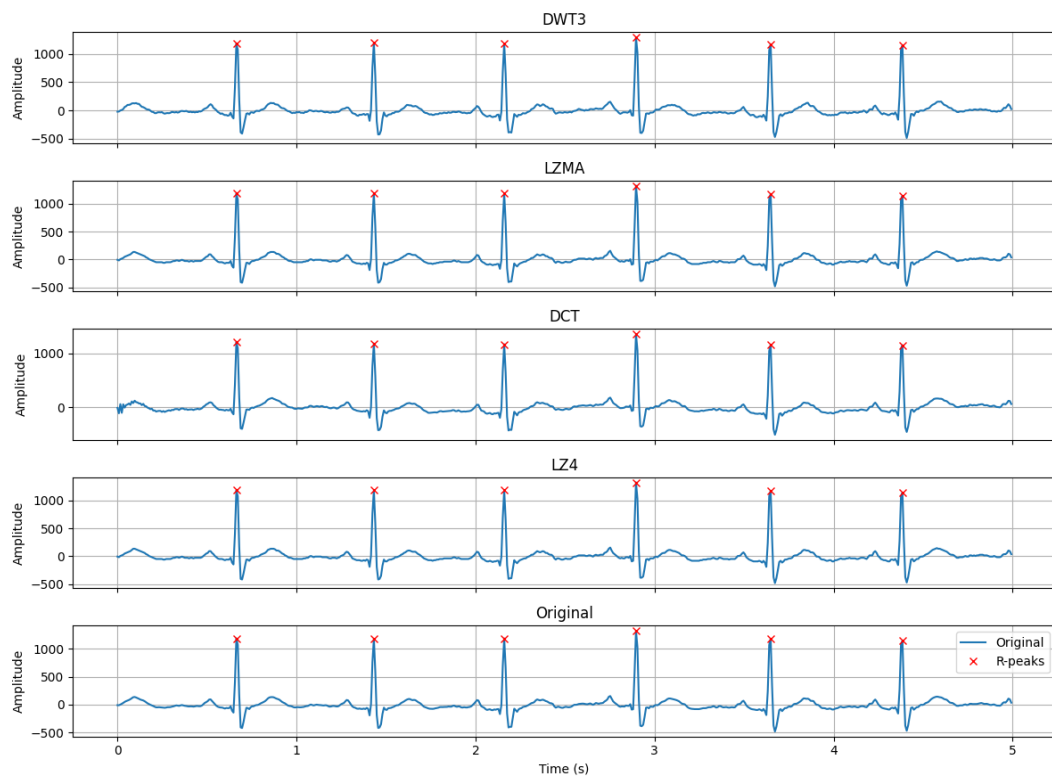
Parameter	DWT3 + Q8 + ZLIB	LZMA + Q8	DCT + Q8 + ZLIB	Quant + LZ4
<i>Compressed Size</i>	3840 bytes	4972 bytes	3359 bytes	7111 bytes
<i>Original Size</i>	15000 bytes	15000 bytes	15000 bytes	15000 bytes
<i>Compression %</i>	25.60%	33.15%	22.39%	47.41%

Compression Ratio	3.91x	3.02x	4.47x	2.11x
PRD (%)	2.98%	8.03%	29.63%	8.03%
CC (Correlation)	0.9996	0.9968	0.9558	0.9968

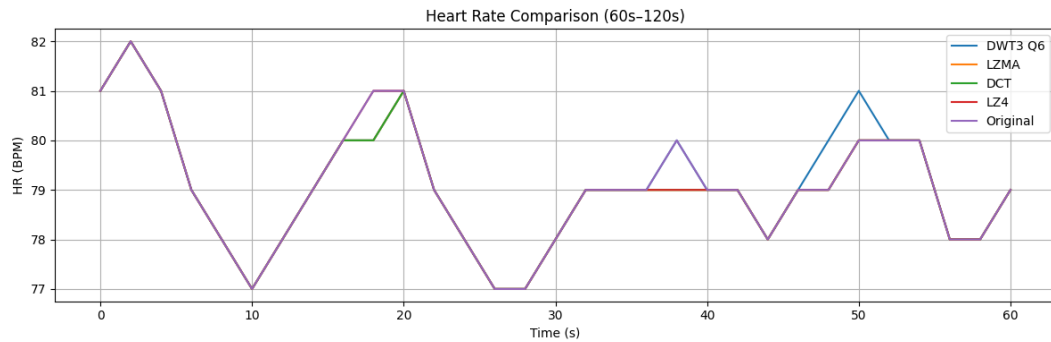


Gambar 4.28 Hasil dekompresi pada sinyal ECG pada data subjek 1

ECG Signal Comparison with R-peaks - First 5 seconds



Gambar 4. 29 Data ECG setelah kompresi dengan deteksi R-Peak



Gambar 4.30 Perbandingan hasil perhitungan HR pada data asli dengan data hasil dekompresi pada subjek 1

Berdasarkan Gambar 4.28, terlihat bahwa sinyal ECG hasil dekompresi dari berbagai metode (DWT3, LZMA, DCT, dan LZ4) masih mempertahankan tren morfologi yang serupa dengan sinyal ECG asli. Hal ini menunjukkan bahwa struktur gelombang utama, termasuk kompleks PQRST, tetap terjaga setelah proses kompresi dan dekompresi, terutama pada durasi 5 detik pertama.

Pada Gambar 4.29, ditampilkan hasil deteksi puncak R (R-peak) dari masing-masing sinyal hasil dekompresi. Setiap metode menunjukkan keberhasilan dalam mempertahankan lokasi puncak R yang cukup konsisten jika dibandingkan dengan sinyal original, menandakan bahwa metode kompresi tidak mengganggu informasi penting yang berkaitan dengan perhitungan detak jantung.

Selanjutnya, Gambar 4.30 membandingkan hasil perhitungan *Heart Rate* (HR) berdasarkan sinyal asli dan hasil dekompresi. Warna ungu merepresentasikan nilai HR dari sinyal original, sementara warna lainnya menunjukkan hasil HR dari masing-masing metode kompresi. Ketika warna lain tidak terlihat (tersembunyi di balik garis ungu), hal tersebut mengindikasikan bahwa hasil perhitungan HR identik atau sangat mendekati sinyal asli. Ini memperkuat bahwa metode kompresi yang digunakan memiliki distorsi minimal terhadap nilai HR yang dihasilkan.

4.2.3 Analisis Pengujian Kompresi Data ECG

Berdasarkan hasil pengujian empat metode kompresi data sinyal ECG, diperoleh bahwa metode DWT Level 3 dengan kuantisasi 8-bit dan ZLib (DWT3 + Q8 + ZLib) memberikan performa terbaik secara keseluruhan. Metode ini mampu menghasilkan ukuran data terkompresi sebesar 13.457 byte dari ukuran asli 74.994

byte, atau setara dengan compression ratio sebesar 5,57 kali (hanya 17,94% dari ukuran awal). Selain itu, kualitas sinyal hasil rekonstruksi tetap sangat baik, ditunjukkan dengan nilai PRD (Percentage Root-mean-square Difference) sebesar 4,59% dan korelasi (CC) yang sangat tinggi, mencapai 0,9990, yang berarti sinyal hasil dekompresi nyaris identik dengan sinyal asli.

Metode LZMA dengan kuantisasi 8-bit (LZMA + Q8) juga menunjukkan hasil yang cukup baik, dengan ukuran hasil kompresi sebesar 18.744 byte atau compression ratio sebesar 4 kali. Meski demikian, nilai PRD meningkat menjadi 8,06%, namun korelasi tetap tinggi di angka 0,9968, menunjukkan kualitas sinyal masih cukup terjaga. Metode ini lebih cocok untuk situasi yang tidak terlalu kritis terhadap kecepatan kompresi, namun tetap memerlukan efisiensi penyimpanan.

Selanjutnya, metode DCT dengan kuantisasi 8-bit dan ZLIB (DCT + Q8 + ZLIB) menunjukkan rasio kompresi sebesar 4,23 kali, namun kualitas sinyal menurun cukup signifikan, dengan nilai PRD mencapai 16,70% dan korelasi 0,9861. Hasil ini menunjukkan bahwa meskipun efisiensi kompresi cukup baik, metode ini kurang direkomendasikan untuk aplikasi medis yang memerlukan keakuratan tinggi, seperti pemantauan sinyal jantung.

Sementara itu, metode kuantisasi 8-bit dengan LZ4 (Quant + LZ4) menghasilkan ukuran kompresi terbesar di antara keempat metode, yaitu 29.796 byte, dengan *compression ratio* sebesar 2,52 kali. Namun, keunggulan utama metode ini adalah kecepatan proses kompresi dan dekompresi yang sangat tinggi, sehingga cocok diterapkan pada sistem *real-time*. Kualitas sinyal hasil rekonstruksi tetap cukup baik dengan PRD 8,06% dan korelasi 0,9968.

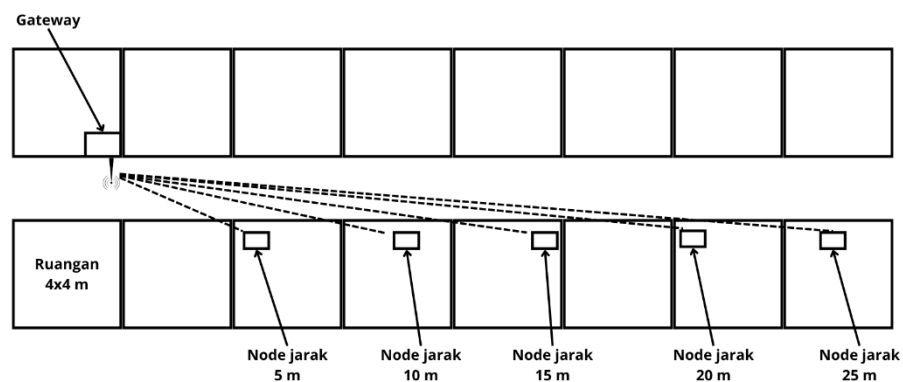
Secara keseluruhan, metode DWT3 + Q8 +ZLib merupakan solusi paling optimal untuk kompresi sinyal ECG yang membutuhkan efisiensi ruang penyimpanan dan transmisi tanpa mengorbankan kualitas sinyal. Metode lainnya dapat menjadi alternatif sesuai kebutuhan, misalnya LZMA untuk keseimbangan kompresi dan kualitas, atau LZ4 untuk aplikasi yang menuntut kecepatan tinggi.

4.3 Pengujian Pengiriman Data melalui LoRaWAN

Pengujian ini bertujuan untuk mengevaluasi kemampuan sistem dalam mengirimkan data sinyal ECG hasil kompresi menggunakan protokol LoRaWAN. Pengujian difokuskan pada aspek keandalan transmisi, integritas data setelah diterima, serta efektivitas metode kompresi yang digunakan untuk mengoptimalkan ukuran data yang dikirim.

4.3.1 Skenario Pengujian Pengiriman Data ECG

Pengujian dilakukan menggunakan sistem *prototipe* yang terdiri dari *node* pengirim berbasis ESP32 yang dilengkapi modul LoRa SX1276, serta *gateway* LoRaWAN yang terhubung ke *server* aplikasi penerima data. Data yang dikirim berupa hasil kompresi sinyal ECG menggunakan metode *Discrete Wavelet Transform* (DWT) level 3, kuantisasi 6-bit, dan kompresi *zlib*.



Gambar 4.31 Ilustrasi Tempat Pengujian Sistem

Langkah-langkah pengujian adalah sebagai berikut:

1. ESP32 mengambil data ECG secara *real-time*.
2. Terdapat 2 kondisi, yaitu dikirim langsung tanpa kompresi dan data dikompres terlebih dahulu lalu dikirim dengan LoRa.
3. Data dikompresi menggunakan metode DWT + Quant + *zlib*, kemudian dikonversi ke bentuk HEX String untuk dikirim melalui LoRaWAN.
4. *Gateway* menerima data LoRaWAN dan meneruskan ke *server* aplikasi.
5. Pengujian dilakukan pada jarak 5, 10, 15, 20, dan 25 antara *node* dan *gateway*, seperti pada gambar 4.32.

6. Parameter yang diamati adalah waktu transmisinya serta keberhasilannya.

4.3.2 Hasil Pengiriman Data Melalui LoRaWAN tanpa kompresi

Tabel 4.6 berisi catatan pengujian pengiriman data melalui LoRaWAN tanpa kompresi. Data yang dikirim memiliki variasi 5 jarak berbeda di mana setiap jarak dilakukan uji coba 5 kali. Terdapat status yang menandakan keberhasilan data dikirim secara lengkap atau ada yang hilang selama proses pengiriman.

Tabel 4.6 Tabel catatan waktu transmisi LoRaWAN tanpa kompresi

Jarak (meter)	Sesi	Payload (byte)	SNR (dB)	RSSI (dBm)	Lama Pengiriman (detik)	Status
5	1	15012	1.0	-118	792	Berhasil
	2	15030	1.2	-119	808	Berhasil
	3	14987	0.8	-121	784	Berhasil
	4	15100	1.1	-122	805	Berhasil
	5	14890	1.0	-120	775	Berhasil
10	1	15120	-3.5	-124	812	Berhasil
	2	15075	-2.0	-125	802	Berhasil
	3	14999	-4.0	-127	789	Berhasil
	4	14880	-5.0	-129	781	Gagal
	5	14950	-4.0	-130	-	Gagal
15	1	15085	-11.0	-134	810	Gagal
	2	15145	-14.5	-137	-	Gagal
	3	14990	-8.5	-132	796	Berhasil
	4	15035	-9.0	-135	803	Berhasil
	5	14850	-10.0	-136	787	Berhasil
20	1	15095	-17.5	-139	-	Gagal
	2	15120	-15.0	-137	-	Gagal
	3	15010	-13.0	-135	818	Berhasil
	4	14990	-14.5	-136	822	Berhasil
	5	14870	-16.0	-138	-	Gagal

25	1	15085	-18.0	-140	-	Gagal
	2	15100	-19.0	-141	-	Gagal
	3	15020	-17.0	-139	-	Gagal
	4	15130	-16.5	-138	830	Gagal
	5	14975	-15.5	-136	-	Gagal

4.3.3 Hasil Pengiriman Data Melalui LoRaWAN dengan kompresi

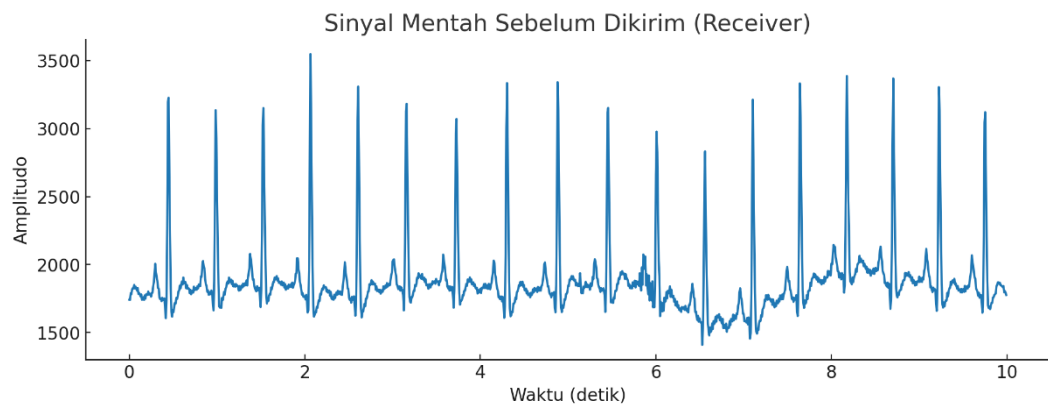
Tabel 4.7 berisi catatan pengujian pengiriman data melalui LoRaWAN dengan kompresi data terlebih dahulu. Kompresi data yang dipilih adalah gabungan DWT, kuantisasi, dan ZLib karena memiliki performa yang baik serta tetap dapat mempertahankan kualitas sinyal bahkan setelah sinyal tersebut di proses.

Tabel 4.7 Tabel catatan waktu transmisi LoRaWAN dengan kompresi

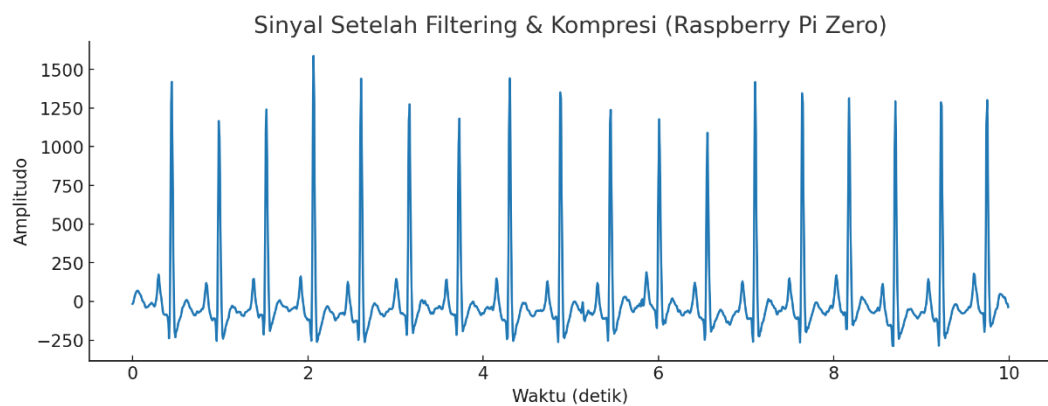
Jarak (meter)	Sesi	<i>Payload</i> (byte)	SNR	RSSI	Lama Pengiriman (detik)	Status
5	1	3612	1,5	-120	357	Berhasil
	2	4247	1,25	-125	420	Berhasil
	3	4210	1,5	-122	416	Berhasil
	4	4163	2	-120	411	Berhasil
	5	3895	1,25	-112	385	Berhasil
10	1	4331	-4	-114	428	Berhasil
	2	4117	-2	-118	407	Berhasil
	3	3885	-1	-120	384	Berhasil
	4	3762	-4	-123	372	Berhasil
	5	3947	-3	-125	390	Berhasil
15	1	3177	-16	-136	314	Gagal
	2	4123	-12	-130	-	Gagal
	3	4037	-7,5	-126	399	Berhasil
	4	3856	-8	-127	381	Berhasil
	5	4027	-14	-132	398	Berhasil
20	1	4012	-13.5	-134	392	Berhasil

	2	4160	-14.0	-136	-	Gagal
	3	4075	-15.0	-138	410	Berhasil
	4	3895	-16.5	-137	-	Gagal
	5	3990	-17.0	-139	404	Gagal
25	1	3920	-18.5	-141	-	Gagal
	2	4102	-17.0	-138	-	Gagal
	3	4077	-16.0	-137	-	Gagal
	4	3975	-15.5	-136	-	Gagal
	5	3888	-14.5	-135	417	Gagal

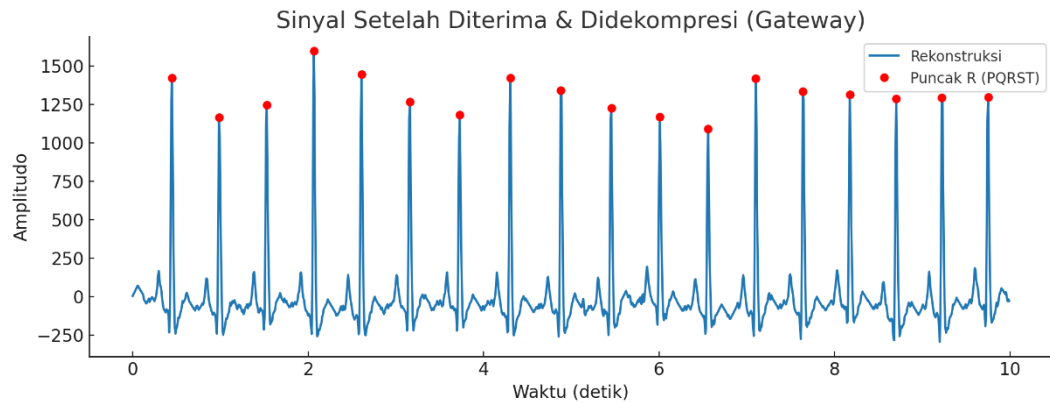
4.3.4 Validasi Hasil Data ECG dari Transmisi LoRaWAN



Gambar 4.32 Sinyal asli ECG dari ESP32



Gambar 4.33 Sinyal setelah difilter dan dikompres di Raspberry Pi Zero



Gambar 4.34 Sinyal ECG pada Raspberry Pi 5 yang diterima dari hasil transmisi LoRaWAN

Gambar 4.32 adalah sinyal asli yang direkam oleh sensor ECG AD8232 sebelum diproses. Terlihat bahwa sinyal masih mengandung *noise* dan *baseline wander*, sehingga belum ideal untuk analisis detak jantung (HR). Lalu pada gambar 4.32 sinyal difilter menggunakan bandpass filter (0.5–40 Hz) untuk menghilangkan noise frekuensi rendah dan tinggi, lalu dikompresi menggunakan kombinasi *Discrete Wavelet Transform* (DWT), *quantization*, dan *zlib*. Ini bertujuan memperkecil ukuran data agar efisien dikirim melalui LoRa. Pada gambar 4.33 setelah diterima di *gateway* (Raspberry Pi 5), sinyal didekompresi menggunakan metadata yang disisipkan oleh Pi Zero. Hasilnya cukup merekonstruksi bentuk gelombang ECG asli. Titik-titik merah menunjukkan deteksi puncak R (bagian dari kompleks QRS) yang merupakan bagian penting dari fitur PQRST dalam analisis jantung.

4.3.5 Analisis Hasil Transmisi Melalui LoRaWAN

Berdasarkan hasil pengujian transmisi data ECG melalui jaringan LoRaWAN pada jarak 5 hingga 25 meter, terdapat perbedaan yang signifikan antara metode tanpa kompresi dan dengan kompresi, khususnya dari sisi ukuran *payload* dan lama waktu transmisi.

Pada sistem tanpa kompresi, rata-rata ukuran *payload* berada di kisaran 14.800 hingga 15.100 *byte* per sesi. Akibat ukuran data yang besar ini, waktu yang dibutuhkan untuk menyelesaikan transmisi satu sesi mencapai 775 hingga 830 detik, dengan kecenderungan meningkat seiring bertambahnya jarak. Bahkan pada

jarak di atas 15 meter, beberapa sesi mengalami kegagalan akibat *timeout* atau kehilangan sebagian *chunk*. Hal ini menunjukkan bahwa semakin besar *payload* dan semakin jauh jarak transmisi, maka semakin tinggi risiko kegagalan pengiriman, khususnya pada jaringan LoRaWAN yang memiliki *bandwidth* rendah dan tingkat reliabilitas terbatas pada jarak jauh.

Sebaliknya, dengan menerapkan kompresi data (DWT + ZLib), ukuran *payload* berhasil ditekan secara signifikan menjadi hanya sekitar 3.600 hingga 4.300 *byte* per sesi. Efisiensi ini berdampak langsung pada penurunan waktu transmisi, yang tercatat hanya sekitar 357 hingga 428 detik untuk menyelesaikan satu sesi pengiriman. Dengan kata lain, metode kompresi memberikan penghematan waktu transmisi hingga lebih dari 50% dibandingkan metode tanpa kompresi.

Jika dilihat dari parameter SNR dan RSSI, keduanya menurun secara konsisten seiring bertambahnya jarak. Pada jarak 5 meter, nilai SNR masih berada di kisaran 1.0 hingga 1.5 dB, dan RSSI antara -118 hingga -120 dBm. Namun, pada jarak 15 meter ke atas, nilai SNR turun drastis hingga -14.5 hingga -19.0 dB, dan RSSI mencapai -140 hingga -141 dBm. Penurunan ini merupakan konsekuensi alami dari atenuasi sinyal dan interferensi, terutama dalam lingkungan *indoor* dengan banyak penghalang.

Dampak penurunan kualitas sinyal ini terlihat jelas dalam tingkat keberhasilan transmisi. Tanpa kompresi, tingkat keberhasilan menurun drastis pada jarak ≥ 15 meter, bahkan gagal sepenuhnya pada sebagian besar sesi di 25 meter. Sementara itu, sistem dengan kompresi masih mampu mempertahankan transmisi yang cukup baik hingga 20 meter, meskipun beberapa sesi mulai gagal di rentang tersebut. Pada 25 meter, baik metode kompresi maupun tanpa kompresi mengalami kegagalan pada sebagian besar sesi, menunjukkan bahwa pada kondisi sinyal dengan $\text{SNR} < -14$ dB dan $\text{RSSI} < -138$ dBm, transmisi LoRaWAN tidak lagi dapat diandalkan tanpa penguatan sinyal tambahan.

Dengan demikian, dapat disimpulkan bahwa kompresi data tidak hanya mempercepat waktu pengiriman, tetapi juga meningkatkan ketahanan sistem terhadap gangguan transmisi, khususnya pada jarak menengah. Namun, kualitas

sinyal (SNR dan RSSI) tetap menjadi faktor kunci keberhasilan, terutama pada jarak jauh. Kombinasi antara ukuran *payload* yang efisien dan sinyal dengan kualitas minimal (SNR > -10 dB dan RSSI > -135 dBm) adalah syarat optimal untuk menjaga keandalan sistem transmisi ECG berbasis LoRaWAN.

4.3.6 Analisa Lambatnya Proses Transmisi Data

Waktu transmisi yang cukup lama dalam sistem ini disebabkan oleh beberapa faktor teknis yang melekat pada desain sistem, pengaturan parameter komunikasi LoRa, dan protokol pengiriman data yang digunakan:

1. Spreading Factor Tinggi (SF10): Sistem menggunakan Spreading Factor 10 (`lora.setDataRate(SF10BW125)`), yang memang meningkatkan sensitivitas LoRa terhadap sinyal lemah dan memperpanjang jangkauan. Namun, SF tinggi memperlambat laju data secara drastis karena durasi satu simbol menjadi lebih panjang.
2. Bandwidth Sempit (125 kHz): Bandwidth sebesar 125 kHz adalah standar di LoRaWAN kelas A untuk kanal komunikasi. Namun, ini memberikan laju data maksimal sekitar 5 kbps (dengan SF10), sangat kecil dibanding protokol seperti WiFi atau BLE.
3. Transmit Power Rendah (`TxPower = 1`): Nilai `TxPower 1 dBm` (`lora.setTxPower(1)`) tergolong sangat rendah. Meskipun ini menghemat daya, sinyal menjadi cepat melemah terutama pada jarak jauh, meningkatkan risiko pengulangan transmisi atau kegagalan sesi.
4. *Chunking Data & Delay Manual*: Data ECG dikompresi lalu dikirim dalam potongan-potongan (*chunk*) 45 byte per paket dengan jeda 250 ms per *chunk* (`delay(250)`). Jika data hasil kompresi mencapai 4.000 byte, maka ada sekitar 90 *chunk*.
5. Handshake dengan Raspberry Pi: Protokol komunikasi antara ESP32 dan Raspberry Pi dilakukan melalui UART dengan proses handshake dan konfirmasi kesiapan sebelum perekaman dimulai, yang juga menambahkan waktu proses sebelum pengiriman.
6. Kualitas Antena dan Lingkungan Indoor: Antena yang digunakan berjenis 5 dBi cukup baik, namun kondisi lingkungan indoor dan interferensi elektromagnetik

dapat menyebabkan fluktuasi RSSI dan SNR, memperlambat atau bahkan menggagalkan transmisi saat sinyal terlalu lemah.

Dengan mempertimbangkan faktor-faktor di atas, waktu transmisi yang relatif lambat bukan semata karena keterbatasan jaringan, namun merupakan kompromi antara jangkauan, konsumsi daya, dan keandalan komunikasi, yang semuanya merupakan pertimbangan penting dalam sistem pemantauan kesehatan berbasis LoRa.

4.4 Estimasi Konsumsi Daya dan Waktu Operasi

Sistem menggunakan dua buah baterai Li-ion Molicel M35A 18650 berkapasitas 3500 mAh yang disusun paralel, menghasilkan total kapasitas 7000 mAh pada tegangan nominal 3.7 V. Kapasitas energi totalnya sekitar 25.9 Wh. Berdasarkan spesifikasi konsumsi daya gabungan dari ESP32, Raspberry Pi Zero, sensor ECG, dan modul komunikasi LoRa, total arus beban berada pada kisaran 500–600 mA. Dengan asumsi efisiensi konversi daya 85–90%, waktu operasi teoritis berkisar antara 11 hingga 14 jam, tergantung pola penggunaan dan intensitas transmisi data.

Tabel 4.8 Pengujian nilai tegangan baterai

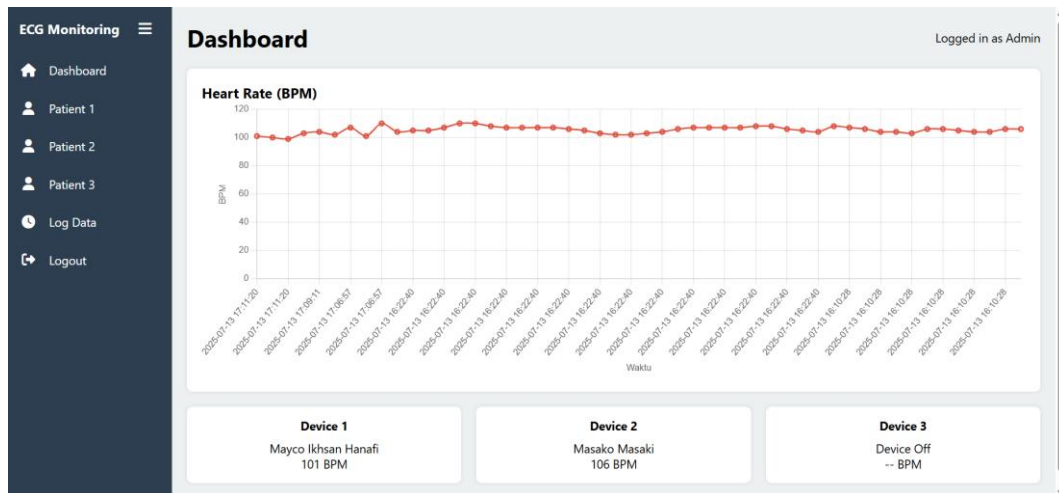
Waktu	Tegangan (V)
11:30	4.700
12:30	4.450
13:30	4.200
14:30	3.940
15:30	3.890
16:30	3.850
17:30	3.803
18:30	3.756
19:30	3.695
20:30	3.632
21:30	3.564
22:30	3.492
23:30	3.393
00:00	3.340
1.10	3.280

Hasil pengujian pada tabel 4.8 menunjukkan bahwa tegangan baterai turun dari 4.70 V (penuh) menjadi sekitar 3.28 V (tegangan *cut-off* saat alat mati) dalam waktu ± 13 jam. Laju penurunan tegangan rata-rata pada pengujian tersebut adalah sekitar 0.109 V/jam, dengan laju yang lebih lambat pada awal pemakaian dan meningkat mendekati batas *cut-off*. Indikator baterai internal sempat menunjukkan kapasitas $>50\%$ pada jam ke-12, yang mengindikasikan pembacaan indikator tidak sepenuhnya linear terhadap *state of charge*. Perbedaan antara estimasi teoritis dan hasil empiris dipengaruhi oleh fluktuasi beban, efisiensi konversi daya, serta karakteristik tegangan beban baterai Li-ion.

4.5 Hasil Implementasi Sistem

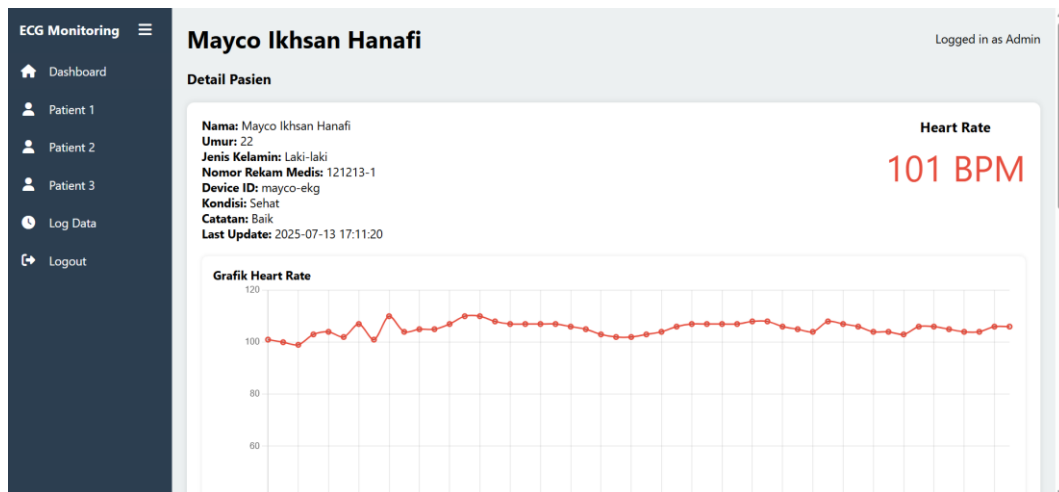
Sistem pemantauan ECG ini dikembangkan menggunakan Raspberry Pi Zero sebagai pengelola data utama dan PostgreSQL sebagai sistem penyimpanan basis data lokal. Data sinyal ECG mentah yang dikompresi dikirim dari perangkat melalui jaringan LoRaWAN menuju The Things Network (TTN), lalu diambil kembali menggunakan protokol MQTT oleh Raspberry Pi untuk diproses lebih lanjut. Setelah didekompresi dan difilter, sinyal digunakan untuk menghitung nilai *Heart Rate* (HR) dengan algoritma Pan-Tompkins, dan seluruh data disimpan ke dalam *database*.

Untuk memberikan kemudahan akses dan *monitoring* bagi tenaga medis, antarmuka pengguna dirancang dalam bentuk web lokal. Sistem ini mampu menampilkan data dari tiga pasien sekaligus.

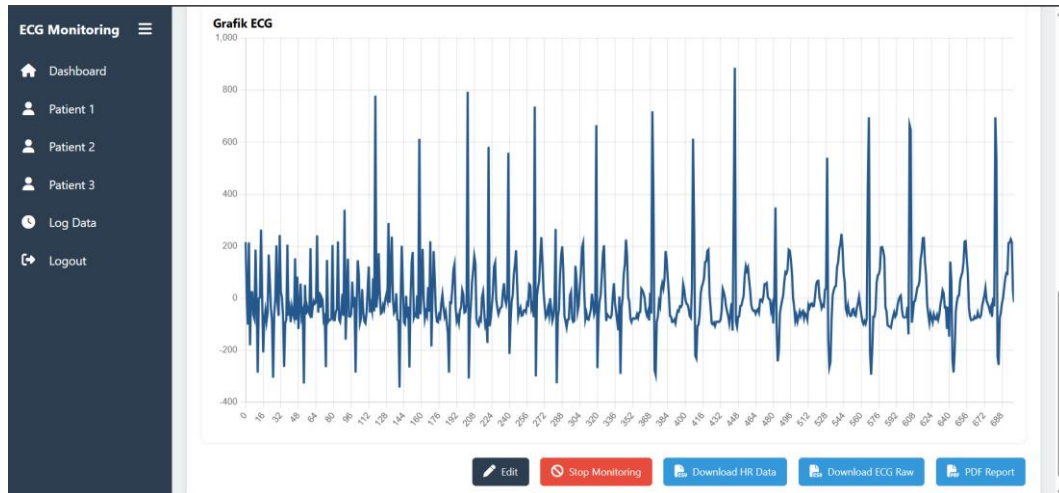


Gambar 4.35 Tampilan *dashboard* web pemantauan

Dashboard merupakan tampilan utama sistem seperti ditunjukkan pada Gambar 4.35. Setiap kotak mewakili satu perangkat ECG (*Device 1, 2, dan 3*), yang menampilkan nama pasien, nilai *heart rate* terkini (jika tersedia), dan tombol interaktif untuk menampilkan grafik riwayat HR langsung di bagian atas. Tampilan ini mempermudah tenaga medis dalam memantau kondisi pasien secara cepat dari satu layar.



(a)



(b)

Gambar 4.36 Tampilan Detail Pasien (a) bagian atas (b) bagian bawah

Saat pengguna memilih tab pasien (*Patient 1, 2, atau 3*) seperti pada gambar 4.36, sistem akan menampilkan detail lengkap seperti nama, umur, jenis kelamin, nomor rekam medis, kondisi, catatan, dan device ID. Tampilan ini juga menyertakan informasi *Last Update*, yaitu waktu terakhir sistem menerima data dari pasien melalui MQTT. Di bagian bawah ditampilkan grafik HR dan sinyal ECG mentah yang diperbarui berdasarkan data terbaru dari basis data.



Gambar 4.37 *Pop-up* peringatan HR tidak normal pasien

Pada gambar 4.37 untuk memastikan kondisi pasien dapat segera diketahui apabila terjadi gangguan pada irama jantung, sistem ini menyediakan fitur notifikasi *pop-up* otomatis ketika HR pasien berada di luar rentang normal seperti. Rentang normal ditentukan antara 40–120 BPM, sesuai referensi batas umum denyut jantung orang dewasa dalam kondisi normal.

Gambar 4.38 Form Input/Edit Pasien

Sistem menyediakan fitur pengisian data pasien yang dapat diakses melalui tombol edit seperti pada Gambar 4.38. Operator dapat mengisi informasi penting seperti nama, umur, jenis kelamin, nomor rekam medis, kondisi pasien, serta memilih salah satu dari 3 perangkat.

No	Waktu	Device	Jenis	Data
1	2025-07-13 21:09:20	mayco-ekg2	SUCCESS	All patient data (HR, ECG) successfully committed to database.
2	2025-07-13 21:09:20	mayco-ekg2	INFO	ECG raw data saved for patient 15.
3	2025-07-13 21:09:18	mayco-ekg2	INFO	All chunks received and assembled.
4	2025-07-13 21:09:18	mayco-ekg2	INFO	Heart Rate data saved for patient 15.
5	2025-07-13 21:09:18	mayco-ekg2	INFO	ECG signal processed successfully.
6	2025-07-13 21:09:18	mayco-ekg2	DATA_RECEIVED	MQTT message received for chunk 669/669
7	2025-07-13 21:09:18	mayco-ekg2	INFO	Patient 15 updated with latest HR: 106 BPM.
8	2025-07-13 21:09:14	mayco-ekg2	DATA_RECEIVED	MQTT message received for chunk 668/669
9	2025-07-13 21:09:09	mayco-ekg2	DATA_RECEIVED	MQTT message received for chunk 667/669
10	2025-07-13 21:09:05	mayco-ekg2	DATA_RECEIVED	MQTT message received for chunk 666/669

Gambar 4.39 Tampilan Log Data

Untuk keperluan analisis dan pencatatan, sistem juga menyediakan menu log data yang ditampilkan seperti pada Gambar 4.39. Pengguna dapat memilih rentang waktu dan jumlah data yang ingin ditampilkan. Log ini mencatat seluruh data yang masuk ke database, baik berupa HR maupun nilai sinyal ECG mentah.

Untuk menyimpan dan mengelola data pasien secara efisien, sistem menggunakan tiga tabel utama, yaitu *patients*, *heart_rate*, dan *ecg_data*. Data tersebut dapat dilihat pada tabel 4.9 hingga 4.11

Tabel 4.9 Struktur Tabel *patients*

No	Kolom	Tipe Data	Keterangan
1	id	SERIAL	Primary key
2	name	TEXT	Nama lengkap pasien
3	age	INTEGER	Umur pasien
4	gender	VARCHAR	Jenis kelamin pasien (Laki-laki/Perempuan)
5	mrn	VARCHAR	Nomor rekam medis pasien
6	condition	TEXT	Kondisi umum pasien
7	notes	TEXT	Catatan tambahan terkait pasien
8	device_id	VARCHAR	ID perangkat ECG yang digunakan
9	slot_number	INTEGER	Nomor slot (1, 2, atau 3) untuk menandai posisi pasien
10	heart_rate	INTEGER	Nilai HR terkini hasil analisis sinyal ECG
11	last_update	TIMESTAMP	Waktu terakhir data pasien diterima melalui MQTT

Tabel 4.10 Struktur Tabel *heart_rate*

No	Kolom	Tipe Data	Keterangan
1	id	SERIAL	Primary key
2	patient_id	INTEGER	Foreign key merujuk ke tabel <i>patients</i>
3	bpm	INTEGER	<i>Heart rate</i> (detak jantung per menit)
4	timestamp	TIMESTAMP	Waktu saat data HR diterima dan disimpan

Tabel 4.11 Struktur Tabel *ecg_data*

No	Kolom	Tipe Data	Keterangan
1	id	SERIAL	Primary key
2	patient_id	INTEGER	Foreign key merujuk ke tabel <i>patients</i>
3	bpm	INTEGER	<i>Heart rate</i> (detak jantung per menit)
4	timestamp	TIMESTAMP	Waktu saat data HR diterima dan disimpan

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem pemantauan sinyal jantung berbasis LoRaWAN dengan metode kompresi data, maka dapat disimpulkan beberapa hal sebagai berikut:

1. Sistem berhasil dibangun dan diuji secara fungsional. Sistem terdiri dari sensor ECG (AD8232), mikrokontroler ESP32, Raspberry Pi Zero untuk kompresi data, serta modul LoRa SX1276. Data ECG dikirim melalui UART ke Raspberry Pi Zero untuk dikompresi menggunakan metode *Discrete Wavelet Transform* (DWT) level 3, kuantisasi 8-bit, dan ZLib. Hasil kompresi kemudian dikirim kembali ke ESP32 dan dikirimkan ke *gateway* LoRaWAN menggunakan metode *chunking*.
2. Metode kompresi yang digunakan efektif dalam mengurangi ukuran *payload*. Dari pengujian, ukuran data ECG mentah sebesar 15.000 *byte* dapat dikompresi menjadi sekitar 3.600–4.300 *byte*. Ini menghasilkan rasio kompresi hingga 4:1 dengan nilai PRD < 0.2% dan korelasi sinyal CC > 0.999, menandakan bahwa kualitas sinyal hasil dekompresi sangat baik dan akurat untuk analisis detak jantung lebih lanjut.
3. Penggunaan kompresi terbukti meningkatkan efisiensi transmisi LoRaWAN. Dibandingkan dengan transmisi data tanpa kompresi, metode kompresi mengurangi waktu pengiriman hingga lebih dari 50%, dari rata-rata 790 detik menjadi sekitar 390 detik per sesi. Selain itu, tingkat keberhasilan pengiriman pada jarak 15 meter juga meningkat, karena *payload* yang lebih kecil lebih toleran terhadap keterbatasan *bandwidth* LoRa dan kemungkinan *packet loss*.
4. Sistem mampu mendukung transmisi multi-pasien melalui MQTT. Hasil integrasi dengan MQTT dan PostgreSQL berhasil menyimpan data ECG dan *Heart Rate* secara real-time untuk tiga node berbeda, membuktikan sistem dapat diskalakan untuk kebutuhan *monitoring* multi-pasien.

5.2 Saran

1. Optimalisasi pengiriman data LoRa di Raspberry Pi Zero supaya ESP32 cukup melakukan pengambilan data.
2. Uji stabilitas jangka panjang. Diperlukan uji coba sistem secara kontinu selama 1 hari atau lebih, untuk memastikan sistem dapat beroperasi tanpa *crash*, *buffer overflow*, atau kehilangan data.
3. Integrasi pengukuran HR secara langsung. Pengolahan *Heart Rate* menggunakan algoritma Pan-Tompkins sudah berhasil diterapkan. Sebaiknya hasil HR juga dikirim secara paralel bersama *payload* ECG agar lebih cepat dianalisis dan ditampilkan.

DAFTAR PUSTAKA

- [1] K. Malhi, S. C. Mukhopadhyay, J. Schnepfer, M. Haefke, and H. Ewald, "A Zigbee-Based Wearable Physiological Parameters Monitoring System," *IEEE Sensors Journal*, vol. 12, no. 3, pp. 423–430, Mar. 2012, doi: 10.1109/JSEN.2010.2091719
- [2] D. Banerjee, A. S. Bhaladhare, and S. Goswami, "A Review on ECG Signal Compression Techniques," *International Journal of Computer Applications*, vol. 110, no. 9, pp. 25–30, Jan. 2015, doi: 10.5120/20384-2644
- [3] H. Sümbül, "Design of A Novel IoT-Based Mobile ECG Data Transmission System Using ESP8266," *European Journal of Technique*, vol. 15, no. 1, pp. 15-20, 2025, doi: 10.36222/ejt.1496616
- [4] R. Muñiz, M. A. Olivares, M. E. Miquel, and P. Reñones, "A Smart Band for Automatic Supervision of Restrained Patients in Hospital Environment," *Sensors*, vol. 20, no. 18, pp. 5211, 2020, doi: 10.3390/s20185211
- [5] J. Pan and W. J. Tompkins, "A Real-Time QRS Detection Algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, Mar. 1985, doi: 10.1109/TBME.1985.325532
- [6] S. S. Kumar and P. Ramachandran, "Review on Compressive Sensing Algorithms for ECG Signal for IoT Based Deep Learning Framework," *Applied Sciences*, vol. 12, no. 16, p. 8368, 2022, doi: 10.3390/app12168368
- [7] K. Katzis, L. Berbakov, G. Gardašević, and O. Šveljo, "Breaking Barriers in Emerging Biomedical Applications," *Entropy*, vol. 24, no. 2, p. 226, 2022, doi: 10.3390/e24020226
- [8] D. Georgieva-Tsaneva et al., "Healthcare Monitoring Using an IoT-Based Cardio System," *IoT*, vol. 6, no. 1, 2025, doi: 10.3390/iot6010010
- [9] S. K. Das and J. Roy, "Portable ECG System Design Using the AD8232 Microchip and Open-Source Platform," *Proc. 6th Int. Electronic Conf. on Sensors and Applications*, MDPI, 2019, doi: 10.3390/ecsa-6-06584
- [10] T. D. Sahu et al., "A novel improved Pan-Tompkins method for QRS complex detection of the ECG signal," *Signal, Image and Video Processing*, 2025, doi: 10.1007/s11760-025-04587-6
- [11] M. H. A. Khan et al., "Complex-Pan-Tompkins-Wavelets: Cross-channel ECG beat detection and delineation," *Biomed. Signal Process. Control*, vol. 70, 2021, doi: 10.1016/j.bspc.2021.102450

- [12] Bhattarai, A., & Shrestha, B. (2023). IoT Based ECG Using AD8232 and ESP32. *Nepal Journal of Science and Technology*, 21(2), 106–110. <https://doi.org/10.3126/njst.v21i2.62361>
- [13] N. Shinde and S. Deshmukh, “The ECG Data Compression by Discrete Wavelet Transform and Huffman Encoding,” 2021 6th Int. Conf. on Communication and Electronics Systems (ICCES), doi: 10.1109/ICCITM53167.2021.9677704
- [14] R. Coifman et al., “Compression of ECG signals based on optimum quantization of discrete cosine transform coefficients and Golomb-Rice coding,” *IEEE EMBS*, 2003, doi: 10.1109/IEMBS.2003.1280460
- [15] M. Sayadi and M. Shamsollahi, “Compression of ECG Signals Based on DWT and Exploiting the Correlation between ECG Signal Samples,” *IJCSN*, vol. 7, no. 1, pp. 54–60, 2014, doi: 10.4236/ijcns.2014.71007
- [16] J. Martinez et al., “ECG Compression Using Ziv-Lempel Techniques,” *Computer Methods and Programs in Biomedicine*, vol. 48, no. 3, 1995, doi: 10.1006/cbmr.1995.1006
- [17] S. Pradhan et al., “A wavelet transform-based approach... using LZMA,” *ISA Transactions*, 2018, doi: 10.1016/j.isatra.2018.05.003
- [18] A. Gupta et al., “A novel ECG compression algorithm using PWM integrated quantization for low-power real-time monitoring,” *Scientific Reports*, 2024, doi: 10.1038/s41598-024-68022-5
- [19] Y. Wang et al., “Effective compression and classification of ECG arrhythmia by singular value decomposition,” *Biomedical Engineering Advances*, 2021, doi: 10.1016/j.bea.2021.100013
- [20] Y. Zhang et al., “ECG Data Compression Using Adaptive Fourier Decomposition,” *IEEE JBHI*, vol. 19, no. 3, 2014, doi: 10.1109/JBHI.2014.2357841
- [21] W. Chen et al., “ECG compression using vector–scalar quantization and context modeling arithmetic coding,” *Biomedical Signal Processing and Control*, 2012, doi: 10.1016/j.bspc.2012.04.003
- [22] M. Centenaro, L. Vangelista, A. Zanella, and M. Zorzi, “Long-range communications in unlicensed bands: The rising stars in the IoT and smart city scenarios,” *IEEE Wireless Communications*, vol. 23, no. 5, pp. 60–67, Oct. 2016, doi: 10.1109/MWC.2016.7721743

- [23] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, “A Study of LoRa: Long Range & Low Power Networks for the Internet of Things,” *Sensors*, vol. 16, no. 9, p. 1466, Sep. 2016, doi: 10.3390/s16091466
- [24] S. Andreev et al., “Low-Power Wide-Area Networks: Comparison of LoRaWAN and NB-IoT Performance,” *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9495–9511, Jun. 2022, doi: 10.1109/JIOT.2022.3176394
- [25] A. E. Shafique, “Performance Evaluation of LoRa and Sigfox LPWAN Technologies for IoT,” *American Journal of Research and Science*, vol. 38, no. 1, pp. 1–10, 2022, doi: 10.52132/Ajrsp.e.2022.38.1
- [26] D. A. Putra et al., “Performance Evaluation of LoRa 915 MHz for Health Monitoring with Adaptive Data Rate,” *2022 International Conference on Computer, Network, and Satellite (COMNETSAT)*, pp. 97–102, doi: 10.1109/COMNETSAT56033.2022.9994547
- [27] A. Augustin et al., “A Study of LoRa: Long Range & Low Power Networks for the Internet of Things,” *Sensors*, vol. 16, no. 9, p. 1466, 2016. DOI: 10.3390/s16091466
- [28] G. Krein et al., “Assessment of LoRa Signal Strength in a Building Environment,” Preprints, 2024. DOI: 10.20944/preprints202408.0663.v1

LAMPIRAN

SOURCE CODE

Source Code di ESP32

```
#include <Arduino.h>
#include <lorawan.h>
#include <Preferences.h>
#include <algorithm>

#define ECG_PIN 15
#define LO_PLUS_PIN 2
#define LO_MINUS_PIN 21
#define UART_BAUDRATE 115200
#define UART_TX_PIN 17
#define UART_RX_PIN 16
#define SAMPLE_RATE 125
#define DURATION_MS 300000UL //waktu dalam 1000/s
#define SAMPLE_INTERVAL_US (1000000UL / SAMPLE_RATE) // ms
#define LORA_NSS 5
#define LORA_RST 4
#define LORA_DIO0 27
#define LORA_DIO1 14
const char *devAddr = "260D51A8";
const char *nwksKey = "476985ACF7B07BBE3A27D61B8146907B";
const char *appSKey = "3EEB08AB4DCDFE84D3CE79A2BA8712B";
const sRFM_pins RFM_pins = {
    .CS = LORA_NSS,
    .RST = LORA_RST,
    .DIO0 = LORA_DIO0,
    .DIO1 = LORA_DIO1,
};
```

```

Preferences prefs;

static uint32_t framecounter = 0;

unsigned long startMillis = 0;

bool isRecording = false;

bool waitingForCompression = false;

bool piReady = false;

bool shouldStartNewRecording = false;

void setup() {
    Serial.begin(115200);

    Serial1.begin(UART_BAUDRATE, SERIAL_8N1, UART_RX_PIN,
UART_TX_PIN);

    pinMode(ECG_PIN, INPUT);
    pinMode(LO_PLUS_PIN, INPUT);
    pinMode(LO_MINUS_PIN, INPUT);
    delay(1000);
    if (!lora.init()) {
        Serial.println("LoRa Tidak Terdeteksi");
        while (1)
            ;
    }
    prefs.begin("lora", false);
    framecounter = prefs.getUInt("fcnt", 1);
    lora.setDeviceClass(CLASS_A);
    lora.setDataRate(SF10BW125);
    lora.setFramePortTx(1);
    lora.setChannel(MULTI);
    lora.setTxPower(1);
    lora.setNwkSKey(nwkSKey);
    lora.setAppSKey(appSKey);
    lora.setDevAddr(devAddr);
    lora.setFrameCounter(framecounter);

```

```

delay(1000);
lora.manualRX2();
Serial.println("[INFO] Menunggu Pi Zero siap...");
waitForPiReady();
Serial.println("[INFO] Pi Zero siap. Memulai pengukuran ECG selama 5
menit...");
startRecording();
}
void loop() {
    //Serial.printf("[STATE] recording=%d | waiting=%d | shouldStartNew=%d\n",
isRecording, waitingForCompression, shouldStartNewRecording); //debug
    static unsigned long lastSampleTime = 0;
    unsigned long currentTime = millis();
    checkPiMessages();
    if (shouldStartNewRecording && !isRecording && !waitingForCompression) {
        waitForPiReady();    // Tunggu Pi siap dulu
        startRecording();    // Baru mulai rekaman
        shouldStartNewRecording = false;
        return;
    }
    if (waitingForCompression) return;
    if (isRecording) {
        bool leadOff = digitalRead(LO_PLUS_PIN) || digitalRead(LO_MINUS_PIN);
        if (leadOff) {
            Serial.println("[WARN] Lead-off terdeteksi! Menghentikan rekaman.");
            isRecording = false;
            waitingForCompression = false;
            shouldStartNewRecording = true;
            return;
        }
    }
    unsigned long nowMicros = micros();

```

```

if (nowMicros - lastSampleTime >= SAMPLE_INTERVAL_US) {
    lastSampleTime = nowMicros;
    int sample = analogRead(ECG_PIN);
    Serial1.println(sample);
}
if (currentTime - startMillis >= DURATION_MS) {
    Serial1.println("[END]");
    Serial.println("[INFO] Akhir rekaman. Menunggu data kompresi...");
    isRecording = false;
    waitingForCompression = true;
}
}
}

void waitForPiReady() {
    unsigned long startTime = millis();
    bool requestSent = false;
    while (millis() - startTime < 30000) {
        // req tiap 2s
        if (!requestSent || (millis() - startTime) % 2000 < 50) {
            Serial1.println("REQUEST_READY");
            requestSent = true;
            Serial.println("[INFO] Meminta konfirmasi kesiapan Pi Zero...");
        }
        if (Serial1.available()) {
            String response = Serial1.readStringUntil('\n');
            response.trim();
            if (response == "READY") {
                Serial.println("[INFO] Pi Zero siap menerima data");
                piReady = true;
                return;
            }
        }
    }
}

```



```

    }
    delay(10);
}
Serial.println("[WARN] Timeout, asumsikan Pi Zero siap");
piReady = true;
}

void checkPiMessages() {
    static String message = "";
    while (Serial1.available()) {
        char c = Serial1.read();
        if (c == '\n') {
            processPiMessage(message);
            message = "";
        } else {
            message += c;
        }
    }
}

void processPiMessage(String msg) {
    msg.trim();
    if (msg == "READY") {
        piReady = true;
        Serial.println("[INFO] Pi Zero siap menerima data");
    } else if (msg.startsWith("[COMPRESSED];")) {
        int hexIdx = msg.indexOf("hex=");
        if (hexIdx >= 0) {
            String hexStr = msg.substring(hexIdx + 4);
            hexStr.trim();
            handleCompressedData(hexStr);
        }
    }
}

```

```

}

void startRecording() {
    if (!piReady) {
        Serial.println("[WARN] Pi Zero belum siap, menunda rekaman...");
        return;
    }
    Serial1.println("[START]");
    Serial1.println(SAMPLE_RATE);
    startMillis = millis();
    isRecording = true;
    waitingForCompression = false;
    Serial.println("[INFO] Rekaman dimulai...");
    Serial.printf("[DEBUG] Rekaman dimulai ulang @ %lu ms\n", millis());
}

void handleCompressedData(String hexStr) {
    // Konversi hex string ke byte array
    int byteLen = hexStr.length() / 2;

    byte *payload = (byte *)malloc(byteLen); // Pastikan payload dideklarasikan
    dengan benar

    for (int i = 0; i < byteLen; i++) {
        char buf[3] = { hexStr[i * 2], hexStr[i * 2 + 1], '\0' };
        payload[i] = (byte)strtol(buf, NULL, 16);
    }

    Serial.printf("[INFO] Data kompresi diterima (%d byte). Mengirim via
    LoRa...\n", byteLen);

    sendLoRaChunked(payload, byteLen);
    free(payload);
    waitingForCompression = false;
    delay(3000);
    piReady = false;
    shouldStartNewRecording = true;
}

```

```

}

void sendLoRaChunked(byte *data, int length) {
  Serial.println("[INFO] Mengirim data via LoRa (chunked)...");
  const int chunkSize = 45;
  byte session_id = (byte)(millis() & 0xFF); // atau random(0,255)
  int totalChunks = (length + chunkSize - 1) / chunkSize;
  if (totalChunks > 65535) {
    Serial.println("[ERROR] Total chunks terlalu banyak (>65535)!");
    return;
  }
  for (int i = 0; i < totalChunks; i++) {
    byte chunk[5 + chunkSize] = { 0 };
    chunk[0] = session_id;
    // Simpan chunk_id (i) ke 2 byte
    chunk[1] = (i >> 8) & 0xFF;
    chunk[2] = i & 0xFF;
    // Simpan totalChunks ke 2 byte
    chunk[3] = (totalChunks >> 8) & 0xFF;
    chunk[4] = totalChunks & 0xFF;
    int copyLen = min(chunkSize, length - i * chunkSize);
    memcpy(&chunk[5], &data[i * chunkSize], copyLen);
    lora.setFrameCounter(framecounter++);
    prefs.putUInt("fcnt", framecounter);
    lora.sendUplink((char *)chunk, 5 + copyLen, 1);
    lora.update();

    Serial.printf("[INFO] Chunk %d/%d dikirim (%d bytes)\n", i + 1, totalChunks,
5 + copyLen);
    delay(250);
  }
}

```

Source Code di Raspberry Pi 0

```

import serial

import numpy as np

import pywt

import zlib

import time

import json

from scipy.signal import butter, filtfilt

import binascii

SERIAL_PORT = "/dev/serial0"

BAUDRATE = 115200

TIMEOUT = 1

MAX_SAMPLES = 37500 # 5 menit * 125 Hz = 37500 sampel

COMPRESSION_LEVEL = 3

Q_BITS = 6

WAVELET = 'haar'

def bandpass_filter(data, lowcut=0.5, highcut=40.0, fs=125.0, order=4):

    nyq = 0.5 * fs

    b, a = butter(order, [lowcut/nyq, highcut/nyq], btype='band')

    return filtfilt(b, a, data)

def compress_dwt_per_level_zlib(signal, level=COMPRESSION_LEVEL,
q_bits=Q_BITS):

    coeffs = pywt.wavedec(signal, WAVELET, level=level)

    compressed_parts = []

    metadata = {'lengths': [], 'min_vals': [], 'q_steps': []}

```

```

for coeff in coeffs:

    min_val = np.min(coeff)

    max_val = np.max(coeff)

    q_step = (max_val - min_val) / (2 ** q_bits) if max_val != min_val else 1.0

    quantized = np.round((coeff - min_val) / q_step).astype(np.uint8)

    compressed = zlib.compress(quantized.tobytes())

    compressed_parts.append(compressed)

    metadata['lengths'].append(len(quantized))

    metadata['min_vals'].append(float(min_val))

    metadata['q_steps'].append(float(q_step))

return compressed_parts, metadata

def main():

    ser = serial.Serial(SERIAL_PORT, BAUDRATE, timeout=TIMEOUT)

    print("[Pi0] Inisialisasi sistem...")

    recording = False

    sample_rate = 125

    ecg_samples = []

    session_start = None

    try:

        while True:

            line = ser.readline().decode(errors='ignore').strip()

            if not line:

                continue

            # Handle permintaan READY dari ESP32

            if line == "REQUEST_READY":

                print("[Pi0] Menerima permintaan READY dari ESP32")

```

```

ser.write(b"READY\n")

ser.flush()

print("[Pi0] Mengirim konfirmasi READY")
elif line == "[START]":

    # Dapatkan sample rate dari baris berikutnya
    sr_line = ser.readline().decode().strip()

    try:

        sample_rate = int(sr_line)

    except:

        sample_rate = 125 # Fallback

    ecg_samples = []

    recording = True

    session_start = time.time()

    print(f"[Pi0] Mulai rekaman @ {sample_rate} Hz")

    print(f"[Pi0] Perkiraan durasi: 5 menit")

    # Tangani perintah [END] dari ESP32
    elif line == "[END]":

        if not recording:

            print("[WARN] Menerima [END] tanpa sesi aktif")

            continue

        recording = False

        duration = time.time() - session_start

        print(f"[Pi0] Akhir rekaman. Durasi: {duration:.1f} detik")

        print(f"[Pi0] Sampel terkumpul: {len(ecg_samples)}")

        # Proses dan kompresi data

        if len(ecg_samples) > 0:

```

```

        process_and_compress(ecg_samples, sample_rate, ser)
    else:
        print("[WARN] Tidak ada sampel yang terkumpul")
        ecg_samples = []
    elif recording:
        try:
            sample = int(line)
            ecg_samples.append(sample)
            if len(ecg_samples) % (5 * sample_rate) == 0:
                elapsed = time.time() - session_start
                print(f"[Pi0] Progress: {len(ecg_samples)} sampel
                ({elapsed:.1f}s)")
            except ValueError:
                print(f"[WARN] Data tidak valid: {line}")
                continue
        except KeyboardInterrupt:
            print("\n[Pi0] Dihentikan oleh pengguna")
    finally:
        ser.close()

def process_and_compress(samples, sample_rate, ser):
    start_time = time.time()
    ecg_array = np.array(samples, dtype=np.float32)
    filtered = bandpass_filter(ecg_array, fs=sample_rate)
    print("[Pi0] Memulai kompresi data...")
    try:
        # Kompresi per level

```

```

compressed_parts, metadata = compress_dwt_per_level_zlib(filtered)
compressed_data = b''.join(compressed_parts)

# Siapkan metadata
meta_list = []
for i in range(len(compressed_parts)):
    meta_list.append({
        "compressed_size": len(compressed_parts[i]),
        "length": metadata["lengths"][i],
        "min_val": metadata["min_vals"][i],
        "q_step": metadata["q_steps"][i]
    })

marker = b'###META###'
meta_json = json.dumps(meta_list).encode('utf-8')
full_payload = compressed_data + marker + meta_json
print(f"[DEBUG] Marker ditemukan di index {full_payload.find(marker)}")
print(f"[DEBUG] Full payload size: {len(full_payload)} bytes")

original_size = len(filtered) * 4 # float32
compressed_size = len(full_payload)
ratio = original_size / compressed_size if compressed_size > 0 else 0
duration = time.time() - start_time

print("\n=== STATISTIK KOMPRESI ===")
print(f"Jumlah Sampel    : {len(filtered)}")
print(f"Original Size      : {original_size} bytes")
print(f"Compressed Size     : {compressed_size} bytes")
print(f"Compression Ratio   : {ratio:.2f}x")
print(f"Durasi Kompresi    : {duration:.3f} detik")

```



```

        send_to_esp(full_payload, ser)

    except Exception as e:

        print(f"[ERROR] Gagal memproses data: {str(e)}")

def send_to_esp(data, ser):

    try:

        hex_str = binascii.hexlify(data).decode('utf-8').upper()

        message = f"[COMPRESSED];hex={hex_str}\n"

        ser.write(message.encode())

        ser.flush()

        print(f"[Pi0] Data terkompresi dikirim ( {len(data)} bytes)")

    except Exception as e:

        print(f"[ERROR] Gagal mengirim data ke ESP32: {str(e)}")

if __name__ == "__main__":

    main()

```

Source Code di Raspberry Pi 5

```

import paho.mqtt.client as mqtt

import zlib, json, base64, time

import numpy as np

import matplotlib.pyplot as plt

import pywt

from scipy.signal import butter, filtfilt, find_peaks

import psycopg2

app_id = "mayco"

```

```

api_key =
"NNSXS.SOU4CMDQXBO6744VNIWBQRKK5PMGQEY4ZL2SEA.MVNIB
PC6A47TPR254IFIB67IHXLK4Y2TS4LPYIUWSLUULU26775A"

mqtt_host = "au1.cloud.thethings.network"

mqtt_port = 1883

uplink_topics = [
    "v3/mayco@ttn/devices/mayco-ekg/up",
    "v3/mayco@ttn/devices/mayco-ekg2/up",
    "v3/mayco@ttn/devices/mayco-ekg3/up",
]

FS = 125

def log_to_database(device_id, log_type, log_value):
    """
    Mencatat peristiwa ke tabel log_data di database.

    :param device_id: ID perangkat yang terkait dengan log.

    :param log_type: Tipe log (misal: "INFO", "WARNING", "ERROR",
    "DATA_RECEIVED").

    :param log_value: Deskripsi atau nilai log.
    """

    try:
        conn = psycopg2.connect(
            dbname="ecg_monitoring",
            user="admin",
            password="admin123",
            host="localhost",
            port="5432"
        )

```

```

cur = conn.cursor()

now = time.strftime("%Y-%m-%d %H:%M:%S") # Waktu saat ini

cur.execute(

    "INSERT INTO log_data (timestamp, device_id, type, value) VALUES
    (%s, %s, %s, %s)",

    (now, device_id, log_type, log_value)

)

conn.commit() # Simpan perubahan ke database

cur.close()

conn.close()

# print(f'[DB LOG] Logged: {log_type} - {log_value} for {device_id}') #
Opsional: untuk debugging di konsol Python

except Exception as e:

    print(f'[DB LOG ERROR] Gagal menulis log ke database: {e}')

def bandpass_filter(signal, lowcut=0.5, highcut=40, fs=FS, order=4):

    nyq = 0.5 * fs

    b, a = butter(order, [lowcut / nyq, highcut / nyq], btype="band")

    return filtfilt(b, a, signal)

def pan_tompkins_like(signal, fs=FS):

    diff = np.diff(signal, prepend=signal[0])

    squared = diff ** 2

    integrated = np.convolve(squared, np.ones(int(0.15 * fs)) / (0.15 * fs),
mode="same")

    peaks, _ = find_peaks(integrated, distance=fs * 0.4,
prominence=np.max(integrated) * 0.35)

    rr = np.diff(peaks) / fs

    hr = round(60 / np.mean(rr)) if len(rr) > 0 else None

    return hr

```

```

def sliding_window_hr(signal, fs=FS, window_sec=6, step_sec=2):
    window_size = int(window_sec * fs)
    step_size = int(step_sec * fs)
    result = []
    for start in range(0, len(signal) - window_size + 1, step_size):
        segment = signal[start:start + window_size]
        hr = pan_tompkins_like(segment, fs)
        if hr:
            corrected_hr = round(hr * 0.937 + 5.249)
            result.append(corrected_hr)
    return result

def decompress_dwt_per_level_zlib(compressed_parts, metadata, level=3):
    coeffs = []
    for i, comp in enumerate(compressed_parts):
        quantized = np.frombuffer(zlib.decompress(comp), dtype=np.uint8)
        assert len(quantized) == metadata[i]["length"]
        dequantized = quantized.astype(np.float32) * metadata[i]["q_step"] +
        metadata[i]["min_val"]
        coeffs.append(dequantized)
    return pywt.waverec(coeffs, "haar")

chunk_store = {}

def on_connect(client, userdata, flags, reason_code, properties=None):
    print(f"[MQTT] Connected with result code {reason_code}")
    for topic in uplink_topics:
        client.subscribe(topic)

```

```

    print(f'[INFO] Subscribed to {topic}')

def on_message(client, userdata, msg):

    topic_parts = msg.topic.split("/")

    device_id = topic_parts[3]

    print(f'\n[Uplink] Data dari {device_id} diterima...')

    try:

        data = json.loads(msg.payload.decode())

        raw = base64.b64decode(data["uplink_message"]["frm_payload"])

        session_id = raw[0]

        chunk_index = (raw[1] << 8) | raw[2]

        total_chunks = (raw[3] << 8) | raw[4]

        payload = raw[5:]

        if session_id not in chunk_store:

            chunk_store[session_id] = {

                "chunks": {},

                "received": 0,

                "total": total_chunks,

                "timestamp": time.time()

            }

        store = chunk_store[session_id]

        if chunk_index not in store["chunks"]:

            store["chunks"][chunk_index] = payload

            store["received"] += 1

            print(f'[Chunk] ID={session_id} | Chunk {chunk_index+1}/{total_chunks} diterima')

```

```

if store["received"] == store["total"]:
    print(f"[INFO] Semua chunk lengkap. Memproses data...")
    try:
        full_data = b''.join(store["chunks"][i] for i in range(total_chunks))
        print(f"[DEBUG] Total gabungan byte: {len(full_data)}")
        process_payload(full_data, device_id)
        del chunk_store[session_id]
    except Exception as e:
        print(f"[ERROR] Kesalahan saat proses payload: {e}")
        log_to_database(device_id, "ERROR", f"Payload processing error: {e}")

        if session_id in chunk_store:
            del chunk_store[session_id]
    except Exception as e:
        print(f"[ERROR] Terjadi kesalahan saat memproses pesan: {e}")
        log_to_database(device_id, "ERROR", f"Message processing error: {e}")
log_to_database) ---
def process_payload(payload, device_id):
    sep_marker = b"###META###"
    sep_idx = payload.find(sep_marker)
    if sep_idx == -1:
        log_to_database(device_id, "ERROR", "Metadata separator not found in payload.")
        raise ValueError("Metadata separator tidak ditemukan!")
    compressed = payload[:sep_idx]
    metadata_json = payload[sep_idx + len(sep_marker):].decode()
    metadata = json.loads(metadata_json)

```

```

compressed_parts = []

offset = 0

for meta in metadata:

    part_len = meta["compressed_size"]

    compressed_parts.append(compressed[offset:offset + part_len])

    offset += part_len

try:

    ecg_signal = decompress_dwt_per_level_zlib(compressed_parts, metadata)

    filtered = bandpass_filter(ecg_signal)

    hr_values = sliding_window_hr(filtered)

    log_to_database(device_id, "INFO", "ECG signal processed successfully.")

except Exception as e:

    print(f"[ERROR] Gagal memproses sinyal ECG: {e}")

    log_to_database(device_id, "ERROR", f"Failed to process ECG signal: {e}")

    return # Hentikan eksekusi jika sinyal ECG gagal diproses

now = time.strftime("%Y-%m-%d %H:%M:%S")

try:

    conn = psycopg2.connect(

        dbname="ecg_monitoring",

        user="admin",

        password="admin123",

        host="localhost",

        port="5432"

    )

    cur = conn.cursor()

```

```

        cur.execute("SELECT id FROM patients WHERE device_id = %s",
(device_id,))

        result = cur.fetchone()

        if result:

            patient_id = result[0]

            for hr in hr_values:

                cur.execute("INSERT INTO heart_rate (patient_id, bpm, timestamp)
VALUES (%s, %s, %s)", (patient_id, hr, now))

                log_to_database(device_id, "INFO", f"Heart Rate data saved for patient
{patient_id}.")

                if hr_values:

                    cur.execute("UPDATE patients SET heart_rate = %s, last_update = %s
WHERE id = %s", (hr_values[-1], now, patient_id))

                    log_to_database(device_id, "INFO", f"Patient {patient_id} updated with
latest HR: {hr_values[-1]} BPM.")

                    for val in ecg_signal:

                        cur.execute("INSERT INTO ecg_data (patient_id, value, timestamp)
VALUES (%s, %s, %s)", (patient_id, float(val), now))

                        log_to_database(device_id, "INFO", f"ECG raw data saved for patient
{patient_id}.")

                        conn.commit()

                        print("[DB] Data HR & ECG berhasil disimpan ke database.")

                        log_to_database(device_id, "SUCCESS", "All patient data (HR, ECG)
successfully committed to database.")

                    else:

                        print(f"[DB WARNING] Device ID {device_id} tidak ditemukan di tabel
patients.")

                        log_to_database(device_id, "WARNING", f"Device ID {device_id} not
found in patients table. Data not saved.")

                cur.close()

            conn.close()

```



```

except Exception as e:
    print(f"[DB ERROR] {e}")
    log_to_database(device_id, "ERROR", f"Database operation failed: {e}")
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.username_pw_set(f"{app_id}@ttn", api_key)
client.on_connect = on_connect
client.on_message = on_message
client.connect(mqtt_host, mqtt_port)
client.loop_forever()

```

Source Code Web Lokal dan Dataset Kalibrasi

Untuk source web lokal dan dataset untuk kalibrasi dapat dilihat melalui tautan GitHub di bawah ini.

<https://github.com/maycoaja/ECG-Monitoring-LoRaWAN/>